

Technical & Product Specification

A complete build specification for a STEM & electronics e-commerce platform — storefront, checkout, payments and full-CRUD administration — on React and MySQL.

DOCUMENT	TechStar Store — Technical & Product Specification, v1.0
STATUS	Design complete — approved for implementation planning. No code written.
REFERENCE MODEL	bafredo.co.tz (BAFREDO Electronics Limited) — reverse-engineered feature parity target
STACK	React 18 + TypeScript (Vite) · Node.js 20 + Express · MySQL 8
SCOPE	Public storefront · Customer accounts · Checkout & mobile-money payments · Admin console with full CRUD on every entity
PREPARED FOR	TechStar Store

Contents

1. Introduction, Scope and Method

1.1 What this document is · 1.2 What was built vs. what was researched · 1.3 How the reference platform was analysed · 1.4 Assumptions and decisions register · 1.5 Legal and originality boundary · 1.6 Glossary

2. Reference Platform Analysis (bafredo.co.tz)

2.1 Runtime architecture · 2.2 Complete route inventory · 2.3 Complete API surface · 2.4 Client state model · 2.5 Catalogue shape and scale · 2.6 Third-party libraries · 2.7 Gaps and defects to fix in TechStar

3. Product Definition

3.1 Vision · 3.2 Personas · 3.3 Capability map · 3.4 User journeys · 3.5 Non-goals · 3.6 Success metrics

4. Technology Stack and Architecture

4.1 Stack decision · 4.2 System topology · 4.3 Request lifecycle · 4.4 Environments · 4.5 Repository layout

5. Brand and Design System

5.1 Brand definition · 5.2 Colour · 5.3 Typography · 5.4 Spacing, radius, elevation · 5.5 Iconography · 5.6 Grid and breakpoints · 5.7 Motion · 5.8 Component tokens

6. Information Architecture and Routing

6.1 Sitemap · 6.2 Public route table · 6.3 Admin route table · 6.4 Navigation model · 6.5 Route guards · 6.6 URL and slug rules

7. Storefront UI/UX Specification — screen by screen

7.1 Global chrome · 7.2 Home · 7.3 Catalogue · 7.4 Product detail · 7.5 Search · 7.6 Cart and wishlist · 7.7 Checkout · 7.8 Payment · 7.9 Order confirmation and history · 7.10 Account · 7.11 Content pages · 7.12 Blog · 7.13 Services · 7.14 Auth surfaces · 7.15 Errors and empty states

8. Component Catalogue

8.1 Primitives · 8.2 Composites · 8.3 Layout · 8.4 Admin components · 8.5 Props contracts

9. Data Model — MySQL

9.1 Modelling principles · 9.2 Entity–relationship overview · 9.3 Table-by-table specification · 9.4 Indexing strategy · 9.5 Referential integrity and delete semantics · 9.6 Full DDL

10. REST API Specification

10.1 Conventions · 10.2 Errors · 10.3 Auth endpoints · 10.4 Catalogue · 10.5 Cart, wishlist, orders · 10.6 Payments · 10.7 Content · 10.8 Admin · 10.9 Rate limits

11. Authentication, Authorisation and Security

11.1 Identity model · 11.2 Token strategy · 11.3 OAuth · 11.4 Email verification and password reset · 11.5 RBAC matrix · 11.6 Input validation · 11.7 Upload security · 11.8 OWASP checklist

12. Business Rules and Domain Logic

12.1 SKU scheme · 12.2 Product grouping and variants · 12.3 Pricing and discounts · 12.4 Stock and reservation · 12.5 Cart rules · 12.6 Shipping · 12.7 Order state machine · 12.8 Payment reconciliation · 12.9 Delegated (friend) payment · 12.10 Invoicing · 12.11 Returns

13. Payments in Tanzania

13.1 Landscape · 13.2 Manual Lipa Namba flow · 13.3 Aggregator integration · 13.4 Cash on delivery · 13.5 Webhooks and idempotency · 13.6 Reconciliation ledger

14. Admin Console Specification

14.1 Shell and navigation · 14.2 Dashboard · 14.3 Products · 14.4 Categories and subcategories · 14.5 Orders · 14.6 Invoices · 14.7 Customers · 14.8 Shipping methods · 14.9 Banners · 14.10 Services · 14.11 Partners · 14.12 Blog and comments · 14.13 Campaigns · 14.14 Searches · 14.15 Currency · 14.16 Settings · 14.17 Staff and roles · 14.18 Audit log · 14.19 Universal CRUD contract

15. Media Pipeline

15.1 Upload flow · 15.2 Derivative sizes · 15.3 Storage adapters · 15.4 Cleanup and orphans

16. Search, Filtering, Sorting, Pagination

16.1 Search engine choice · 16.2 Query grammar · 16.3 Facets · 16.4 Analytics of searches · 16.5 Pagination contracts

17. Messaging: Email, SMS, Realtime, Campaigns

17.1 Transactional email · 17.2 SMS providers · 17.3 Templates · 17.4 Campaign engine · 17.5 Realtime events · 17.6 Queues and scheduling

18. Analytics and Reporting

18.1 Dashboard metrics · 18.2 Aggregation queries · 18.3 Charts · 18.4 Exports

19. Frontend Architecture

19.1 Folder structure · 19.2 State management · 19.3 Data fetching · 19.4 Forms and validation · 19.5 Error boundaries · 19.6 Code splitting · 19.7 Accessibility · 19.8 Internationalisation

20. Backend Architecture

20.1 Folder structure · 20.2 Layering · 20.3 Transactions · 20.4 Error taxonomy · 20.5 Logging and tracing · 20.6 Caching · 20.7 Background jobs · 20.8 Configuration

21. Quality: Testing, Performance, SEO

21.1 Test pyramid · 21.2 Test data · 21.3 Performance budgets · 21.4 SEO plan · 21.5 Analytics instrumentation

22. Deployment and Operations

22.1 Topology · 22.2 CI/CD · 22.3 Migrations · 22.4 Backups and DR · 22.5 Monitoring · 22.6 Runbooks · 22.7 Local development on XAMPP

23. Delivery Plan

23.1 Phases · 23.2 Work breakdown · 23.3 Estimates · 23.4 Acceptance criteria · 23.5 Risk register

24. Appendices

A Full catalogue taxonomy · B Validation message catalogue · C UI copy deck · D Environment variables · E Complete MySQL DDL · F Seed data plan · G Order status reference · H Alternative PHP/Laravel mapping · I Reference screenshots

Introduction, Scope and Method

1.1 What this document is

This is the complete build specification for **TechStar Store**, an online shop selling STEM kits, educational electronics and electronic components, parts, tools and instruments. It defines the product, the user experience, the visual system, the database, the API, the business rules, the administration console and the operational plan in enough detail that an engineering team can implement the platform without needing to invent behaviour.

The brief was explicit: build a store that matches `bafredo.co.tz` feature-for-feature and screen-for-screen, rebranded as TechStar Store, on React and MySQL, with an administration area that can create, read, update and delete every object in the system. This document is written to that brief.

Every statement about the reference platform in Part Two is derived from direct observation of the live site — its JavaScript bundles, its rendered DOM, its stylesheets and its public API responses — not from assumption. Where the reference platform's behaviour could not be observed because it sits behind an authenticated wall, this document says so explicitly and states the design decision TechStar will take instead.

READING ORDER

Parts 1–3 are for everyone: they define what is being built and why. Parts 4–8 are for designers and frontend engineers. Parts 9–13 are for backend engineers. Part 14 is for whoever builds the admin console — it is the largest single chapter because the brief asks for total CRUD coverage. Parts 15–22 are cross-cutting. Part 23 is for whoever plans the work. The appendices are lookup tables you will return to during implementation.

1.2 What was built versus what was researched

No application code has been written. This document is the deliverable. What *was* produced alongside it is a research archive, stored next to this PDF, containing the raw evidence behind every claim in Part Two:

Artefact	Contents
<code>research/api/*.json</code>	Live responses captured from the reference platform's public API: category list, subcategory list, product samples, grouped-product samples, banners, services, partners, shipping methods, currency configuration, region and district lists, order statistics, and product counts.
<code>research/screenshots/*.png</code>	Full-page renders of the reference storefront at desktop (1440 px) and mobile (390 px) widths: home, catalogue, product detail, checkout, about, contact.
<code>research/ui/source-ui-label-inventory.txt</code>	Every user-facing string extracted from all 65 lazy-loaded JavaScript chunks — screen titles, field labels, button text, validation messages, empty-state copy, toast messages and confirmation prompts. This is the source for Appendices B and C.
<code>research/ui/source-home-dom-outline.txt</code>	A structural outline of the fully rendered home page DOM (2,732 nodes), used to derive the exact composition of the header, mega-menu, product card, off-canvas panels and footer.

1.3 How the reference platform was analysed

The reference site is a client-rendered single-page application. Its served HTML is an empty shell, so ordinary page fetching reveals nothing. Five techniques were used in combination.

1.3.1 Bundle extraction

The application ships one entry bundle (`app.js`, 144 KB), one vendor bundle (2.0 MB) and 65 lazily loaded route chunks (2.4 MB total). The webpack chunk manifest inside the entry bundle maps numeric chunk IDs to content-hashed filenames; that manifest was parsed to enumerate and download all 65 chunks. Because production Vue builds preserve string literals verbatim, every route path, API path, form label, validation message and toast string survives minification intact.

1.3.2 Route and store recovery

Vue Router route records and Pinia store definitions were located by pattern in the entry bundle and extracted with brace-matched slicing, yielding 14 complete store definitions with their state shapes and action bodies. Because each action contains its own HTTP call, the store bodies double as an API contract.

1.3.3 API surface reconstruction

All calls flow through one helper of the form `request(path, method, body)`. Matching that call signature across the entry bundle and all 65 chunks recovered **90 distinct endpoint/verb pairs**, including path parameters and query strings, listed in full in §2.3.

1.3.4 Live API probing

The recovered public read endpoints were called directly. This produced real response payloads with real field names, types and nesting — the difference between guessing a schema and knowing one. Authenticated endpoints returned `401` and were not pursued; no credentials were used, no accounts were created, and no write operation was ever sent.

1.3.5 Headless rendering

Key pages were rendered in headless Chrome with a 20-second virtual-time budget, then captured both as full-page screenshots and as complete post-hydration DOM dumps. The DOM dumps give exact element nesting, utility classes, ARIA attributes and copy; the screenshots give proportion, rhythm and colour.

SCOPE OF OBSERVATION

The reference platform's administration console at `/dashboard/*` is behind authentication and was never entered. Everything Part 14 says about administration screens is reconstructed from the shipped client code for those screens — table column headers, form fields, modal titles, validation messages, bulk-action confirmation prompts and empty states are all recovered strings. What could *not* be recovered is server-side admin behaviour: exact permission rules, audit trails, and any admin-only response fields. Those are specified fresh in this document as TechStar's own design, and are marked as such.

1.4 Assumptions and decisions register

Where the brief left a choice open, a decision was made rather than a question asked, so that implementation is not blocked. Each is recorded here with its rationale and the alternative that was rejected. Any of these can be reversed; the cost of reversal is noted.

#	Decision	Rationale	Alternative & reversal cost
D1	Server runtime is Node.js 20 + Express 4 + TypeScript	The brief fixed React and MySQL but not the server. One language across client and server halves the hiring surface, allows shared validation schemas and shared TypeScript types, and matches the reference platform's own Node/Mongo lineage so behaviour maps cleanly.	PHP 8 + Laravel 11, which would suit the XAMPP install already present on the workstation. Reversal is moderate: the data model, API contract, business rules and all UI work in this document are runtime-neutral and carry over unchanged. Appendix H gives the Laravel mapping.
D2	Data access via Prisma ORM over MySQL 8	Declarative schema doubles as living documentation, migrations are generated and reviewable, and the generated client is fully typed. Raw SQL remains available for the reporting queries in Part 18 where ORM query builders are a poor fit.	Knex query builder or raw <code>mysql2</code> . Low reversal cost — Appendix E gives the complete hand-written DDL, so the schema is not Prisma-dependent.
D3	Build tool is Vite , not Next.js	The reference product is a client-rendered SPA with an admin console behind auth; server-side rendering buys little for the admin half and adds hosting complexity. SEO for the catalogue is addressed by prerendering the public routes (§21.4) rather than by adopting a full SSR framework.	Next.js App Router for SSR/ISR on catalogue pages. Moderate reversal cost, best decided before Phase 2. If organic search is the primary acquisition channel, revisit this early.
D4	Styling is Bootstrap 5.3 + SCSS tokens , consumed through React Bootstrap	The reference UI is built on Bootstrap 5.2 utilities. Keeping the same primitive vocabulary means the layouts in Part 7 translate one-to-one, and the grid, off-canvas, modal, toast, accordion and dropdown behaviours all come for free and already match.	Tailwind CSS. High reversal cost mid-project, negligible if decided now. Tailwind would require every layout in Part 7 to be re-expressed.
D5	Client state is Zustand + TanStack Query	Zustand is the closest React analogue to the reference platform's Pinia stores, so the 14 recovered store shapes port directly. TanStack Query handles server cache, retries and invalidation, which the reference platform hand-rolls (and gets wrong in places — see §2.7).	Redux Toolkit. Low reversal cost; more boilerplate for the same result.
D6	Product variants modelled as first-class rows in a variant table , not as separate product rows	The reference platform creates one product document per attribute combination and re-groups them at read time by name string. This is the root cause of several defects catalogued in §2.7. A proper parent/variant split fixes them without changing any screen.	Copy the reference model exactly. Not recommended; see §2.7 and §12.2 for the specific failures it causes.
D7	Images stored on local disk behind a storage adapter , with S3 and Cloudinary adapters provided	Keeps development and first production deployment simple and cheap, with no vendor lock-in. The adapter interface in §15.3 means switching to object storage later is a configuration change, not a rewrite.	Cloudinary from day one, as the reference platform uses. Low reversal cost either way, by design.
D8	Payments launch with manual mobile-money (Lipa Namba) plus cash on delivery , with an aggregator adapter ready	This is exactly what the reference platform does, and it requires no merchant integration to go live. The payment adapter interface in §13.3 means adding Selcom, ClickPesa, AzamPay or DPO later is an implementation of one interface plus a webhook route.	Aggregator-first. Higher upfront cost and a commercial onboarding dependency; better as Phase 4.

#	Decision	Rationale	Alternative & reversal cost
D9	Currency is TZS with zero fraction digits , configurable at runtime	Matches the reference platform's live currency configuration and Tanzanian retail convention. Money is stored as integer minor units regardless, so a future multi-currency change is additive.	Fixed hard-coded currency. Trivial difference; runtime configuration is specified because the admin screen for it exists in the brief.
D10	Interface language is English , with Swahili strings externalised from day one	The reference platform is English throughout except its mobile-money payment instructions, which are Swahili. Externalising strings costs almost nothing now and avoids a full retrofit when Swahili is added.	Hard-coded English. Low cost now, high cost later.

1.5 Legal and originality boundary

Building a competing store with the same feature set, the same information architecture and the same interaction patterns is ordinary commercial practice, and this document specifies exactly that. Three things must nevertheless be original to TechStar, and the specification is written on that basis:

- **Brand assets.** Name, logo, wordmark, tagline and colour identity are TechStar's own. Part 5 defines a distinct palette rather than reusing the reference platform's.
- **Editorial and legal copy.** Product descriptions, About, FAQ, Privacy Policy, Terms & Conditions and blog articles must be written for TechStar. This document specifies the *structure* of those pages — section headings, ordering, field lengths — and never supplies the reference platform's prose as body text. Legal pages in particular must be drafted or reviewed against Tanzanian consumer and data-protection law for TechStar's own operations.
- **Catalogue content.** Product names, photography and descriptions must be TechStar's own or supplied under licence by TechStar's distributors. What Appendix A reproduces is the *taxonomy* — a category and subcategory tree for electronic components — which is an industry-standard classification and the right starting shape for any electronics retailer, not proprietary content.
- **Registration identifiers.** The reference platform publishes its own TIN and certificate of incorporation numbers in its footer. TechStar's footer must carry TechStar's own registration details or none at all.

1.6 Glossary

Term	Meaning in this document
Reference platform	<code>bafredo.co.tz</code> , the site TechStar Store is specified to match.
Product group	A named product as a shopper perceives it — one card in a grid, one detail page. May contain one or many variants.
Variant	A specific purchasable configuration of a product group, with its own price, stock and SKU. A resistor product group might have variants for 220 Ω , 1 k Ω and 10 k Ω .
Attribute	A named dimension along which variants differ — Resistance, Voltage, Colour, Capacity.
SKU	Stock keeping unit. A structured 7-digit code, defined in §12.1.
Lipa Namba	Tanzanian mobile-money merchant till number. Customers push funds to it from M-Pesa, Tigo Pesa (Mixx by Yas), Airtel Money or Halopesa.
Delegated payment	Placing an order and nominating a third party — a parent, employer or sponsor — to pay it. Called "friend payment" on the reference platform.

Term	Meaning in this document
Minor units	The smallest indivisible unit of a currency. All money in the database is stored as an integer count of minor units to avoid floating-point error.
Soft delete	Marking a row as deleted with a timestamp rather than removing it, so history and foreign keys survive.
Facet	A filterable attribute in the catalogue — category, subcategory, price band, availability.

Reference Platform Analysis

This part records what the reference platform actually is. It is the factual base for every requirement that follows. Nothing here is inferred from marketing material or from what an electronics store "usually" does.

2.1 Runtime architecture

Layer	Observed
Client framework	Vue 3, compiled with webpack 5. Build name <code>bafredo-client</code> . Composition API with <code><script setup></code> (evident from the compiled <code>__name</code> component descriptors).
Routing	Vue Router 4, history mode, 25 public routes plus a nested <code>/dashboard</code> tree of 26 admin routes. All non-critical routes lazily loaded — 65 async chunks.
State	Pinia, 14 stores. Cart and wishlist mirrored into <code>localStorage</code> ; auth session persisted to <code>localStorage</code> under the key <code>authDetails</code> as <code>{isAuthenticated, user, token}</code> .
CSS	Bootstrap 5.2.2 loaded from jsDelivr CDN, plus 43 component-scoped stylesheets. Single typeface: Montserrat . Brand green <code>#246345</code> .
API	Same-origin, mounted at <code>/api</code> . Base URL resolved at runtime: production host → <code>https://www.bafredo.co.tz/api</code> ; otherwise <code>window.location.origin</code> . A development constant <code>https://dev.bafredo.co.tz/api</code> and a <code>http://localhost:2700</code> literal are both still present in the shipped production bundle.
Server data store	MongoDB with Mongoose. Proven by every API response carrying <code>_id</code> in 24-character hex ObjectId form plus a Mongoose <code>__v</code> version key, and by aggregation-shaped responses where <code>_id</code> holds a grouping key (a product name, or a month number) rather than a document identifier.
Realtime	Socket.IO client, connecting to the API origin with <code>auth: { _id }</code> . Used for admin notification of new orders.
Media	Cloudinary. Every image is stored once as WebP and served through three named transformations — see §2.5.
Analytics	Google Analytics 4 (<code>G-6MDZJR4J3C</code>), loaded in the HTML shell. An Ahrefs site-verification meta tag is present.
Auth	Bearer token in <code>localStorage</code> . Social sign-in via server-side redirect endpoints <code>/auth/google</code> and <code>/auth/facebook</code> , returning to a client route <code>/OAuthRedirecting</code> that extracts the token.

SECURITY OBSERVATIONS CARRIED FORWARD AS REQUIREMENTS

Three things in the reference implementation must *not* be reproduced. (1) The session token is kept in `localStorage`, which is readable by any injected script; TechStar uses an `httpOnly` refresh cookie plus a short-lived in-memory access token (§11.2). (2) Development hostnames and a localhost port are shipped in the production bundle, disclosing internal topology; TechStar's build strips non-production configuration (§22.2). (3) The realtime socket authenticates by passing a raw user identifier as `auth._id`, which is a guessable value rather than a proof of identity — anyone can claim to be any user; TechStar authenticates sockets with the access token (§17.5).

2.2 Complete route inventory

All 51 routes, recovered from the router definition. Route names are the platform's own.

2.2.1 Public routes

Path	Route name	Purpose
/	home	Landing page: banner carousel, three product carousels, partners strip.
/products/all	products	Full catalogue grid with sort and price filter.
/products/:id	productsByCategory	Catalogue grid filtered to one subcategory (<code>:id</code> is a subcategory identifier).
/product/:id	productView	Product detail. <code>:id</code> is the URL-encoded <i>product name</i> , not an identifier.
/search	search_product	Search results, driven by a <code>?search=</code> query parameter.
/checkout	checkout	Single-page checkout: identity, shipping method, shipping details, payment method, order summary.
/orders	orders_all	Signed-in customer's order history.
/orders/:id	single_order	Order detail with status, items, totals, cancel and reorder.
/invoice/:id	shared_invoice	Printable invoice, shareable by link.
/friend_payment/:id	friend_payment	Delegated payment page — a third party pays an existing order.
/profile	profile	Account settings: profile, shipping address, password.
/blogs	blogsView	Article index.
/blog/:id	single_blog	Article with comment thread.
/service/:id	single_service	Service detail page (e.g. PCB printing).
/about-us	aboutUs	Story, vision, mission, core values, interactive map.
/contact-us	contactUs	Contact details and enquiry form.
/faqs	faqs	Accordion of frequently asked questions.
/privacy-policy	privacyPolicy	Privacy policy.
/terms-and-conditions	termsAndCondition	Terms and conditions.
/emailVerification/:id	emailVerification	Consumes an email verification token.
/resetPassword/:id	reset password	Consumes a password reset token.
/OAuthRedirecting	oauth_redirecting	Landing point after social sign-in; extracts token, then redirects.
/404	404	Not-found page.

2.2.2 Admin routes (children of `/dashboard`)

Path	Route name	Purpose
/dashboard	dashboard	Analytics home: sales and orders charts, customer count, best sellers.
/dashboard/products	products	Product table with search, bulk delete, per-row discount.
/dashboard/create_product	create_product	Product creation with attribute matrix.
/dashboard/product/:id/edit	edit_product	Product editing.
/dashboard/categories	categories	Category table, create/edit modal, bulk delete.
/dashboard/subcategories	subcategories	Subcategory table, create/edit modal, bulk delete.

Path	Route name	Purpose
/dashboard/orders	orders	Order table with status filter tabs and search.
/dashboard/invoices	invoices	Invoice list.
/dashboard/create_invoice	create_invoice	Manual invoice builder with product search.
/dashboard/invoice/:id	invoice	Invoice detail, print, delete.
/dashboard/campaigns	campaigns	Campaign list with aggregate counters.
/dashboard/campaigns/sms/create	create_sms_campaign	SMS campaign composer.
/dashboard/campaigns/email/create	create_email_campaign	Email campaign composer.
/dashboard/campaigns/contacts	contacts	Contact list (customers as a marketing audience).
/dashboard/articles	articles	Blog article cards.
/dashboard/create_article	create_article	Article composer with rich text.
/dashboard/article/:id/edit	edit_article	Article editing.
/dashboard/banners	banners	Home banner table.
/dashboard/create_banner	create_banner	Banner builder with live preview.
/dashboard/banner/:id/edit	edit_banner	Banner editing, separate image-replacement modal.
/dashboard/services	services	Service cards.
/dashboard/create_service	create_service	Service composer.
/dashboard/service/:id/edit	edit_service	Service editing.
/dashboard/partners	partners	Partner logo cards.
/dashboard/create_partner	create_partner	Partner creation.
/dashboard/partner/:id/edit	edit_partner	Partner editing.
/dashboard/shipping_methods	shippingMethods	Shipping method table with create/edit modal.
/dashboard/searches	searches	Search-term analytics table.
/dashboard/currency	currency	Display currency and fraction digits.
/dashboard/settings	settings	Hero images for About, Privacy and Terms pages.

ADMIN SIDEBAR, VERBATIM

Dashboard · Products · Categories · Subcategories · Orders · Invoices · Campaigns · Blog · Shipping · Customers · Banners · Services · Searches · Partners · Currency Formatting · Settings · Sign out. Note the asymmetry: a *Customers* entry exists in the navigation but the only customer-facing admin route is the campaign contacts list. TechStar closes this gap with a full customer management module (§14.7).

2.3 Complete API surface

Ninety endpoint/verb pairs recovered from the bundles. **PUBLIC** confirmed reachable without a token; **TOKEN** requires a signed-in user; **ADMIN** requires an administrator. Authentication class is inferred from the calling screen except where marked *verified*.

2.3.1 Identity and account

Verb	Path	Access	Notes
POST	/register	PUBLIC	Returns 200 with <code>{token, _id, accountType, username, email, validEmail}</code> .
POST	/login	PUBLIC	Accepts email or phone in one field. Returns token plus profile including <code>phone, address, shippingDetails, profileImage</code> .
GET	/auth/google, /auth/facebook	PUBLIC	Server-side OAuth redirect entry points.
POST	/email/request	TOKEN	Re-send verification email.
POST	/email/verify	PUBLIC	Consume verification token.
POST	/password/reset	PUBLIC	Request a reset link.
PATCH	/password/reset/:token	PUBLIC	Set new password using token.
PATCH	/password	TOKEN	Change password while signed in.
GET / PATCH	/profile	TOKEN	Read and update profile including nested <code>shippingDetails</code> .
POST	/profile/image	TOKEN	Multipart upload, field name <code>images</code> .

2.3.2 Catalogue

Verb	Path	Access	Notes
GET	/categories?orderBy=	PUBLIC <i>verified</i>	17 categories returned.
POST DELETE	/categories	ADMIN	DELETE takes <code>{categories:[ids]}</code> for bulk removal.
PATCH DELETE	/category/:id	ADMIN	Single update / delete.
GET	/subcategories	PUBLIC <i>verified</i>	102 subcategories, each embedding <code>{category:{_id,name}}</code> and a denormalised <code>numberOfProducts</code> .
POST DELETE	/subcategories	ADMIN	Bulk delete returns <code>retainedSubcategories</code> .
PATCH DELETE	/subcategory/:id	ADMIN	
GET	/products	ADMIN	Flat, ungrouped product list for the admin table.
POST DELETE	/products	ADMIN	POST body is an <i>array</i> — one element per variant. DELETE takes <code>{products:[ids]}</code> .
GET DELETE	/product/:id	mixed	GET public, DELETE admin.
GET	/product/:id/related	PUBLIC	Related products, same subcategory.
GET PATCH	/product_grouped?id=	mixed	GET returns all variants sharing a name. PATCH updates the group.

Verb	Path	Access	Notes
GET	/products/grouped	PUBLIC verified	Aggregated catalogue feed. Query parameters <code>skip</code> , <code>limit</code> , <code>orderBy=orders</code> , <code>query=random</code> , <code>subcategory</code> .
GET	/products/grouped/count	PUBLIC verified	Returned <code>617</code> .
PATCH	/discounts	ADMIN	Percentage plus start and end date.
GET	/search?query=&orderBy=&limit=	PUBLIC verified	Returns grouped-product records.
POST	/searches	PUBLIC	Records the search term for analytics.
GET	/searches?skip=&limit=, /search_count	ADMIN verified 401	Search analytics.
GET	/best_selling? region=&skip=&limit=	ADMIN	Best sellers, filterable by delivery region.
GET POST	/product_reviews/:productId	mixed	Read public, write requires a token.
DELETE	/review/:id	TOKEN	

2.3.3 Commerce

Verb	Path	Access	Notes
POST	/carts	TOKEN	Server-side cart persistence: <code>{cartItems:[{_id, quantity}]}</code> .
POST	/wishlists	TOKEN	<code>{wishlistItems:[{_id}]}</code> .
GET POST	/orders	TOKEN	POST body: <code>{username, phone, products, shippingMethod, paymentMethod, paymentNumber, shippingDetails, delegatedPayerEmail}</code> . Response may carry a <code>url</code> the client redirects to.
GET PATCH DELETE	/order/:id	mixed	PATCH changes status (admin) — DELETE is the customer cancel action.
GET	/orders_stats?period=	PUBLIC verified	Periods observed: <code>last12months</code> , and by inference <code>last7days</code> , <code>last30days</code> . Returns <code>[{_id, totalCosts, numberOfOrders}]</code> .
POST	/payment/:orderId	PUBLIC	Used by the delegated-payment page, which a non-account holder opens by link.
GET POST	/invoices	ADMIN	
GET DELETE	/invoice/:id	mixed	GET is shareable by link.
GET POST	/shipping_methods	mixed	GET public <i>verified</i> ; POST admin.
PATCH DELETE	/shipping_method/:id	ADMIN	

Verb	Path	Access	Notes
GET POST	/currency	mixed	GET public <i>verified</i> .
GET	/locations/regions , /locations/districts?region=	PUBLIC <i>verified</i>	25 mainland regions; districts per region.
GET	/users/count	ADMIN <i>verified</i> 401	Dashboard customer counter.

2.3.4 Content, marketing and settings

Verb	Path	Access	Notes
GET POST	/banners	mixed	GET public <i>verified</i> . POST is multipart with fields <code>background</code> , <code>link</code> , <code>button</code> , <code>title</code> , <code>subtitle</code> , <code>images</code> .
GET PATCH DELETE	/banner/:id	mixed	
PATCH	/banners/:id/image	ADMIN	Image replaced separately from text.
GET POST	/services	mixed	GET public <i>verified</i> .
GET PATCH DELETE	/service/:id	mixed	
GET	/latest_services	PUBLIC <i>verified</i>	Feeds the Services dropdown.
GET POST	/partners	mixed	GET public <i>verified</i> .
GET PATCH DELETE	/partner/:id	mixed	
GET POST	/articles	mixed	GET returned an empty array — the blog is live but unpopulated.
GET PATCH DELETE	/article/:id	mixed	
GET	/latest_articles , /articles/comment_count	mixed	
GET POST	/comments , /comments/:articleId	mixed	Read public <i>verified</i> ; write requires a token.
POST	/contact_us	PUBLIC	Contact enquiry.
GET	/contacts	ADMIN	Marketing audience list.

Verb	Path	Access	Notes
GET POST DELETE	/campaigns/email , /campaigns/sms	ADMIN	Plus /campaigns/sms/:id for detail.
POST DELETE	/images	ADMIN	Generic image endpoint used by every uploader.
GET	/settings	ADMIN <i>verified 404 anonymously</i>	
GET PATCH	/settings/pages/image	ADMIN	Hero images for About, Privacy, Terms.

2.4 Client state model

The 14 Pinia stores, with the state each holds. TechStar's Zustand slices in §19.2 map one-to-one onto this list, with the corrections noted.

Store	State	Notable behaviour
auth	isAuthenticated, user, token, resetPasswordToken	Rehydrated from <code>localStorage.authDetails</code> at module load.
product	allProducts, allProductsByName, bestSellingProducts, newProducts, productSubcategories, randomProducts, relatedProducts, totalProductsCount, visitedProducts, fetchedSingleProduct, imagesOfFetchedSingleProduct	Eleven parallel collections of the same entity — a cache-coherency problem TechStar avoids by using a single query cache keyed by request.
cart	cart[] of {product: {_id,name,price,image}, quantity}	Entirely client-side until checkout; mirrored to <code>localStorage</code> . Getter <code>computedCartPrice</code> sums price × quantity.
wishlist	wishlist[], same shape as cart	Carries a quantity field it never uses meaningfully.
order	currentOrder, myOrders	<code>currentOrder</code> persisted to <code>localStorage</code> so checkout survives a refresh.
category / subcategory	categories[] / subcategories[]	Both short-circuit if the array is already non-empty — a permanent cache with no invalidation.
banner	allBanners, homeBanners, fetchedBanner	
service / partner	services/partners, latest*, fetched*, imagesOf*	Identical shape; strong candidates for one shared abstraction.
currency	currencyData	Drives <code>Intl.NumberFormat</code> money formatting everywhere.
location	regions, districts	Districts refetched on region change.
shippingMethods	allShippingMethods	
navigation	authFormState, nextRoute	Opens the login/register modal and remembers where to send the user afterwards.
notification	header, text, isSuccess	Single global toast.
socket	socketId, socket	

2.5 Catalogue shape and scale

Measure	Observed value
Categories	17
Subcategories	102
Product groups	617
Largest subcategory	Passive devices — 86 products
Empty subcategories	13 of 102 carry zero products but remain visible in the mega-menu with a "0" badge
Currency	TZS, <code>maximumFractionDigits: 0</code>
Price range observed	TZS 198 (0.001 µF ceramic capacitor) to TZS 850,000 (digital signal generator)
Shipping methods	3 — Savings (TZS 15,000), Cash on delivery (TZS 5,000), Office Pickup (TZS 0)
Delivery geography	25 mainland regions; Dar es Salaam resolves to 6 districts
Blog articles	0 — module shipped, unused
Product reviews	0 — module shipped, unused
Services	1 — "PCB Printing"

2.5.1 SKU scheme

The reference platform uses a strict positional SKU, and it is worth keeping because it makes physical stock control in a component store tractable.

```
Category      2 digits   e.g. "05"      Sensors & Conditioning Boards
Subcategory   4 digits   e.g. "0510"    = category "05" + sequence "10" (Voltage)
Product       7 digits   e.g. "0510003" = subcategory "0510" + sequence "003"

Verified examples
0111004  Hardware Dev Tools (01) > Raspberry Pi boards (0111) > item 004
0510003  Sensors (05) > Voltage (0510) > item 003
```

2.5.2 Media transformations

Each image is stored once and exposed at three sizes. Every product, banner, service and partner record carries the same triple.

Key	Transformation	Used by
<code>sm</code>	thumbnail, width 200	Cart and wishlist rows, gallery thumbnails, admin table rows
<code>md</code>	thumbnail, width 600	Product cards in grids and carousels
<code>lg</code>	original	Product detail hero, banner artwork

DO NOT COPY THIS DETAIL

The reference platform's `sm` and `lg` transformation URLs pin a stale version segment (`v1668076879`) while `md` omits it, so the three sizes are not guaranteed to be the same asset after a re-upload. It also applies a *face-detection* crop gravity (`g_face`) to photographs of circuit boards, which is meaningless here and can crop badly. TechStar's pipeline (§15.2) uses content-addressed derivative names and centre-weighted crops.

2.6 Third-party libraries in use

Library	Purpose	React equivalent adopted by TechStar
Bootstrap 5.2.2	Grid, components, utilities	Bootstrap 5.3 + React Bootstrap
Swiper	Banner and product carousels	Swiper React
PrimeVue DataTable	Admin tables, sorting, selection	TanStack Table + custom shell (\$8.4)
Chart.js	Dashboard analytics	Chart.js via react-chartjs-2
Quill	Rich text for products, articles, services	TipTap (\$8.4.6) — Quill's HTML output is harder to sanitise reliably
FilePond	Drag-and-drop image upload	FilePond React
Leaflet + OpenStreetMap	Map on About page	React-Leaflet
vue-multiselect	Searchable selects, tag entry	react-select
Vuelidate	Form validation	React Hook Form + Zod
Moment.js	Dates (290 KB, deprecated upstream)	date-fns — 20× smaller, tree-shakeable
Socket.IO client	Realtime admin notifications	Socket.IO client
FontAwesome	Icons	Lucide React + Bootstrap Icons (\$5.5)

2.7 Defects and gaps in the reference platform

Feature parity is the goal; bug parity is not. These are concrete faults observed in the reference implementation. Each is listed with the section where TechStar's corrected behaviour is specified. Fixing them costs little at design time and is expensive later.

#	Defect	Consequence	Fixed in
R1	Product detail URLs use the product <i>name</i> as the path segment	Renaming a product breaks every inbound link, every shared URL and every search-engine result. Names containing <code>/</code> , <code>%</code> , <code>#</code> or <code>&</code> require double-encoding; the code contains a manual <code>%25</code> unescape to work around this. Two products may not share a name, ever.	\$6.6
R2	Variants are separate product rows re-grouped by name at read time	Grouping is a string join. A trailing space or a case difference silently splits one product into two. Observed in live data: <code>"Raspberry Pi 4 4gb Model B Starter Kit "</code> carries both double spaces and trailing whitespace.	\$12.2
R3	<code>addToCart</code> computes an index, then pushes when the item is absent, then indexes with the stale value	The first add of any item stores quantity 1 but skips the increment branch; subsequent adds work. Quantity handling is inconsistent between first and later additions.	\$12.5

#	Defect	Consequence	Fixed in
R4	<code>visitProduct</code> deduplicates with <code>find(e => { e.name })</code> — a block body with no <code>return</code>	The predicate always returns <code>undefined</code> , so the cache never hits and every visit refetches and appends a duplicate entry, growing memory for the session's lifetime.	\$19.3
R5	Category and subcategory stores never invalidate	An administrator adding a category cannot see it in the storefront navigation without a hard reload.	\$19.3
R6	Cart holds a price copied at add-to-cart time	If price or stock changes while an item sits in the cart, the customer checks out against a stale price. There is no re-validation before order creation.	\$12.5
R7	No stock check between add-to-cart and order placement	Overselling. The product page shows available stock but nothing enforces it downstream.	\$12.4
R8	Success is asserted by exact status code (<code>if (201 !== status) throw</code>)	A <code>200</code> from a proxy, or a <code>204</code> , is treated as failure. Inconsistent across stores: services and partners expect <code>201</code> on <i>delete</i> , which no correct server would return.	\$10.1
R9	Session token in <code>localStorage</code> ; socket authenticated by raw user id	Token theft via any injected script; socket identity is spoofable by anyone who knows or guesses an id.	\$11.2, \$17.5
R10	Development hostnames and <code>localhost:2700</code> shipped in the production bundle	Internal topology disclosure.	\$22.2
R11	Empty subcategories rendered in the mega-menu with a "0" badge	Thirteen dead ends in primary navigation.	\$7.1.3
R12	Bulk category delete cascades to all subcategories <i>and all their products</i>	One mis-click destroys a large part of the catalogue. The confirmation prompt warns, but the action is irreversible and unaudited.	\$9.5, \$14.19
R13	No customer management screen despite a "Customers" sidebar entry	Administrators cannot view, search, suspend or assist an individual customer.	\$14.7
R14	Two identical footer columns both headed "COMPANY INFORMATION"	Visible on every page. Cosmetic, but it is on every page.	\$7.1.5
R15	Typos in shipped UI: "No products to diplay", "Sarturday", "Opps!"	Erodes trust at exactly the moments the user is already unsure.	Appendix C
R16	A product with price 0 is purchasable ("R-DRONE NAKED — TZS 0" is live)	Free orders. No validation prevents a zero or negative price.	\$12.3

#	Defect	Consequence	Fixed in
R17	No pagination on the full catalogue grid — <code>limit=10000</code>	A single response carrying the entire catalogue. Fine at 617 products, fails at 6,000.	\$16.5
R18	Order cancellation is <code>DELETE /order/:id</code>	Cancellation should be a state transition that preserves the record, not a deletion. Financial and audit history is lost.	\$12.7

Product Definition

3.1 Vision

TechStar Store is the place a Tanzanian student, hobbyist, technician or engineer goes to find the part they need, understand what it does, and get it delivered or collected — without leaving the country's payment and logistics reality. It sells two adjacent things: **STEM** — kits, robots, drones, starter bundles, teaching aids — and **electronics** — components, modules, boards, instruments, tools and consumables.

Those two halves have different buyers with different needs, and the design must serve both:

- A **STEM buyer** browses. They arrive without a part number, respond to imagery and bundles, and need to be told what a thing is for. They are served by the home page carousels, the category tree, rich descriptions and "what's in the box" content.
- An **electronics buyer** searches. They arrive knowing "74HC14" or "10 kΩ 1/4 W" and want the fastest path from search box to cart, with accurate stock and a clear price. They are served by search quality, SKU structure, variant selection and stock accuracy.

3.2 Personas

Persona	Goal	What the platform must get right
Amina University student, Dar es Salaam	Buy an Arduino starter kit for a coursework project, on a budget, this week.	Browsable categories; clear kit contents; low-cost shipping options; mobile money without a bank card; a mobile experience that works on a mid-range Android phone over 3G.
Joseph Electronics technician, Mwanza	Restock twenty different components in one order and have them shipped upcountry.	Fast repeat search; accurate stock; a cart that holds many small-value lines; upcountry shipping with a predictable cost; an invoice he can show his employer.
Grace Secondary-school science teacher, Dodoma	Get a quotation for lab equipment that her school's procurement office will pay against.	Admin-issued invoices; delegated payment so the school pays; a shareable invoice link; clear terms.
Daniel Startup hardware engineer	Source a specific sensor module and understand its interface before committing.	Rich technical descriptions; multiple product photographs; related products; reviews; realistic lead-time signals.
Neema TechStar store manager	Keep the catalogue accurate, fulfil today's orders, and know what is selling.	Fast product entry including variants; an order queue with status; stock adjustment; a dashboard that answers "how are we doing" in one screen.
Frank TechStar marketing	Tell 400 past customers about a new Raspberry Pi bundle.	Contact list; SMS and email campaigns; scheduling; search-term analytics to know what people wanted and did not find.

3.3 Capability map

Everything the platform does, grouped. Each capability names the part of this document that specifies it. "Parity" means the reference platform has it; "New" means TechStar adds it to close a gap identified in §2.7.

Domain	Capabilities	Status	Spec
Catalogue	Category → subcategory → product hierarchy; mega-menu navigation	Parity	§7.1.3
	Product groups with attribute-based variants, each with own price, stock, SKU	Improved	§12.2
	Multi-image galleries with three derivative sizes	Parity	§15.2
	Related products; best sellers; new arrivals; randomised discovery rail	Parity	§7.2
Discovery	Keyword search with term logging and admin analytics	Parity	§16
	Sort by best selling, newest, price; min/max price filter	Parity	§7.3
	Faceted filtering by availability and price band; paginated grids	New	§16.3
Shopping	Cart and wishlist, persistent across sessions and devices when signed in	Improved	§12.5
	Guest checkout with name and phone only	Parity	§7.7
	Shipping method selection with per-method cost and rules	Parity	§12.6
	Server-side price and stock re-validation before order creation	New	§12.5
Payment	Mobile money by Lipa Namba with payment-number capture	Parity	§13.2
	Cash on delivery	Parity	§13.4
	Delegated payment — nominate a third party to pay by link	Parity	§12.9
	Aggregator adapter for automatic confirmation and webhooks	New	§13.3
Orders	Order history, detail, reorder, cancel	Improved	§7.9
	Explicit order state machine with an immutable event log	New	§12.7
	Printable invoices, admin-issued and shareable by link	Parity	§12.10
Accounts	Email/phone + password; Google and Facebook sign-in	Parity	§11.3
	Email verification; password reset; profile and avatar	Parity	§11.4
	Saved shipping address with region → district pickers	Parity	§7.10
Content	Home banners with per-banner colours, copy and link	Parity	§14.9
	Blog with comments; services pages; partner logos	Parity	§14.12
	About, FAQ, Privacy, Terms with admin-managed hero images	Parity	§14.16
Marketing	SMS and email campaigns, immediate or scheduled, all or selected contacts	Parity	§17.4
	Contact list with search	Parity	§14.13
Administration	Full create/read/update/delete on every entity in the system	Improved	§14
	Sales and orders analytics over 7-day, 30-day and 12-month windows	Parity	§18
	Customer management: search, detail, order history, suspend	New	§14.7
	Staff accounts with roles and granular permissions	New	§14.17
	Audit log of every administrative mutation	New	§14.18

3.4 Primary user journeys

J1 — Browse to purchase, guest, mobile money

```
Home
├ banner "BUY NOW" → Product detail
│   └ select variant (if any) → set quantity → ADD TO CART
│       └ cart off-canvas slides in, badge increments
│           └ VIEW CART → CHECKOUT
│               └ enter name + phone (no account required)
│                   └ choose shipping method → cost added to total
│                       └ if method requires it: shipping details form
│                           (receiver name, region → district, street, postal code)
│                               └ choose payment method: Lipa Namba | Cash on delivery
│                                   └ if Lipa Namba: enter paying phone number (255XXXXXXXXX)
│                                       └ PLACE ORDER
│                                           └ Order confirmation + payment instructions
│                                               └ customer pushes funds to Lipa Namba
│                                                   └ admin marks payment received
│                                                       └ SMS + email confirmation to customer
```

J2 — Search to purchase, returning customer

```
Header search "74HC14" → Search results (term logged for analytics)
├ Product detail → ADD TO CART → cart merges with server-side cart
│   └ CHECKOUT (identity, address and last shipping method prefilled)
│       └ PLACE ORDER → order appears in /orders immediately
```

J3 — Delegated payment (school or employer pays)

```
Customer builds order → CHECKOUT
├ enters a delegated payer email address
│   └ PLACE ORDER → order created with status AWAITING_PAYMENT
│       └ payer receives an email containing /friend_payment/:orderId
│           └ payer opens link (no account needed)
│               └ sees order summary + payment methods
│                   └ enters payment number → MAKE PAYMENT
│                       └ both customer and payer notified on confirmation
```

J4 — Administrator fulfils the day

```
Sign in → Dashboard (today's orders, revenue, low stock)
├ Orders → filter "Pending"
│   └ open order → verify payment reference → set status PAID
│       └ set status PROCESSING → pick and pack (stock decremented on PAID)
│           └ set status SHIPPED → customer notified by SMS
│               └ set status DELIVERED → order closes
```

J5 — Administrator adds a product with variants

```
Products → NEW PRODUCT
└─ choose subcategory (SKU prefix derived automatically)
└─ name, rich-text description, images (drag & drop, max 100)
└─ Add Attributes → define columns e.g. "Resistance"
    └─ add rows: 220Ω | 500 | 200      (value | price | stock)
                  1kΩ  | 500 | 350
                  10kΩ | 500 | 180
└─ Submit → one product group + three variants, SKUs auto-assigned
```

3.5 Non-goals for version 1

Stated so that scope stays honest. Each is a deliberate exclusion, not an oversight.

- **Multi-vendor marketplace.** TechStar is the single seller. No vendor onboarding, no split settlement.
- **Multi-warehouse inventory.** One stock pool. Location-aware stock is a Phase 5 candidate.
- **Multi-currency pricing.** Prices are held in TZS. The money column type (\$9.1) is chosen so that adding currencies later is additive, not a migration.
- **Card payments.** Mobile money and cash cover the market. Card acquiring can be added through the same payment adapter.
- **Native mobile applications.** The web experience is specified mobile-first and installable as a PWA (\$21.3).
- **Automated courier integration.** Shipping is a flat per-method cost, as on the reference platform. Courier APIs are a Phase 5 candidate.
- **Full Swahili translation.** Strings are externalised from day one (D10) but only English and the Swahili payment instructions ship in v1.

3.6 Success metrics

Metric	Target	Why it matters / how measured
Search success rate	> 80%	Share of searches that lead to a product view. Measured from the search log (\$16.4). Directly exposes catalogue gaps.
Cart-to-order conversion	> 35%	Checkout is the narrowest point of the funnel; the single-page design exists to protect this number.
Payment confirmation lag	< 30 min median	Time from order placement to payment marked received. The case for aggregator integration (\$13.3) is made or lost here.
Stock accuracy	> 98%	Orders not requiring a stock correction. Protects against the overselling defect R7.
Mobile Largest Contentful Paint	< 2.5 s on 3G	Most traffic will be mid-range Android on mobile data. Budgets in \$21.3.
Admin product entry time	< 3 min per product	Catalogue growth is bounded by data entry speed. The variant matrix (\$14.3.2) exists for this reason.
Order fulfilment lead time	< 24 h to SHIPPED	Measured from the order event log (\$12.7).

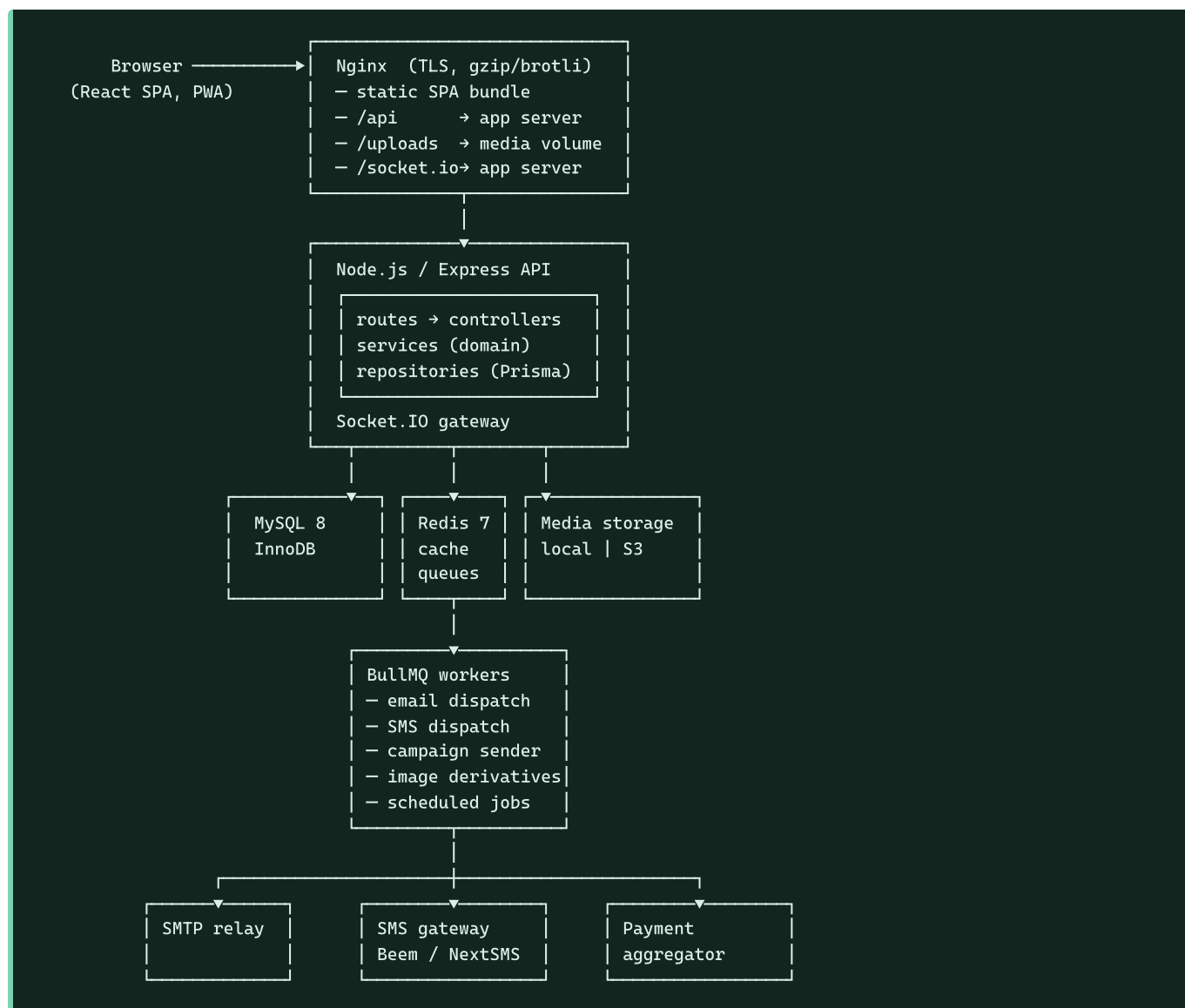
Technology Stack and Architecture

4.1 Stack decision

Concern	Choice	Why this and not the obvious alternative
UI framework	React 18.3 + TypeScript 5.4	Fixed by the brief. TypeScript is added because the domain has many similar-shaped objects (product / variant / cart item / order item) that are easy to confuse at runtime and trivial to distinguish at compile time.
Build	Vite 5	Sub-second cold start and instant HMR. Rollup production build with automatic route-level code splitting, matching the reference platform's 65-chunk strategy without hand configuration.
Routing	React Router 6.22 (data router)	Loaders and actions let route-level data requirements be declared next to the route, which keeps the 51-route map in Part 6 tractable.
Server state	TanStack Query 5	Directly fixes defects R4 and R5: request-keyed cache, automatic invalidation on mutation, stale-while-revalidate, retry and deduplication — all of which the reference platform hand-rolls incorrectly.
Client state	Zustand 4	Cart, wishlist, auth session, UI state. Small, unopinionated, and shaped almost identically to the Pinia stores being ported.
Styling	Bootstrap 5.3 + SCSS + React Bootstrap	Decision D4. Preserves the reference platform's layout vocabulary so Part 7 translates literally.
Forms	React Hook Form 7 + Zod 3	Zod schemas are shared with the server, so a field's rules are written once and enforced on both sides. This is the single highest-leverage choice in the stack.
Server runtime	Node.js 20 LTS + Express 4 + TypeScript	Decision D1.
Database	MySQL 8.0, InnoDB, utf8mb4_0900_ai_ci	Fixed by the brief. MySQL 8 is required, not 5.7: the reporting queries in Part 18 use window functions and CTEs, and the search design uses ngram-capable full-text indexes.
Data access	Prisma 5	Decision D2.
Cache / queue	Redis 7 + BullMQ	Campaign sending, email and SMS dispatch, image derivative generation and scheduled jobs must not run inside a request. Redis also backs rate limiting and session revocation.
Realtime	Socket.IO 4	Admin order notifications and live order status for the customer.
Object storage	Local disk via adapter; S3 and Cloudinary adapters supplied	Decision D7.

Concern	Choice	Why this and not the obvious alternative
Image processing	sharp	Generates the <code>sm / md / lg</code> WebP derivatives locally, removing the Cloudinary dependency and the transformation defects noted in §2.5.2.
Email	Nodemailer over SMTP, React Email templates	Provider-neutral. Works with Zoho, Google Workspace, SES or a local relay.
SMS	Adapter with Beem Africa and NextSMS implementations	Both are Tanzanian gateways with sender-ID registration; the adapter means neither is load-bearing.
Testing	Vitest, React Testing Library, Supertest, Playwright	Same runner for client and server unit tests; Playwright for the seven critical journeys in §21.1.
Process management	Docker Compose for development; PM2 or systemd behind Nginx in production	§22.1.

4.2 System topology



4.2.1 Why the API is same-origin

The API is mounted at `/api` on the same origin as the SPA, exactly as the reference platform does. This is not cosmetic: it lets the refresh token live in a `SameSite=Strict` httpOnly cookie without a cross-site cookie exemption, removes CORS preflight latency from every mutation, and keeps one TLS certificate. Development uses a Vite proxy so the same-origin assumption holds locally.

4.3 Request lifecycle

Every API request passes through the same ordered pipeline. The order matters and is normative.

#	Stage	Responsibility
1	<code>requestId</code>	Assign a ULID, attach to the async-local context, echo as <code>X-Request-Id</code> . Every log line and error response carries it.
2	<code>helmet</code>	Security headers including a strict Content-Security-Policy (§11.8).
3	<code>cors</code>	Same-origin by default; an explicit allowlist only when an alternate front end is deployed.
4	Body parsing	JSON limited to 256 KB; multipart handled by <code>multer</code> only on upload routes, with per-file and per-request caps (§11.7).
5	Rate limiting	Redis-backed sliding window. Tiers in §10.9.
6	Authentication	Verify the access token if present; populate <code>req.auth = {userId, role, permissions}</code> . Never rejects here — absence of a token is a valid state for public routes.
7	Route match	Express router.
8	Authorisation	Route-declared guard: <code>requireAuth()</code> , <code>requireRole()</code> , <code>requirePermission()</code> . Rejects with 401 or 403.
9	Validation	Zod parse of <code>params</code> , <code>query</code> and <code>body</code> . On failure emits the 422 shape in §10.2. The parsed, typed value replaces the raw input — controllers never see unvalidated data.
10	Controller	Translates HTTP to a service call and back. Contains no business logic.
11	Service	Domain logic, invariants, transaction boundaries. The only layer allowed to enforce a business rule.
12	Repository	Prisma access. No domain logic, no HTTP concepts.
13	Serialisation	Explicit response DTO. Never returns an ORM entity directly — this is what stops password hashes and internal flags leaking.
14	Audit	Successful mutations by staff append an audit record (§14.18) on the same transaction.
15	Error handler	Maps typed domain errors to status codes (§20.4); logs with request id; never leaks stack traces in production.

4.4 Environments

Environment	Purpose	Configuration
<code>local</code>	Developer workstation	Docker Compose brings up MySQL, Redis and MailHog. Media on local disk. SMS adapter is a console stub. Payments in sandbox. Seed data from Appendix F.
<code>test</code>	Automated suites	Ephemeral MySQL schema per run, migrated then truncated between tests. All outbound adapters are fakes. No network egress permitted.
<code>staging</code>	Pre-production verification	Production topology at smaller size. Anonymised copy of production data. Real SMS and email to an allowlist only. Payment aggregator in sandbox. <code>noindex</code> enforced at Nginx.

Environment	Purpose	Configuration
production	Live	Full topology per §22.1. Debug endpoints absent from the build, not merely disabled.

CONFIGURATION RULE

Configuration is read from the environment exactly once, at boot, through a Zod schema that fails fast on a missing or malformed variable (§20.8). No `process.env` access anywhere else in the codebase. This is what prevents the reference platform's failure mode of shipping a development hostname to production (R10).

4.5 Repository layout

A single repository with npm workspaces. One `git clone`, one `npm install`, one command to run everything.

techstar-store/	
├─ apps/	
│ └─ web/	React SPA – storefront and admin console
│ └─ public/	
│ └─ src/	
│ └─ vite.config.ts	
│ └─ api/	Express API + Socket.IO gateway + workers
│ └─ prisma/	
│ └─ schema.prisma	
│ └─ migrations/	
│ └─ seed.ts	
│ └─ src/	
├─ packages/	
│ └─ contracts/	Zod schemas + inferred TS types shared by web and api
│ └─ ui/	Design-system components and SCSS tokens
│ └─ config/	Shared eslint, tsconfig, prettier
├─ docker/	
│ └─ docker-compose.yml	
│ └─ nginx/	
├─ docs/	This specification and derived artefacts
├─ scripts/	Migration, backup, seed and ops helpers
└─ package.json	Workspace root

WHY PACKAGES/CONTRACTS IS THE KEYSTONE

Every request and response shape in Part 10 is defined once as a Zod schema in this package. The API validates against it; the React forms validate against it; TypeScript types are inferred from it on both sides. A field added to a form without a corresponding schema change fails to compile. This single package is what makes a specification this large stay true to the code six months after it is written.

Brand and Design System

The reference platform's visual system is deliberately plain: a dark utility bar, a dark header with a wide search field, a white category strip, generous white space, flat white product cards on a white page, and a single saturated green used for every affordance. It is not fashionable and it is exactly right for the job — a components catalogue is read, not admired, and the restraint keeps 617 product photographs from fighting each other.

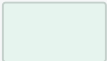
TechStar keeps that system's structure and proportion precisely, and replaces only the identity: name, wordmark and hue. Everything specified below is normative. Where a value is inherited from the reference platform because it works, that is stated.

5.1 Brand definition

Attribute	Definition
Name	TechStar Store . Written as two words, both capitalised, no hyphen, no camel case. "TechStar" alone is acceptable in body copy after first use.
Wordmark	TECHSTAR in Montserrat ExtraBold, letter-spacing <code>0.02em</code> , followed by Store in Montserrat Light at the same optical size, closed by a full stop in the primary green. Mirrors the reference lockup's weight contrast without copying its forms.
Mark	A four-point star formed from two overlapping chevrons, in primary green on white or white on primary green. Square-safe at 32 px for the favicon; readable at 16 px.
Tagline	<i>Build. Learn. Innovate.</i> — three words, full stops, set in the wordmark's light weight beneath it at roughly 28% of its cap height, in the accent amber.
Voice	Direct and technical. Say what a thing does and what it costs. No exclamation marks in interface copy. Error messages state the problem and the fix, never blame the user.
Clear space	Minimum clear space around the lockup equals the cap height of T on all sides.
Minimum size	Full lockup no smaller than 120 px wide. Below that, the mark alone.

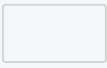
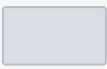
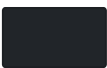
5.2 Colour

5.2.1 Brand palette

Swatch	Token	Hex	Use
	<code>--ts-primary</code>	<code>#0E7C5A</code>	Primary actions, active nav, price emphasis, badges, off-canvas headers, admin table headers. The single dominant colour.
	<code>--ts-primary-dark</code>	<code>#0A5C43</code>	Hover and active states of primary buttons; headings in the admin console.
	<code>--ts-primary-pale</code>	<code>#7BD1B0</code>	Icons and accents on dark backgrounds — the utility bar icons and the wordmark's inner strokes.
	<code>--ts-primary-tint</code>	<code>#E6F4EE</code>	Selected table rows, active filter chips, subtle success surfaces.
	<code>--ts-accent</code>	<code>#F2A007</code>	Sale and discount badges, "new" flags, the tagline, dashboard highlight bars. Used sparingly — never for a primary action.

Swatch	Token	Hex	Use
	<code>--ts-ink</code>	<code>#1F1F1F</code>	Main header bar and footer background.
	<code>--ts-ink-soft</code>	<code>#2E2E2E</code>	Top utility bar; footer legal strip.

5.2.2 Neutrals and semantics

Swatch	Token	Hex	Use
	<code>--ts-surface</code>	<code>#FFFFFF</code>	Page background, card background.
	<code>--ts-surface-alt</code>	<code>#F6F7F8</code>	Order summary panel, admin page background, zebra rows.
	<code>--ts-surface-sunken</code>	<code>#ECEFF1</code>	Product image wells, disabled inputs.
	<code>--ts-border</code>	<code>#D8DEE3</code>	Card and input borders, dividers.
	<code>--ts-text-muted</code>	<code>#8B98A2</code>	Subcategory eyebrow on cards, timestamps, placeholder text.
	<code>--ts-text-secondary</code>	<code>#4A5560</code>	Body copy in dense areas, table cells, form labels.
	<code>--ts-text</code>	<code>#212529</code>	Default body text and headings.
	<code>--ts-success</code>	<code>#1E8E4A</code>	Success toasts, in-stock, paid, delivered.
	<code>--ts-danger</code>	<code>#C0392B</code>	Errors, destructive actions, out of stock, cancelled.
	<code>--ts-warning</code>	<code>#D98200</code>	Pending payment, low stock, scheduled campaigns.
	<code>--ts-info</code>	<code>#1D6FA5</code>	Informational notices, processing state.

CONTRAST

Every foreground/background pair used for text must reach WCAG 2.1 AA — 4.5:1 for body, 3:1 for text at 18.66 px bold or 24 px regular. Two pairs need care and are specified explicitly: `--ts-text-muted` on white reaches 3.1:1, so it is permitted only for non-essential metadata at 14 px or larger, never for a label a user must read to act; and white on `--ts-accent` fails, so accent badges use `--ts-text` for their label, not white.

5.2.3 Status colour mapping

State	Token	Rendering
Order — Pending / Awaiting payment	<code>--ts-warning</code>	Amber pill, dark text
Order — Paid / Processing	<code>--ts-info</code>	Blue pill, white text
Order — Shipped	<code>--ts-primary</code>	Green pill, white text

State	Token	Rendering
Order — Delivered / Completed	<code>--ts-success</code>	Green pill, white text, check icon
Order — Cancelled / Refunded	<code>--ts-danger</code>	Red outline pill, red text
Stock — In stock (> 5)	<code>--ts-success</code>	Green dot + count
Stock — Low (1–5)	<code>--ts-warning</code>	Amber dot + "Only n left"
Stock — Out (0)	<code>--ts-danger</code>	Red dot + "Out of stock"; Add to cart disabled

5.3 Typography

Montserrat is the single family, inherited from the reference platform and kept: it is geometric, has a wide weight range, renders legibly at small sizes on Android, and is free. Self-host WOFF2 subsets rather than loading from a CDN — it removes a third-party request from the critical path and keeps the page working when the CDN is slow, which on Tanzanian mobile networks is often.

Role	Size	Weight	Line height	Notes
Display / banner title	40 / 28 px	800	1.05	Desktop / mobile. Colour set per banner record.
Page heading (h1)	28 / 22 px	700	1.2	Uppercase in section headers, as the reference does ("OUR PRODUCTS").
Section heading (h2)	22 / 18 px	700	1.25	
Card / panel heading (h3)	17 / 16 px	600	1.3	
Body	15 px	400	1.6	Maximum measure 72 characters.
Body small	13 px	400	1.5	Table cells, helper text.
Product card title	13 px	600	1.35	Uppercase, truncated to one line with ellipsis.
Product card eyebrow	11 px	400	1.3	Uppercase, muted — the subcategory name.
Price	15 px	600	1.3	Primary green. Discounted price shows the original struck through in muted grey immediately before it.
Button	14 px	600	1	Uppercase, letter-spacing <code>0.04em</code> .
Label	12 px	600	1.3	Uppercase, letter-spacing <code>0.05em</code> , secondary text colour.
Caption / meta	11 px	400	1.4	Muted.
Monospace	13 px	400	1.5	SKUs, order numbers, payment references. <code>ui-monospace</code> , "Cascadia Mono", <code>Consolas</code> , <code>monospace</code> .

FONT LOADING

Ship four weights only — 300, 400, 600, 800 — as Latin-subset WOFF2, roughly 22 KB each. Preload the 400 and 600 files. Use `font-display: swap` with a metric-matched fallback stack so the layout does not reflow when the webfont arrives.

5.4 Spacing, radius, elevation

A 4 px base scale. Bootstrap's spacer utilities are remapped onto it so `.p-3`, `.my-4` and friends produce these values.

Token	Value	Typical use
<code>--sp-1</code>	4 px	Icon-to-label gap
<code>--sp-2</code>	8 px	Inside chips and badges; between form label and control
<code>--sp-3</code>	12 px	Card inner padding at mobile; grid gutter at mobile
<code>--sp-4</code>	16 px	Default card padding; grid gutter
<code>--sp-5</code>	24 px	Between stacked form fields; panel padding
<code>--sp-6</code>	32 px	Between page sections at mobile
<code>--sp-7</code>	48 px	Between page sections at desktop
<code>--sp-8</code>	64 px	Above the footer; around empty states

Token	Value	Use
<code>--radius-sm</code>	3 px	Inputs, buttons, badges
<code>--radius-md</code>	6 px	Cards, panels, modals
<code>--radius-pill</code>	999 px	Status pills, cart count badge, the card's "ADD TO CART" reveal
<code>--shadow-1</code>	0 1px 2px rgba(16,24,32,.06)	Resting card
<code>--shadow-2</code>	0 4px 12px rgba(16,24,32,.10)	Hovered card, dropdown, popover
<code>--shadow-3</code>	0 12px 32px rgba(16,24,32,.16)	Modal, off-canvas panel

5.5 Iconography

Lucide React as the primary set, with Bootstrap Icons filling gaps. Both are outline sets at a 24 px grid with a 1.5 px stroke; both tree-shake to only the icons imported, which the reference platform's FontAwesome bundle does not. Brand logos for social links and payment networks come from Simple Icons.

Meaning	Icon	Placement
Cart	<code>shopping-cart</code>	Header, with a pill badge showing item count
Wishlist	<code>heart</code>	Header with count badge; product card action
Quick view	<code>eye</code>	Product card action, opens the preview modal
Search	<code>search</code>	Header field; mobile search trigger
Account	<code>user</code>	Header when signed in
Menu	<code>menu</code>	Mobile, opens the left off-canvas navigation
Chevrons	<code>chevron-left/right/down</code>	Carousel arrows, dropdown indicators, pagination

Meaning	Icon	Placement
Edit / delete	<code>square-pen</code> / <code>trash-2</code>	Admin row actions
Add	<code>plus</code>	Admin create buttons; quantity stepper
Success / error / warning	<code>check-circle</code> / <code>alert-circle</code> / <code>alert-triangle</code>	Toasts, field errors, confirmations
Print	<code>printer</code>	Invoice and order detail
WhatsApp	Simple Icons <code>whatsapp</code>	Fixed floating button, bottom-right, all storefront pages

5.6 Grid and breakpoints

Bootstrap's 12-column grid at its default breakpoints, with a container capped at 1200 px. Product grid column counts are specified per breakpoint because they are the single most visible responsive decision in the product.

Name	Min width	Container	Product grid	Layout behaviour
<code>xs</code>	0	fluid, 12 px gutter	2 columns	Off-canvas navigation; search collapses to an icon that opens a modal; utility bar hidden; footer columns stack.
<code>sm</code>	576 px	540 px	2 columns	As above with larger type.
<code>md</code>	768 px	720 px	3 columns	Utility bar appears; horizontal category strip replaces the off-canvas; card hover actions become available.
<code>lg</code>	992 px	960 px	4 columns	Inline header search field appears; checkout splits into form + sticky summary; product detail splits gallery / buy-box.
<code>xl</code>	1200 px	1140 px	6 columns	Full mega-menu with two panels; carousels show 6 cards.
<code>xxl</code>	1400 px	1140 px	6 columns	Container does not grow past 1140 px — product photography does not benefit from more width and the measure gets unreadable.

5.7 Motion

Interaction	Duration / easing	Behaviour
Button / link hover	120 ms ease-out	Background and border colour only. No movement.
Card hover	160 ms ease-out	Shadow 1 → 2; the "ADD TO CART" pill slides up from the card's lower edge, as on the reference platform. Desktop only — suppressed below <code>md</code> and on coarse pointers.
Off-canvas cart / wishlist / nav	250 ms cubic-bezier(.25,.8,.25,1)	Slide from the edge with a scrim fade. Focus moves into the panel; Esc closes; focus returns to the trigger.
Modal	200 ms ease-out	Fade plus 8 px upward translate.
Toast	200 ms in, auto-dismiss 5 s	Bottom-right. Errors do not auto-dismiss.
Carousel	400 ms ease-in-out	Banner autoplay 6 s, pauses on hover and on focus within.

Interaction	Duration / easing	Behaviour
Skeleton shimmer	1.4 s linear, infinite	Any content area whose load exceeds 300 ms.

REDUCED MOTION

Under `prefers-reduced-motion: reduce`, all transitions collapse to 0 ms, carousel autoplay is disabled, and the shimmer becomes a static tint. This is a hard requirement, not an enhancement.

5.8 Component tokens

Component	Specification
Button, primary	Height 40 px (48 px on touch), padding 0 20 px, radius <code>sm</code> , background <code>--ts-primary</code> , white uppercase 14/600 label. Hover <code>--ts-primary-dark</code> . Focus: 2 px outline in <code>--ts-primary</code> offset 2 px. Disabled: 45% opacity, no pointer events. Loading: label replaced by a 16 px spinner, width held constant so the layout does not jump.
Button, secondary	Same geometry; transparent background, 1 px <code>--ts-primary</code> border, <code>--ts-primary</code> label. Hover fills with <code>--ts-primary-tint</code> .
Button, danger	Same geometry; <code>--ts-danger</code> . Always paired with a confirmation dialog when the action is irreversible.
Input	Height 40 px, padding 0 12 px, 1 px <code>--ts-border</code> , radius <code>sm</code> , 15 px text. Focus: border <code>--ts-primary</code> plus a 3 px tint ring. Invalid: border <code>--ts-danger</code> , message beneath in 12 px <code>--ts-danger</code> preceded by an <code>alert-circle</code> icon, wired with <code>aria-describedby</code> .
Select	As input, with a <code>chevron-down</code> 16 px inset 12 px from the right. Searchable selects (subcategory, contacts) use <code>react-select</code> styled to match.
Product card	White, 1 px <code>--ts-border</code> , radius <code>md</code> , shadow 1. Image well is a 1:1 square with <code>object-fit: contain</code> on <code>--ts-surface-sunken</code> — components photographed on white must not be cropped. Body padding 12 px: eyebrow, title, price, then a two-icon action row (wishlist, quick view). Hover reveals the ADD TO CART pill.
Badge	Pill radius, 11/700 uppercase, padding 2 px 8 px. Discount badges sit top-left of the image well; "NEW" top-right.
Table (admin)	Header row <code>--ts-primary-dark</code> with white 12/600 uppercase labels. Rows 44 px, zebra on <code>--ts-surface-alt</code> , hover <code>--ts-primary-tint</code> . Selection checkbox in the first column; sticky header on scroll; horizontal scroll inside its own container below <code>lg</code> .
Off-canvas	Width 380 px desktop, 100% below <code>sm</code> . Header <code>--ts-primary</code> with a white uppercase title and a white close button. Body scrolls; a footer pinned to the bottom holds the subtotal and the primary action.
Modal	Centred, max-width 520 px (900 px for the product quick-view). Radius <code>md</code> , shadow 3. Header <code>--ts-primary</code> , white title, white close control.
Toast	Bottom-right, 320 px, white, shadow 2, a 4 px left rule in the semantic colour, bold title plus body, close button.
Empty state	Centred: 48 px muted icon, a 17/600 heading, one sentence of guidance, and one primary action. Every list, grid and table in the product has one, and each is specified in Part 7 or 14 — never a bare blank area.

Information Architecture and Routing

6.1 Sitemap

```
TechStar Store
├── / Home
├── /products Catalogue – all products
│   ├── /products/c/:categorySlug filtered to a category
│   └── /products/s/:subcategorySlug filtered to a subcategory
├── /product/:slug Product detail
├── /search?q= Search results
├── /cart Cart page (off-canvas is the primary surface)
├── /checkout Checkout
├── /order/confirmation/:orderNumber Post-order confirmation + payment instructions
├── /pay/:paymentToken Delegated payment (opened by the nominated payer)
├── /invoice/:invoiceToken Shareable printable invoice
├── /account (requires sign-in)
│   ├── /account/orders Order history
│   ├── /account/orders/:orderNumber Order detail
│   ├── /account/profile Profile
│   ├── /account/address Shipping address
│   └── /account/password Change password
├── /blog Article index
│   └── /blog/:slug Article + comments
├── /services Service index
│   └── /services/:slug Service detail
├── /about About – story, vision, mission, values, map
├── /contact Contact details + enquiry form
├── /faq FAQ accordion
├── /privacy-policy
├── /terms-and-conditions
├── /auth
│   ├── /auth/verify-email/:token
│   ├── /auth/reset-password/:token
│   └── /auth/oauth/callback
├── /admin (requires staff role)
│   ├── /admin Dashboard
│   ├── /admin/products · /new · /:id/edit
│   ├── /admin/categories
│   ├── /admin/subcategories
│   ├── /admin/orders · /:orderNumber
│   ├── /admin/invoices · /new · /:id
│   ├── /admin/customers · /:id
│   ├── /admin/shipping-methods
│   ├── /admin/banners · /new · /:id/edit
│   ├── /admin/services · /new · /:id/edit
│   ├── /admin/partners · /new · /:id/edit
│   ├── /admin/articles · /new · /:id/edit
│   ├── /admin/comments
│   ├── /admin/reviews
│   ├── /admin/campaigns · /sms/new · /email/new · /:id
│   ├── /admin/contacts
│   ├── /admin/searches
│   ├── /admin/currency
│   ├── /admin/settings
│   ├── /admin/staff · /:id
│   └── /admin/audit-log
├── /sitemap.xml
├── /robots.txt
├── /manifest.webmanifest
└── /404
```

DELIBERATE DIVERGENCES FROM THE REFERENCE ROUTE MAP

- `/product/:slug` replaces `/product/:name`. Fixes defect R1 — see §6.6.
- `/products/c/:slug` and `/products/s/:slug` replace the single ambiguous `/products/:id`, which could only ever filter by subcategory and gave no way to view a whole category.
- `/admin` replaces `/dashboard`. Plainer, and it frees "dashboard" to mean the analytics page rather than the whole console.
- `/pay/:paymentToken` replaces `/friend_payment/:orderId`. Handing a raw order identifier to a third party lets anyone with an order id open a payment page; a single-purpose signed token does not (§12.9).
- `/invoice/:invoiceToken` likewise replaces a raw-id URL: invoices contain a customer's name, address, phone and purchase history and must not be enumerable.
- `/cart` exists as a real page, not only an off-canvas panel. The panel remains the primary surface; the page is what a mobile user gets, and what a shared or bookmarked link resolves to.
- `/admin/comments`, `/admin/reviews`, `/admin/staff`, `/admin/audit-log` and `/admin/customers` are new, closing gaps R13 and R12.

6.2 Public route table

Path	Component	Guard	Data loaded on entry
<code>/</code>	HomePage	none	banners (type=home), random products (24), new arrivals (24), best sellers (24), partners, category tree
<code>/products</code>	CataloguePage	none	paginated products, category tree, price range for facets
<code>/products/c/:categorySlug</code>	CataloguePage	none	as above, scoped to the category and its subcategories
<code>/products/s/:subcategorySlug</code>	CataloguePage	none	as above, scoped to one subcategory
<code>/product/:slug</code>	ProductPage	none	product group with variants and images, related products, reviews, review summary
<code>/search</code>	SearchPage	none	search results for <code>?q=</code> ; fires the search-log write once per distinct term per session
<code>/cart</code>	CartPage	none	revalidates every line's price and stock against the server
<code>/checkout</code>	CheckoutPage	cart non-empty	shipping methods, regions, current profile and saved address if signed in; server-side cart revalidation
<code>/order/confirmation/:orderNumber</code>	OrderConfirmationPage	order token in session	order summary, payment instructions for the chosen method
<code>/pay/:paymentToken</code>	DelegatedPaymentPage	valid token	order summary (items, quantities, totals) and payment methods. No customer contact details are exposed to the payer.
<code>/invoice/:invoiceToken</code>	InvoicePage	valid token	invoice with line items; print stylesheet applied
<code>/account/*</code>	AccountLayout + tab	signed in	profile; orders list or detail per tab
<code>/blog</code> , <code>/blog/:slug</code>	BlogIndex, ArticlePage	none	articles paginated; article with comment thread and latest-articles rail
<code>/services</code> , <code>/services/:slug</code>	ServiceIndex, ServicePage	none	services; service with other-services rail
<code>/about</code>	AboutPage	none	site settings (hero image), map coordinates

Path	Component	Guard	Data loaded on entry
/contact	ContactPage	none	none; posts the enquiry form
/faq	FaqPage	none	FAQ entries
/privacy-policy , /terms-and-conditions	LegalPage	none	site settings (hero image); body is static content
/auth/verify-email/:token	VerifyEmailPage	none	posts the token, then redirects home with a toast
/auth/reset-password/:token	ResetPasswordPage	none	none
/auth/oauth/callback	OAuthCallbackPage	none	exchanges the one-time code, then redirects to the stored return route
*	NotFoundPage	none	none

6.3 Admin route table

All admin routes are children of an `AdminLayout` that supplies the sidebar, top bar and permission context. Every route declares the permission it needs; the sidebar renders only entries the signed-in staff member holds a permission for.

Path	Permission	Purpose
/admin	dashboard.view	Analytics home
/admin/products	product.read	Table, search, bulk delete, discount
/admin/products/new	product.create	Create with variant matrix
/admin/products/:id/edit	product.update	Edit, including variant matrix and images
/admin/categories	category.read	Table with create/edit modal
/admin/subcategories	category.read	Table with create/edit modal
/admin/orders	order.read	Table with status tabs
/admin/orders/:orderNumber	order.read	Detail, status transitions, payment reconciliation
/admin/invoices	invoice.read	List
/admin/invoices/new	invoice.create	Builder with product search
/admin/invoices/:id	invoice.read	Detail, print, share link, delete
/admin/customers	customer.read	Search, list
/admin/customers/:id	customer.read	Profile, orders, addresses, suspend
/admin/shipping-methods	shipping.read	Table with create/edit modal
/admin/banners · /new · /:id/edit	banner.*	Table, builder with live preview
/admin/services · /new · /:id/edit	service.*	Cards, composer
/admin/partners · /new · /:id/edit	partner.*	Cards, composer
/admin/articles · /new · /:id/edit	article.*	Cards, rich-text composer

Path	Permission	Purpose
/admin/comments	comment.moderate	Approve, reject, delete
/admin/reviews	review.moderate	Approve, reject, delete
/admin/campaigns	campaign.read	List with counters
/admin/campaigns/sms/new	campaign.create	SMS composer
/admin/campaigns/email/new	campaign.create	Email composer
/admin/campaigns/:id	campaign.read	Detail with delivery report
/admin/contacts	campaign.read	Audience list
/admin/searches	analytics.view	Search-term analytics
/admin/currency	settings.update	Display currency and fraction digits
/admin/settings	settings.update	Site settings and page hero images
/admin/staff · /:id	staff.manage	Staff accounts, roles, permissions
/admin/audit-log	audit.view	Immutable record of every admin mutation

6.4 Navigation model

6.4.1 Header, desktop (≥ md)

Three stacked bars, matching the reference platform's structure exactly. The lower two are sticky.

Bar	Height	Contents
Utility	32 px	Background <code>--ts-ink-soft</code> . Left: phone (<code>tel:</code> link), email (<code>mailto:</code>), location (links to <code>/about#map</code>), each with a pale-green icon at 11 px. Hidden below <code>md</code> . Not sticky.
Primary	72 px	Background <code>--ts-ink</code> . Left: logo lockup linking to <code>/</code> . Centre: search field — a flush input plus a wide primary-green "Search" button, appearing at <code>lg</code> and above. Right: wishlist icon with count badge, cart icon with count badge and running total, and either "Login Register" (primary-green button, opens the auth modal) or an account menu.
Category	48 px	White, 2 px bottom border. Left-aligned: PRODUCTS (link to <code>/products</code>), SHOP (mega-menu), SUPPORT (dropdown: Contact us, Blog, FAQ), SERVICES (dropdown listing each service). The active item carries a 2 px primary-green underline.

6.4.2 The SHOP mega-menu

A two-panel dropdown, 720 px wide, elevated with shadow 2.

- **Left panel, 7 of 12 columns.** All categories as a vertical list, each row showing the category name and a right-aligned chevron. Hovering or focusing a row loads that category's subcategories into the right panel; the row highlights with `--ts-primary-tint`. Clicking navigates to the category page.
- **Right panel, 5 of 12 columns,** separated by a left border. The hovered category's subcategories, each with a small primary-green pill showing its product count. Clicking navigates to the subcategory page.
- **Default state.** On open, the first category is pre-selected so the right panel is never empty.
- **Keyboard.** ↑/↓ move through categories; → enters the subcategory panel; ← returns; Enter navigates; Esc closes and returns focus to the SHOP trigger.

FIXING R11

Subcategories with zero products are **omitted** from the mega-menu rather than shown with a "0" badge. Categories whose every subcategory is empty are omitted too. They remain fully visible and editable in the admin console — this rule affects storefront navigation only, and is driven by the maintained `product_count` column (§9.3).

6.4.3 Header, mobile (< md)

- Single 64 px bar on `--ts-ink`: hamburger, compact logo mark, search icon, wishlist, cart.
- The hamburger opens a left off-canvas: Home · Products · Shop (an accordion of categories, each expanding to its subcategories) · Support · Services · then account links.
- The search icon opens a full-screen modal with an autofocused field and live suggestions.
- The utility bar is hidden; its phone and email move into the off-canvas footer.

6.4.4 Footer

Four columns on `--ts-ink`, stacking to one below `md`, then a legal strip on `--ts-ink-soft`.

Column	Contents
Contact	Legal entity name, P.O. box, city and country, two phone numbers, email address.
Company	Country of registration, registration agency, TIN, certificate of incorporation number, registered address. <i>TechStar's own identifiers only</i> (§1.5).
Policies & Support	Privacy policy · Terms & conditions · FAQ · About us · Contact us · Shipping information · Returns.
Follow us	Facebook, YouTube, LinkedIn, Instagram, X — 24 px brand-coloured marks, each with an accessible name.
Legal strip	© {year} TechStar Store. All rights reserved. centred.

This corrects R14: the reference platform's two footer columns are both headed "COMPANY INFORMATION". Here they are "Contact" and "Company".

6.4.5 Floating WhatsApp button

Fixed 56 px circular button, bottom-right, 24 px inset, WhatsApp green, above all content except modals and off-canvas scrim. Links to `https://wa.me/<number>` with a prefilled message. On a product page the message includes the product name and SKU so the agent has context. Accessible name: "Chat with TechStar on WhatsApp".

6.5 Route guards

Guard	Applies to	Behaviour when it fails
<code>requireAuth</code>	<code>/account/*</code>	Open the auth modal with the intended route stored as <code>nextRoute</code> ; on success, navigate there. Never a bare redirect to a login page — the reference platform's modal-based pattern is better and is kept.
<code>requireCart</code>	<code>/checkout</code>	Redirect to <code>/cart</code> with an informational toast, "Your cart is empty."
<code>requireStaff</code>	<code>/admin/*</code>	If unauthenticated, redirect to <code>/</code> and open the auth modal. If authenticated but not staff, render a 403 page with no navigation hints — do not confirm that <code>/admin</code> exists.
<code>requirePermission(p)</code>	each admin route	Render a 403 panel inside the admin shell, listing whom to ask for access.

Guard	Applies to	Behaviour when it fails
<code>requireToken</code>	<code>/pay/:t</code> , <code>/invoice/:t</code>	Render "This link is no longer valid" with a contact route. Never reveal whether the token was wrong, expired or already used.
<code>requireOrderAccess</code>	<code>/order/confirmation/:n</code>	Accessible to the order's owner, or to any session holding the one-time confirmation token issued at placement — this is what lets a guest see their own confirmation.

GUARDS ARE CONVENIENCE, NOT SECURITY

Every guard above exists to give the user a sensible experience. None of them protects data: the client bundle is public and any route can be entered directly. Authorisation is enforced server-side on every request, per \$11.5. A route guard that has no matching server-side check is a vulnerability, not a feature.

6.6 URL and slug rules

Fixing defect R1 properly requires a stated rule, not just a different path shape.

Rule	Specification
Slug generation	Lowercase; transliterate accents to ASCII; replace every run of non-alphanumeric characters with a single hyphen; trim leading and trailing hyphens; collapse repeats; truncate to 80 characters at a hyphen boundary.
Uniqueness	Enforced by a unique index. On collision, append <code>-2</code> , <code>-3</code> , ... until free.
Stability	A slug is generated once at creation and never regenerated when the name changes. This is the whole point: renaming "Arduino Uno R3" to "Arduino Uno Rev3" must not break the URL.
Renaming	Editing a name offers an optional "update the URL" checkbox, default off. If chosen, the old slug is written to <code>product_slug_history</code> and served as a 301 redirect to the new one, permanently.
Identifier in URL	Only slugs appear in public URLs. Numeric primary keys never do — they leak catalogue size and invite enumeration.
Tokenised URLs	<code>/pay/:t</code> and <code>/invoice/:t</code> carry a 32-byte random token, base62-encoded, stored hashed. Payment tokens are single-use and expire after 14 days; invoice tokens are revocable from the admin console.
Canonical form	Every page emits a <code><link rel="canonical"></code> . Trailing slashes are stripped; uppercase paths 301 to lowercase; query parameters that do not change content (campaign tags, <code>utm_*</code>) are excluded from the canonical URL.
Query parameters	Catalogue state lives in the query string so a filtered view is shareable and survives a refresh: <code>?page</code> , <code>?sort</code> , <code>?minPrice</code> , <code>?maxPrice</code> , <code>?inStock</code> , <code>?q</code> . Defaults are omitted rather than written explicitly.
Reserved slugs	<code>admin</code> , <code>account</code> , <code>api</code> , <code>auth</code> , <code>cart</code> , <code>checkout</code> , <code>search</code> , <code>products</code> , <code>product</code> , <code>blog</code> , <code>services</code> , <code>invoice</code> , <code>pay</code> , <code>new</code> , <code>edit</code> may not be used as any entity slug.

Storefront UI/UX Specification

Each screen below is specified as: *purpose*, *layout* (region by region, desktop then mobile), *elements* (every control, its copy, its behaviour), *states* (loading, empty, error, partial), and *interactions* (what happens on each action). Copy shown in quotation marks is the exact string to implement; the consolidated copy deck is Appendix C.

7.1 Global chrome

7.1.1 Page shell

Every storefront route renders inside `StorefrontLayout` :

```
<SkipLink>                                "Skip to main content" – visible on focus only
<UtilityBar />                             ≥ md only
<StickyHeader>
  <PrimaryBar />
  <CategoryBar />                         ≥ md only
</StickyHeader>
<Breadcrumb />                             every route except Home
<main id="main">{route}</main>
<Footer />
<CartOffcanvas /> <WishlistOffcanvas /> <MobileNavOffcanvas />
<AuthModal /> <QuickViewModal /> <SearchModal />
<ToastRegion aria-live="polite" />
<WhatsAppButton />
<EmailVerificationBanner />              only when signed in and email unverified
```

7.1.2 Search field

Aspect	Specification
Placeholder	"Search products, SKUs, part numbers..."
Suggestions	Fires after 2 characters, debounced 250 ms. A dropdown of up to 8 results, each a 40 px thumbnail, product name, subcategory eyebrow and price. Below them, "See all n results for 'term'".
Keyboard	↓/↑ move through suggestions, Enter opens the highlighted one or submits the raw term if none is highlighted, Esc closes the dropdown and keeps the text.
Submit	Navigates to <code>/search?q=...</code> . The term is logged for analytics once per distinct term per session — not on every keystroke, which is what would make the reference platform's search table useless.
No matches	Dropdown shows "No products match 'term'" plus "Browse all products" and "Ask us on WhatsApp".
Accessibility	<code>role="combobox"</code> with <code>aria-expanded</code> , <code>aria-controls</code> , <code>aria-activedescendant</code> ; the listbox has <code>role="listbox"</code> and each option <code>role="option"</code> . Result count announced in a live region.

7.1.3 Cart and wishlist indicators

- **Cart.** Icon plus a pill badge (distinct item count, not total quantity) and the running total formatted per the currency setting — `TZS 45,000` . Clicking opens the cart off-canvas. Badge animates a single 200 ms scale pulse on change, suppressed under reduced motion.
- **Wishlist.** Icon plus count badge. Clicking opens the wishlist off-canvas.

- **Zero state.** The badge renders "0" and the total "TZS 0" — matching the reference platform, which always shows the badge. This is preferred to hiding it, because a persistent target is easier to hit and to learn.

7.1.4 Breadcrumb

A home icon linking to `/`, then `>`-separated ancestors, the last being plain text in `--ts-text-muted`. Marked up as `<nav aria-label="Breadcrumb">` with an ordered list and `aria-current="page"` on the last item, plus `BreadcrumbList` JSON-LD. Trails: catalogue → Home > Products ; subcategory → Home > {Category} > {Subcategory} ; product → Home > {Category} > {Subcategory} > {Product} ; content pages → Home > {Page} .

7.1.5 Email verification banner

When a signed-in user's email is unverified, a dismissible bar sits directly beneath the header: "Please verify your email address to receive order updates." with a "Send verification link" button. On send, the button becomes a disabled "Verification link sent" for 60 seconds. Dismissal is remembered for the session only. Never shown to guests.

7.2 Home

Purpose. Give a browsing visitor three different ways into the catalogue — promotion, novelty, popularity — and establish that the store is real, stocked and staffed.

7.2.1 Layout, top to bottom

Region	Specification
1. Banner carousel	Full-bleed. Each slide is a two-column split: text on the left (7 of 12), artwork on the right (5 of 12, <code>object-fit: contain</code> , full height). The slide background colour, title text and colour, subtitle text and colour, button label, button background and button text colour all come from the banner record — every banner is independently art-directed by an administrator with no code change. Title at display size, subtitle at h1 size and hidden below <code>lg</code> , then the call-to-action button. Autoplay 6 s, loop, pause on hover and on focus-within. Previous/next chevrons, vertically centred, 40 px circular white with shadow 2, hidden below <code>md</code> where swipe takes over. Dots below on mobile. Height 380 px at <code>lg</code> , 260 px at <code>md</code> , 220 px below.
2. "OUR PRODUCTS"	Section heading, then one sentence of positioning copy (hidden below <code>md</code>), then a horizontal carousel of product cards drawn from a randomised selection. 6 cards visible at <code>xl</code> , 4 at <code>lg</code> , 3 at <code>md</code> , 2 below. Chevron controls sit outside the track on desktop, overlapping its vertical centre.
3. "NEW ARRIVALS"	Identical structure; source is products ordered by creation date descending.
4. "HOT-SELLING PRODUCTS"	Identical structure; source is products ordered by lifetime order count descending. Followed by a centred "VIEW MORE" primary button linking to <code>/products?sort=best-selling</code> .
5. "OUR PARTNERS"	Centred heading, then a horizontally scrolling strip of partner logos — 96 px square, <code>object-fit: contain</code> , greyscale at rest, full colour on hover, with the partner name beneath, truncated. Each links to the partner's site with <code>rel="noopener noreferrer"</code> . Section hidden entirely when there are no partners.

7.2.2 States

State	Rendering
Loading	Banner shows a shimmer block at its final height. Each carousel shows six skeleton cards at final dimensions. No layout shift when real content arrives — this is the single most important performance requirement on the page.
No banners	Carousel region omitted. The page begins with "OUR PRODUCTS". No placeholder, no empty coloured band.

State	Rendering
Empty rail	That section is omitted. A section heading with nothing under it is never rendered.
Partial failure	Each rail loads independently. One failing request degrades one rail to an inline "Could not load this section. Retry" — it never blanks the page.

7.2.3 Product card, definitive specification

This component appears in every grid and carousel in the storefront. Specified once, here.

Element	Specification
Container	White, 1 px <code>--ts-border</code> , radius <code>md</code> , shadow 1 → 2 on hover. The whole card is a link to the product page; nested action buttons stop propagation.
Image well	1:1 aspect ratio, <code>--ts-surface-sunken</code> , <code>object-fit: contain</code> with 8 px inset. Uses the <code>md</code> derivative with an <code>srcset</code> offering <code>sm</code> for narrow viewports. <code>loading="lazy"</code> and explicit <code>width / height</code> to reserve space. <code>alt</code> is the product name.
Badges	Discount badge top-left, accent amber, "–15%". "NEW" badge top-right, primary green, for products created within 30 days. "OUT OF STOCK" ribbon across the lower third of the image well, red at 90% opacity, when total stock is zero.
Eyebrow	Subcategory name, 11 px uppercase, muted, centred, truncated to one line.
Title	Product name, 13 px semibold uppercase, centred, truncated to one line with ellipsis. Full name available as the link's accessible label — never rely on <code>title</code> alone.
Price	Centred, primary green, 15 px semibold. Multi-variant products show "From TZS 200". Discounted products show the original struck through in muted grey before the new price. Zero-price products cannot exist (\$12.3), so no zero case is designed.
Action row	Two 32 px icon buttons, centred: wishlist (<code>heart</code> , filled and green when present in the wishlist) and quick view (<code>eye</code> , opens the preview modal). Visible at all breakpoints; the reference platform hides these below <code>md</code> , which removes the only way to reach quick view on a phone.
Add to cart	A pill button that slides up from the card's lower edge on hover, on desktop only. Label "ADD TO CART". Below <code>md</code> , and on any coarse pointer, it is a permanently visible full-width button in the card footer — hover-only affordances do not exist on touch.
Out of stock	Add to cart replaced by a disabled "OUT OF STOCK" and a secondary "Notify me" that captures an email against the variant.
Multi-variant	Add to cart reads "CHOOSE OPTIONS" and navigates to the product page rather than adding, because there is nothing unambiguous to add.
Keyboard	Card is one tab stop; the two icon buttons and add-to-cart are separate stops in reading order. Focus ring is the 2 px primary-green outline. Hover-revealed controls become visible on focus-within.

7.2.4 Quick-view modal

Max width 900 px. Left (5 of 12): the `lg` image with a thumbnail strip. Right (7 of 12): name, price, stock, a two-line clamped description, variant selector if applicable, quantity stepper, "ADD TO CART", and "View full details" linking to the product page. Close by button, scrim click or Esc. Focus trapped inside while open; returned to the triggering card on close. Adding to cart from the modal closes it and opens the cart off-canvas.

7.3 Catalogue

Purpose. Let a visitor traverse 600+ products, narrowed by category, price and availability, and ordered how they choose. One component serves all three catalogue routes; only the scope differs.

7.3.1 Layout

Region	Specification
Breadcrumb	Per §7.1.4.
Heading	The subcategory name, or category name, or "All Products". Beside it, the result count: "617 products".
Control bar	One row, wrapping. Left: "Sort by:" then three inline text controls — Best Selling , New Arrivals , Price . Price toggles ascending/descending and shows an arrow indicating direction. The active control is primary green with a 2 px underline. Right: a "Filters" button showing the count of active filters, which opens the filter panel; and a view-density toggle (comfortable / compact grid) persisted to <code>localStorage</code> .
Active filter chips	Beneath the control bar when any filter is set: one chip per filter with an <code>x</code> , plus "Clear all". Each chip states its value — "TZS 1,000 – 50,000", "In stock only".
Grid	Product cards. 6 columns at <code>xl</code> , 4 at <code>lg</code> , 3 at <code>md</code> , 2 below. 16 px gutters. Row heights equalised so cards align regardless of name length.
Pagination	Bottom-centred. Previous chevron, page numbers with ellipsis for long ranges, next chevron; plus a "per page" select offering 24 / 48 / 96, default 24. This replaces the reference platform's <code>limit=10000</code> single-shot load (defect R17).
Filter panel	Desktop: a 260 px left column that appears when Filters is toggled, pushing the grid to 9 of 12 columns. Mobile: a full-height off-canvas from the left with a sticky "Apply" footer showing the matching count.

7.3.2 Filters

Filter	Control	Behaviour
Category	Collapsible tree of categories with subcategory children and counts	Selecting a node navigates to its route rather than adding a query parameter, so the URL stays meaningful and the breadcrumb stays correct.
Price	Two numeric inputs, "MIN PRICE" and "MAX PRICE", plus an "Apply" button	Matches the reference platform's modal. Inputs are seeded with the actual minimum and maximum in the current scope. Validation: numeric, non-negative, $\text{min} \leq \text{max}$; on violation the message "Minimum price must be less than maximum price" appears and Apply stays disabled.
Availability	Checkbox, "In stock only"	Excludes products whose total stock across variants is zero.
Discounted	Checkbox, "On offer"	Products with an active discount window.

All filter state lives in the query string (§6.6) so a filtered view can be shared, bookmarked and returned to with the browser's back button.

7.3.3 States

State	Rendering
Loading, first page	Twenty-four skeleton cards. Control bar renders immediately.

State	Rendering
Loading, page change	The existing grid dims to 50% opacity with a spinner overlay; it is not replaced by skeletons. Scroll position returns to the top of the grid, not the top of the page.
Empty, no filters	"No products found" with a <code>package-x</code> icon and a "Browse all categories" action.
Empty, with filters	"No products match your filters" plus a "Clear all filters" primary action and a summary of what was applied. Distinguishing these two is what stops a filtered empty state from reading like an empty catalogue.
Empty subcategory	"No products found in the selected category" plus links to sibling subcategories that do have products.
Error	Inline panel, "We could not load products just now.", with "Try again". Filters and breadcrumb remain usable.

7.4 Product detail

Purpose. Answer "is this the right part, is it in stock, what does it cost" in the first screenful, and support the decision below the fold.

7.4.1 Layout — above the fold

Region	Specification
Gallery (left, 6 of 12)	Main image in a 1:1 well on <code>--ts-surface-sunken</code> , <code>object-fit: contain</code> , using the <code>lg</code> derivative. Click opens a lightbox with pan and zoom; on touch, pinch to zoom. Beneath it a thumbnail strip of <code>sm</code> derivatives, 4 visible with chevron paging, the active one outlined in primary green. Keyboard: <code>←/→</code> change image; thumbnails are a tab-navigable radio group.
Buy box (right, 6 of 12)	An elevated white card, 1 px border, radius <code>md</code> , shadow 1. Contents in order: product name as h1 (uppercase, 22 px, allowed to wrap to three lines); SKU in monospace muted with a click-to-copy control; average rating stars plus "(n reviews)" linking to the reviews tab, omitted when there are none; price block; variant selector; quantity stepper and stock display side by side; then three full-width stacked buttons.
Price block	Current price at 24 px semibold primary green. When discounted: original struck through in muted grey above, then the discounted price, then an amber "SAVE 15%" badge and, if the discount window ends within 7 days, "Offer ends 3 March".
Variant selector	Omitted for single-variant products. One control per attribute, labelled with the attribute name in uppercase. ≤ 6 options render as a pill group; more render as a select. Options that are out of stock are shown struck through and disabled — never hidden, because their absence is information the buyer needs. Choosing a variant updates price, SKU, stock and the gallery if that variant has its own image, and writes the selection to the URL as <code>?v=SKU</code> so a specific variant is shareable.
Quantity	Label "QUANTITY:". A stepper: <code>-</code> button, numeric input, <code>+</code> button, matching the reference platform's arrangement. Minimum 1; maximum is available stock; the input accepts typing and clamps on blur. <code>-</code> is disabled at 1; <code>+</code> disabled at stock, with a tooltip "Only n available".
Stock	Label "AVAILABLE STOCK:" and a read-only field showing the number, as the reference platform does. Above 5: green dot and the count. 1–5: amber dot and "Only n left". 0: red dot and "Out of stock".
Actions	ADD TO CART (primary, full width) · ADD TO WISHLIST (secondary) · BUY NOW (primary, adds to cart and navigates straight to checkout). Out of stock: the first and third are replaced by a disabled "OUT OF STOCK" plus "NOTIFY ME WHEN AVAILABLE".

7.4.2 Layout — below the fold

- **Tabs.** "DESCRIPTION" and "REVIEWS (n)", as on the reference platform, plus "SPECIFICATIONS" when the product has attributes. Implemented as real ARIA tabs with arrow-key navigation. The panel is not lazily unmounted, so browser find-in-page works across it.
- **Description panel.** Sanitised rich text from the admin editor. Permitted elements: `p, strong, em, u, ul, ol, li, h3, h4, a, br, table, thead, tbody, tr, th, td, img, code, pre, blockquote`. Everything else is stripped server-side on save *and* client-side on render — sanitising in one place only is how stored XSS happens.
- **Specifications panel.** A two-column table of attribute names and values for the selected variant.
- **Reviews panel.** Summary at the top: average out of 5, total count, and a five-row histogram with proportional bars. Then a list of reviews, each with rating stars, author display name, relative date, body, and a "Verified purchase" badge where the reviewer has a delivered order containing the product. Sort by most recent or highest rating; paginated at 10. A signed-in user who has not yet reviewed sees the review form; a guest sees "Please sign in to write a review" with a button that opens the auth modal. Own reviews can be edited or deleted.
- **Review form.** Required 1–5 star radio group, optional 80-character title, required body of 10–2,000 characters with a live counter. Submits as `pending` for moderation with the message "Thank you — your review will appear once approved."
- **"RELATED PRODUCTS".** Heading, one line of copy, then up to 12 cards in a carousel drawn from the same subcategory, excluding the current product, ordered by sales. Section omitted when empty rather than rendered with a stray heading.
- **Recently viewed.** Up to 8 cards from a client-side ring buffer, deduplicated by product id. Omitted when fewer than two entries exist.

7.4.3 States and edge cases

Case	Behaviour
Loading	Gallery and buy box render as skeletons at final dimensions. Tabs render with their labels and a skeleton body.
Slug not found	Check <code>product_slug_history</code> ; if the slug was retired, 301 to the current one. Otherwise render the 404 page with a search field and "Browse all products".
Product soft-deleted	410 Gone semantics: "This product is no longer available" plus related products from the same subcategory. Not a bare 404 — the page existed and may be linked from elsewhere.
No images	Placeholder graphic bearing the TechStar mark. Never a broken image icon.
Every variant out of stock	Buy box shows the out-of-stock treatment; the variant selector remains interactive so the buyer can still read prices and register interest.
Price changed since a cached view	The buy box is authoritative. If the cart holds this item at a different price, the cart shows a change notice on its next revalidation (\$12.5).
Quantity exceeds stock at submit	Server rejects; the field is clamped to available stock and a toast explains "Only n available — quantity adjusted."

7.4.4 Structured data

Product	name, description (plain text), sku, image[], brand, category, aggregateRating (when reviews exist)
Offer	price, priceCurrency "TZS", availability (InStock OutOfStock), url, priceValidUntil when discounted
BreadcrumbList	Home > Category > Subcategory > Product
Review[]	author, reviewRating, datePublished, reviewBody

Emitted as a single `application/ld+json` block. Every value must match what is rendered — divergence between structured data and visible content is a manual-action risk with search engines, not a clever optimisation.

7.5 Search results

Purpose. Serve the electronics buyer who arrived with a part number. Speed and precision matter more than presentation.

- Heading: "Search results for '*term*'" with the count beneath: "28 products found".
- The same control bar, filter panel, grid and pagination as the catalogue — one component, one behaviour to learn.
- The searched term is highlighted within result names using a `<mark>` styled with an amber tint.
- Matching is specified in §16.2: SKU exact match ranks first, then name prefix, then name contains, then description full-text. A search for "74HC14" must surface that exact part before anything merely mentioning it.
- **Empty state.** "No products match '*term*'", then in order: spelling suggestion ("Did you mean *arduino*?") when one is available within an edit distance of 2 of an indexed term; up to 6 popular products; "Browse all categories"; and "Ask us on WhatsApp — we may be able to source it", which turns a dead end into a lead.
- Every search with zero results is recorded with its term and result count, and surfaced in the admin console (§16.4) as the catalogue's own to-do list.

7.6 Cart and wishlist

7.6.1 Cart off-canvas

Region	Specification
Header	Primary green, white uppercase title "SHOPPING CART", item count, white close button.
Line item	A 64 px <code>sm</code> thumbnail; name (two-line clamp, links to the product); selected variant attributes in muted 12 px; unit price; a compact quantity stepper; line total in semibold; and a <code>trash-2</code> remove button. Rows separated by a 1 px border.
Change notices	When revalidation finds a difference, an amber inline strip on the affected row: "Price changed from TZS 5,000 to TZS 5,500" or "Only 2 left — quantity reduced". This is the visible half of the fix for defect R6.
Footer	Pinned to the bottom, on <code>--ts-surface-alt</code> . Subtotal, a note that shipping is calculated at checkout, then "CHECKOUT" (primary, full width) and "View cart" (text link).
Empty	Centred <code>shopping-cart</code> icon at 48 px muted, "Your cart is empty", one line of guidance, and a "Browse products" primary button. The reference platform renders a bare <code><h1>Cart is Empty</h1></code> ; this replaces it.
Behaviour	Opens automatically on add-to-cart, with the newly added row briefly highlighted in <code>--ts-primary-tint</code> . Quantity changes are optimistic and reconciled with the server; a rejected change reverts with a toast. Removal offers a 5-second "Undo" in the toast before it is committed.

7.6.2 Cart page

Same data, laid out as a two-column page: a line-item table on the left (8 of 12) with columns Product, Unit price, Quantity, Total, Remove; and an order summary card on the right (4 of 12), sticky on scroll, holding subtotal, estimated shipping once a method is chosen, total, and "PROCEED TO CHECKOUT". Below the table: "Continue shopping" and "Clear cart" (with confirmation). Stacks to one column below `lg`.

7.6.3 Wishlist off-canvas

Header "WISHLIST". Rows carry thumbnail, name, price, current stock status, "Move to cart" and remove. No quantity — a wishlist entry is an intent, not a line item, which is why the reference platform's unused quantity field is dropped. Empty state: heart icon, "Your wishlist is empty", "Browse products". Signed-in wishlists persist server-side and merge on sign-in per §12.5.4.

7.7 Checkout

Purpose. Convert a cart into a paid order with the fewest possible decisions, without forcing account creation. This is the highest-value screen in the product and the one where the reference platform's design is at its strongest: a single page, no steps, no wizard, everything visible at once.

7.7.1 Layout

Two columns at `lg` and above: the form on the left (8 of 12), the order summary on the right (4 of 12), sticky from the top of the viewport. Below `lg`, the summary moves above the form as a collapsed accordion showing the total, expandable to the full breakdown, and the "PLACE ORDER" button becomes a fixed bottom bar showing the total.

Block	Specification
Page heading	"Getting your order" — kept verbatim from the reference platform. It is warmer and less transactional than "Checkout", and the visitor already knows where they are.
Sign-in prompt	Two lines, shown only to guests. "Returning customer? Click here to log in " and "Don't have a TechStar account? Click here to create an account ". Both open the auth modal at the corresponding tab and, on success, refill the form from the profile without losing anything already typed.
1. CLIENT DETAILS	Bordered card. Two fields side by side at <code>md</code> +, stacked below. Username — the buyer's name, placeholder "Username", prefilled from the profile when signed in. Phone — placeholder "255XXXXXXXXX", <code>inputmode="numeric"</code> .
2. SHIPPING METHOD	Bordered card with the heading row "SHIPPING METHOD" on the left and "ESTIMATED COST" right-aligned. One radio per method: label on the left, formatted cost on the right in semibold. The method's description renders beneath its label in muted 12 px — the reference platform hides this in a tooltip, where a customer choosing between "Savings" and "Cash on delivery" will never find it. Selecting a method immediately updates the summary and reveals or hides the two dependent blocks below.
3. SHIPPING DETAILS	Shown only when the selected method has <code>requiresShippingDetails</code> . Fields: Receiver's name (prefilled from Client Details, editable — the recipient is often not the buyer); Email ; Phone ; Country (fixed to "Tanzania", disabled); Region (select of the 25 regions); District (select, disabled and empty until a region is chosen, then loaded for that region); Street address (textarea, 2 rows); Postal code / Zip code (optional). A "Use my saved address" checkbox appears for signed-in users with a saved address, checked by default, and collapses the block to a summary with an "Edit" link.
4. PAYMENT METHOD	Shown only when the selected shipping method has <code>canReceiveMobilePayments</code> ; when it does not — as with Cash on delivery — payment is implicitly cash and this block is replaced by a short explanatory note. Two radios: Lipa Namba (mobile money) and Cash . Choosing Lipa Namba reveals the payment-instructions panel (§7.8.1) and a required Payment number field, placeholder "255XXXXXXXXX", helper text "The mobile money number you will pay from."
5. SOMEONE ELSE PAYING?	A collapsed disclosure, "Someone else is paying for this order". Expanding reveals one field, Payer's email address , with helper text explaining that a payment link will be emailed to them and the order will be held until they pay. Specified in §12.9.

Block	Specification
Order summary	Card on <code>--ts-surface-alt</code> . Header "Your Order" with a cart icon. Then the PLACE ORDER primary button — placed at the <i>top</i> of the summary as the reference platform does, so it is above the fold on desktop. Then a compact line-item list (thumbnail, name, "Quantity: n", line total), then Sub Total, Shipping Cost, a rule, and TOTAL in 18 px semibold. An "Edit cart" link returns to <code>/cart</code> .

7.7.2 Validation

Validated on blur, then on every change once a field has errored, then in full on submit. Messages are the reference platform's own wording where it is already clear.

Field	Rule	Message
Username	required, 2–80 chars	"Username is required"
Phone	required	"Phone is required"
Phone	digits only	"Phone must be a number"
Phone	exactly 12 digits	"Phone number must be 12 characters long"
Phone	starts with <code>255</code>	"Enter your number in the form 255XXXXXXXX"
Shipping method	required	"Please select a shipping method"
Email (shipping)	required, valid	"Email is required" / "Enter a valid email"
Receiver's name	required	"Receiver's name is required"
Region	required	"Region is required"
District	required	"District is required"
Street address	required, 5–255 chars	"Street address is required"
Payment method	required when the block is shown	"Please select a payment method"
Payment number	required when Lipa Namba	"Enter a payment number"
Payment number	digits, exactly 12	"Payment number must be 12 characters long"
Payer's email	valid when present	"Enter a valid email"

On a failed submit, focus moves to the first invalid field, the page scrolls it into view beneath the sticky header, and a live region announces "n fields need attention".

7.7.3 Submit sequence

```
PLACE ORDER pressed
├ client-side validation — fail → focus first error, announce, stop
├ button enters loading state, label replaced by spinner, form disabled
├ POST /orders { customer, items[], shippingMethodId,
                shippingAddress?, paymentMethod, paymentNumber?,
                delegatedPayerEmail?, idempotencyKey }
├ server revalidates EVERY line: product live, variant live,
  price unchanged, stock sufficient (§12.5.3)
├ 409 PRICE_CHANGED → summary updates in place, amber notice
│   "Prices have changed. Please review and
│   place your order again." Button re-enabled.
├ 409 OUT_OF_STOCK → affected lines flagged, quantities clamped,
│   same re-confirm treatment
├ 200/201 → continue
├ order created, stock reserved, order number issued (§12.7)
├ cart cleared client-side and server-side
├ navigate to /order/confirmation/:orderNumber
  (if an aggregator returned a hosted checkout URL, redirect there
  instead and return to the confirmation route afterwards — this is
  the 'url' field the reference platform's client already honours)
```

IDEMPOTENCY

The client generates a UUID when the checkout page mounts and sends it as `idempotencyKey`. The server stores it against the created order and returns the same order for any repeat of the same key within 24 hours. A double-click, a flaky mobile connection or an impatient refresh must never create two orders — on a payment screen this is not a nicety.

7.7.4 States

State	Rendering
Empty cart	Guard redirects to <code>/cart</code> with the toast "Your cart is empty."
Loading reference data	Shipping method list and region select render as skeletons; the identity fields are usable immediately.
Districts loading	District select shows a spinner and is disabled; it never appears empty and enabled.
Submitting	All inputs disabled, button spinning, a scrim over the summary. Navigation away triggers a <code>beforeunload</code> warning.
Network failure	"We could not reach the server. Your cart is safe — please try again." Form re-enabled with all input preserved. The idempotency key is retained so a retry cannot duplicate.
Signed in mid-checkout	Profile data merges into empty fields only; anything the user has already typed is never overwritten.

7.8 Payment

7.8.1 Payment instructions panel

Shown inline at checkout once Lipa Namba is selected, and again on the confirmation page. It is a two-column card of network-specific instructions, one block per mobile money network, plus a QR code for the primary network. The instructions are the operational heart of the manual payment flow and are therefore **content managed in the admin console** (§14.16), not hard-coded — till numbers change, and a code deploy is the wrong way to change one.

Block	Contents
M-Pesa (Vodacom)	Numbered steps: dial the USSD string or open the app, choose "Lipa kwa M-Pesa", enter the till number, enter the amount, enter the PIN, await the confirmation SMS. Accompanied by the app QR code.
Mixx by Yas (Tigo Pesa)	Numbered steps for the Tigo USSD menu and till number.
Airtel Money	Numbered steps for the Airtel USSD menu and till number.
Other networks	Steps for sending to another network, ending at the M-Pesa till number.
Amount	Rendered prominently above the blocks: "Pay exactly TZS 45,000 " with a copy control, and the order number beneath as the reference to quote.
Language	Presented in Swahili with an English toggle. The reference platform ships Swahili only; both matter, because the buyer's phone menus are in one language and the site is in the other.

7.8.2 Order confirmation page

`/order/confirmation/:orderNumber`. Contents in order:

- A success panel: green `check-circle`, "Order placed", the order number in monospace with a copy control, and "We have emailed a copy to *address*" when an email is known.
- **What happens next** — three or four numbered steps that differ by payment method. Lipa Namba: pay, we confirm within one business hour, we pack, we ship. Cash on delivery: we call to confirm, we pack, you pay the courier on delivery. Delegated: we have emailed the payer, the order is held until they pay.
- The payment instructions panel, when the method is Lipa Namba and payment is not yet confirmed.
- Order summary: items, quantities, unit prices, subtotal, shipping, total, delivery address and chosen shipping method.
- Actions: "View order", "Print", "Continue shopping".
- A live status strip that updates over the realtime channel — when an administrator marks payment received, the page changes to "Payment confirmed" without a refresh (§17.5).

7.8.3 Delegated payment page

`/pay/:paymentToken`, opened by the nominated payer, who is usually not a TechStar account holder and may never become one.

Aspect	Specification
Heading	"Payment request" and beneath it " <i>Amina M.</i> has asked you to pay for this order." Only the requester's first name and last initial — never their full contact details.
Order details	Read-only: item thumbnails, names, "Quantity: n", line totals, Sub Total, Shipping Cost, TOTAL. No delivery address, no phone number, no email.
Payment methods	The same radio group and instructions panel as checkout, plus the required payment-number field.
Action	"MAKE PAYMENT" primary button, full width.
After submit	Success panel confirming the payment is recorded and pending verification; both payer and requester are notified.
Token states	Already paid → "This order has already been paid." Expired → "This payment link has expired." plus a "Contact TechStar" action. Invalid → the same generic message as expired, so tokens cannot be probed.
Robots	<code>noindex, nofollow</code> . Never included in the sitemap.

7.9 Order history and detail

7.9.1 Order list — `/account/orders`

- Heading "My orders". Filter chips: All · Pending · Processing · Shipped · Delivered · Cancelled, each with a count.
- Desktop: a table with Order (monospace number), Date, Items (count), Total, Status pill, and a "View" action. Mobile: stacked cards carrying the same fields.
- Sorted newest first; paginated at 20.
- Empty: `package` icon, "You have not placed any orders yet", "Browse products".

7.9.2 Order detail — `/account/orders/:orderNumber`

Region	Specification
Header	"Order <code>TS-2026-000148</code> ", placement date, status pill, and a payment-status pill ("Awaiting payment", "Paid", "Refunded").
Timeline	A vertical stepper built from the order event log: Placed → Paid → Processing → Shipped → Delivered, each completed step showing its timestamp, the current step highlighted, future steps muted. Cancellation replaces the remainder with a red terminal node.
Items	Thumbnail, name (linking to the product), selected attributes, SKU, unit price, "Quantity: n", line total.
Client details	"Client:", "Email:", "Phone:" — falling back to "Anonymous" and "Not specified" as the reference platform does, since guest orders legitimately lack these.
Shipping	Method name and cost; delivery address as "Receiver's name:", "Country:", "City/Town:", "Street Address:", "Postal Code:".
Totals	Sub Total, Shipping Cost, TOTAL.
Actions	Order again — adds every still-available line to the cart and reports what could not be added. Print invoice — opens the print stylesheet. Cancel order — only while the status is Pending or Awaiting payment; opens a confirmation dialog reading "Are you sure you want to cancel this order?" with an optional reason. Pay now — when awaiting payment, reopens the instructions panel.

CANCELLATION IS A TRANSITION, NOT A DELETION

Fixing R18: the reference platform cancels by sending `DELETE /order/:id`. Here, cancelling writes a `CANCELLED` transition and an event-log entry, releases any stock reservation, and keeps the order permanently visible to both customer and administrator. Financial records are never deleted by a customer action.

7.10 Account

`/account` renders a layout with a left navigation rail (a horizontal tab bar below `md`): **Account · Orders · Shipping address · Change password**. Above the rail sit the avatar, display name and email.

Tab	Fields and behaviour
Account	Avatar with "Upload image" (JPEG/PNG/WebP, max 3 MB, cropped square client-side, live preview). Then Email (changing it re-triggers verification and warns so before saving), Username , Phone Number (255XXXXXXXXX) . "Update" primary button, disabled until something changes. Success toast "Successfully updated user account".
Orders	The list from §7.9.1 rendered inside the account shell.

Tab	Fields and behaviour
Shipping address	Receiver's Name , with a "Details similar to my account" checkbox that copies name, email and phone from the profile. Email, Phone, Country (fixed "Tanzania"), Region, District (dependent), Street Address, Postal Code/Zip Code . Saved address prefills checkout. Success toast "Successfully updated shipping details".
Change password	Current password, New password with a live strength meter, Confirm new password . Rules in §11.4.3. On success, all other sessions are revoked and the user is told so: "Password changed. You have been signed out on other devices."

7.11 Content pages

7.11.1 About

Hero band with the admin-managed image behind a dark scrim, the page title centred, and a breadcrumb. Then, in order: **Our Story** — long-form copy with a "Read more" expander that reveals the remainder in place; **Vision** and **Mission** as two cards side by side; **Core values** as a four-up grid of icon, name and one line each; and finally a **map**. The map is Leaflet with OpenStreetMap tiles, a single marker at the store's coordinates with a popup showing name, address and a "Get directions" link. Height 400 px, non-scroll-hijacking (zoom requires Ctrl + wheel or a two-finger gesture), with the address also rendered as text beneath so the information does not depend on tiles loading. The anchor `/about#map` is linked from the utility bar and must scroll to it accounting for the sticky header offset.

7.11.2 Contact

Heading, then two columns. Right (5 of 12) — **CONTACT DETAILS**: address, "E-Mail:", "Tel:" with two numbers, opening hours, and social links, each with an icon. Left (7 of 12) — **GET IN TOUCH WITH US**: Your name (required), Email (required, valid), Subject (optional), Message (required, 10–2,000 characters with a counter), a checkbox "Send me a copy of this message", and an invisible honeypot plus a timing check for spam. "Send Message" primary button. On success the form is replaced in place by a confirmation panel — "Thank you for getting in touch with TechStar Store" — and a "Submit another message" link. Errors: "Name is required", "Email is required", "Email not valid", "Message is required", and on failure "Failed to send message" with the input preserved.

7.11.3 FAQ

Centred heading "Frequently Asked Questions", optional intro, then an accordion grouped by topic — Ordering, Payment, Shipping & Delivery, Returns, Products, Account. One item open by default. Real `<button>` headers with `aria-expanded` and `aria-controls`; chevron rotates on expand. Below the accordion, a support panel: "Can't find your answer?" with "Contact us" and "Chat on WhatsApp". `FAQPage` JSON-LD is emitted. Entries are admin-managed with an ordering field.

7.11.4 Privacy policy and terms

Same shell: hero band with admin-managed image, page title, breadcrumb, then long-form content with a sticky table of contents on the left at `lg` + that highlights the section in view. Sections are anchored so individual clauses can be linked. A "Last updated" date is shown at the top and maintained by the administrator. Both documents must be drafted for TechStar (§1.5); the platform provides the structure, the ordering and the anchors, not the words.

7.12 Blog

- **Index.** Heading "Blog", then a responsive card grid — 3 columns at `lg`, 2 at `md`, 1 below. Each card: 16:9 cover image, title (two-line clamp), publication date, excerpt (three-line clamp), comment count, and "Read more".

Paginated at 9.

- **Article.** Cover image, title as h1, author and date, then sanitised rich-text body constrained to a 72-character measure. A right rail at `Lg` + carries "Latest Articles" — five entries with 56 px thumbnails.
- **Comments.** Beneath the article: count, then a list of comments each with avatar, display name, relative timestamp and body. A signed-in user sees a composer with the placeholder "Write something here" and a "Post comment" button; a guest sees "Please create an account or log in to comment" with a button opening the auth modal. Submitted comments enter moderation with "Thank you for your comment — it will appear once approved." Authors may delete their own.
- **Empty.** "No articles yet" with a `newspaper` icon. The reference platform ships this module with zero content; TechStar should launch with at least three articles or hide the Blog navigation entry entirely — an empty section in primary navigation reads as neglect.

7.13 Services

- **Index.** Card grid of services: image, heading, excerpt, "Learn more".
- **Detail.** Hero image, heading, publication date, rich-text body. A right rail lists "Other Services". At the foot, a call to action: **"ASK FOR DESIGN SERVICE"** with a "Contact Us" button that opens the contact form with the subject prefilled with the service name.
- Each service also appears in the SERVICES header dropdown, sourced from the latest-services endpoint.

7.14 Authentication surfaces

Authentication is a modal, never a page — the reference platform's choice, and the right one, because it preserves cart and checkout context. The modal opens at the tab requested and remembers the route to return to.

Surface	Fields and behaviour
Modal shell	Centred, max-width 460 px, with the TechStar mark in a white circle overlapping the top edge. Header bar in primary green with the current mode as a white uppercase title and an "x" close control. Focus trapped; Esc closes; focus returns to the trigger.
LOGIN	EMAIL OR PHONE — one field accepting either, as the reference platform does; the server resolves which. PASSWORD with a show/hide eye toggle. "Forgot password?" link. "LOGIN" primary button, full width. "Don't have an account? Create your account here ". Then a divider and two social buttons: "Continue with Facebook" (brand blue) and "Continue with Google" (white, outlined, with the Google mark).
REGISTER	USERNAME, EMAIL, PHONE (255XXXXXXXXX), PASSWORD with a strength meter, CONFIRM PASSWORD , and a required checkbox "I agree to the Terms & Conditions and Privacy Policy" with both linked. "REGISTER" primary button. On success the user is signed in immediately, the modal closes, a verification email is sent and the verification banner appears.
FORGOT PASSWORD	One EMAIL field and "SEND RESET LINK". The response is always the same regardless of whether the address exists: "If an account exists for that address, we have sent a reset link." Anything else is an account-enumeration oracle.
Reset page	<code>/auth/reset-password/:token</code> . New password plus confirmation, strength meter, and the same rules as registration. On success: sign in, revoke all other sessions, toast "Password changed", redirect home. Invalid or expired token: "This reset link is no longer valid" with a control to request a new one.
Email verification	<code>/auth/verify-email/:token</code> . Shows a spinner, then either "Email verification successful" or "Verification failed" with a resend control, then redirects home after 3 seconds.

Surface	Fields and behaviour
OAuth callback	<code>/auth/oauth/callback</code> . Full-screen spinner with "Signing you in...", then exchanges the one-time code, then redirects to the stored return route. Failure: "Failed to sign in" with a control to return to the sign-in modal.
Errors	Credential failures render one message above the form — "Email, phone or password is incorrect" — never distinguishing which was wrong. Rate-limit lockout renders "Too many attempts. Please try again in n minutes."

7.15 Errors and empty states

Case	Copy	Actions
404	"Oops!" / "Page not found" / "The page you're looking for doesn't exist."	"Go home" primary; a search field; popular categories. (The reference platform ships this as "Oops!" — see R15.)
403	"You don't have access to this page."	"Go home"; "Sign in as a different user".
500	"Something went wrong at our end."	"Try again"; "Go home"; the request id in small monospace so support can trace it.
Offline	"You appear to be offline."	Auto-retries on reconnect. The cart survives, because it is in local storage.
Maintenance	"TechStar Store is briefly down for maintenance."	Expected return time; WhatsApp contact.
Slow response	After 8 seconds, an inline "This is taking longer than usual..." beneath the spinner	Never an indefinite spinner with no explanation.

ERROR-COPY RULE

Every error message states what happened, whether the user's data is safe, and what to do next. "Failed to place order" alone is not acceptable; "We could not place your order. Nothing has been charged and your cart is unchanged. Please try again." is. This rule applies equally to the admin console.

Component Catalogue

Every component the platform needs, with its responsibility and its public contract. Components live in `packages/ui` when they are design-system primitives with no domain knowledge, and in `apps/web/src/components` when they know about products, orders or carts. That boundary is enforced by lint rule: nothing in `packages/ui` may import from `apps/web`.

8.1 Primitives

Component	Key props	Notes
Button	<code>variant: primary secondary danger ghost link</code> , <code>size: sm md lg</code> , <code>loading</code> , <code>disabled</code> , <code>fullWidth</code> , <code>iconStart</code> , <code>iconEnd</code> , <code>as</code>	Holds its width while loading so the layout never jumps. <code>as="a"</code> renders an anchor with button styling and correct roles.
IconButton	<code>icon</code> , <code>label</code> (required), <code>size</code> , <code>variant</code> , <code>active</code>	<code>label</code> is required and becomes <code>aria-label</code> . An icon-only control with no accessible name fails review.
TextField	<code>label</code> , <code>name</code> , <code>type</code> , <code>placeholder</code> , <code>hint</code> , <code>error</code> , <code>required</code> , <code>prefix</code> , <code>suffix</code> , <code>inputMode</code> , <code>maxLength</code> , <code>showCounter</code>	Generates its own ids and wires <code>aria-describedby</code> to hint and error. Errors render with an icon and set <code>aria-invalid</code> .
TextArea	as <code>TextField</code> plus <code>rows</code> , <code>autoResize</code>	
Select	<code>options</code> , <code>value</code> , <code>onChange</code> , <code>placeholder</code> , <code>disabled</code> , <code>loading</code> , <code>error</code>	Native <code><select></code> for short lists — better on mobile than any custom widget.
SearchSelect	<code>options</code> , <code>value</code> , <code>onChange</code> , <code>isMulti</code> , <code>onSearch</code> , <code>loading</code> , <code>createable</code>	react-select, restyled to the token set. Used for subcategory pickers, campaign contacts and tag entry.
Checkbox, Radio, RadioGroup, Switch	<code>label</code> , <code>description</code> , <code>checked</code> , <code>onChange</code> , <code>error</code>	44 px minimum hit area on touch regardless of visual size.
QuantityStepper	<code>value</code> , <code>min</code> , <code>max</code> , <code>onChange</code> , <code>size</code>	Clamps on blur; disables its bounds; announces changes politely.
Badge, StatusPill	<code>tone: neutral success warning danger info accent</code> , <code>children</code>	<code>StatusPill</code> maps a domain status to a tone via one table, so order status colours are defined once (§5.2.3).
Price	<code>amount</code> , <code>compareAt</code> , <code>from</code> , <code>size</code>	The only place currency formatting happens. Reads the currency setting from context.
Rating	<code>value</code> , <code>count</code> , <code>size</code> , <code>interactive</code> , <code>onChange</code>	Interactive mode is a radio group, not clickable icons.
Skeleton	<code>variant: text rect circle</code> , <code>width</code> , <code>height</code> , <code>lines</code>	Static tint under reduced motion.
Spinner	<code>size</code> , <code>label</code>	Always carries a visually hidden label.
Modal	<code>open</code> , <code>onClose</code> , <code>title</code> , <code>size</code> , <code>footer</code> , <code>closeOnScriim</code>	Focus trap, scroll lock, Esc, focus restoration, <code>aria-modal</code> .
Offcanvas	<code>open</code> , <code>onClose</code> , <code>side</code> , <code>title</code> , <code>footer</code>	Same accessibility contract as <code>Modal</code> .

Component	Key props	Notes
Toast , ToastRegion	tone , title , description , duration , action	Region is <code>aria-live="polite"</code> ; error toasts do not auto-dismiss.
ConfirmDialog	open , title , message , confirmLabel , tone , onConfirm , requireTyping	<code>requireTyping</code> makes the user type a phrase before the confirm button enables — reserved for bulk deletes (§14.19).
Tabs	tabs , active , onChange	Full ARIA tabs pattern with arrow-key navigation.
Accordion	items , allowMultiple , defaultOpen	
Pagination	page , pageCount , onChange , pageSize , onPageSizeChange	Ellipsis for long ranges; <code><nav aria-label="Pagination"></code> .
EmptyState	icon , title , description , action	Used everywhere. No list, grid or table may render a bare blank area.
Breadcrumb	items	Emits <code>BreadcrumbList</code> JSON-LD alongside the markup.

8.2 Storefront composites

Component	Key props	Responsibility
ProductCard	product , onQuickView , density	§7.2.3. Pure presentation — cart and wishlist actions come from hooks, not props, so the card can be dropped anywhere.
ProductGrid	products , loading , skeletonCount , emptyState	Responsive columns per §5.6.
ProductCarousel	title , subtitle , products , loading , viewAllHref	Renders nothing at all when the list is empty — including the heading.
BannerCarousel	banners , autoplayMs	Applies each banner's own colours and copy.
ProductGallery	images , activeIndex , onChange	Main image, thumbnail strip, lightbox.
VariantSelector	attributes , variants , selected , onChange	Pills up to 6 options, select beyond. Marks unavailable combinations disabled rather than hiding them.
BuyBox	product , selectedVariant , onAddToCart , onBuyNow	Price, stock, quantity, the three actions.
CartLineItem	line , onQuantityChange , onRemove , notice	Renders the price/stock change notice when present.
OrderSummary	lines , subtotal , shipping , total , primaryAction	Shared by cart, checkout, confirmation and the delegated payment page.
ShippingMethodPicker	methods , value , onChange	Radio list with cost and description.
AddressForm	value , onChange , errors , showCopyFromProfile	Owns the region → district dependency.
PaymentMethodPicker	methods , value , onChange , paymentNumber , onPaymentNumberChange	Reveals the instructions panel and the number field.
PaymentInstructions	amount , reference , networks , language	Renders admin-managed content (§14.16).
OrderTimeline	events , currentStatus	Built from the immutable event log.

Component	Key props	Responsibility
<code>ReviewList</code> , <code>ReviewForm</code> , <code>ReviewSummary</code>	<code>productId</code> , <code>reviews</code> , <code>summary</code>	Histogram, list, composer.
<code>CommentThread</code> , <code>CommentForm</code>	<code>articleId</code> , <code>comments</code>	Moderation-aware.
<code>RichText</code>	<code>html</code>	The single sanitised-HTML renderer. No other component may set <code>dangerouslySetInnerHTML</code> — enforced by lint rule.
<code>SearchBar</code> , <code>SearchSuggestions</code>	<code>value</code> , <code>onChange</code> , <code>onSubmit</code> , <code>suggestions</code>	Full combobox pattern.
<code>MegaMenu</code>	<code>categories</code> , <code>subcategoriesByCategory</code>	Two-panel, keyboard navigable, hides empty branches.
<code>AuthModal</code>	<code>open</code> , <code>mode</code> , <code>nextRoute</code>	Login, register, forgot-password in one shell.

8.3 Layout components

Component	Responsibility
<code>StorefrontLayout</code>	Utility bar, sticky header, breadcrumb slot, main, footer, all global overlays, WhatsApp button, verification banner.
<code>AdminLayout</code>	Fixed sidebar (collapsible to icons), top bar with breadcrumb, global search, notification bell and account menu; content area; permission context.
<code>AccountLayout</code>	Account header and tab rail.
<code>PageHeader</code>	Title, optional subtitle, optional action cluster. Used by every admin page for a consistent top edge.
<code>Section</code>	Storefront section wrapper: heading, optional lead paragraph, consistent vertical rhythm.
<code>Container</code>	Max-width and gutters per §5.6.

8.4 Admin components

8.4.1 `DataTable`

The workhorse of the admin console. One component serves products, orders, invoices, customers, categories, subcategories, shipping methods, banners, campaigns, contacts, searches, comments, reviews, staff and the audit log.

```

interface DataTableProps<T> {
  columns: Column<T>[]           // key, header, render?, sortable?, width?, hideBelow?
  rows: T[]
  rowKey: (row: T) => string

  loading?: boolean              // renders skeleton rows at final height
  error?: Error                  // renders an inline retry panel
  emptyState: EmptyStateProps

  // selection
  selectable?: boolean
  selectedKeys?: string[]
  onSelectionChange?: (keys: string[]) => void
  bulkActions?: BulkAction[]    // label, icon, tone, requiresConfirm, onRun

  // server-driven state – the table never fetches, it only reports
  sort?: { key: string; dir: 'asc' | 'desc' }
  onSortChange?: (s: Sort) => void
  page: number
  pageSize: number
  total: number
  onPageChange: (p: number) => void
  onPageSizeChange?: (n: number) => void

  search?: { value: string; placeholder: string; onChange: (v: string) => void }
  filters?: FilterDescriptor[]
  rowActions?: (row: T) => RowAction[]
  onRowClick?: (row: T) => void
  density?: 'comfortable' | 'compact'
}

```

Behaviour that is uniform everywhere it is used: sticky header on scroll; sortable headers with an aria-sort attribute; a select-all checkbox that selects the current page only, with a "Select all n matching" affordance when more exist; a bulk-action bar that slides in above the table when anything is selected, showing the count and a "Clear" control; search debounced at 300 ms and written to the query string; row actions in a trailing column, collapsing to an overflow menu below `lg`; and horizontal scroll confined to the table's own container so the page never scrolls sideways.

8.4.2 Other admin components

Component	Responsibility
<code>StatCard</code>	Dashboard tile: label, large value, delta versus the previous period with direction and colour, sparkline, optional click-through.
<code>ChartPanel</code>	Chart.js wrapper with a title, a period selector (Last 7 Days / Last 30 Days / Last 12 Months), a loading skeleton, an empty state and an accessible data table alternative behind a "View as table" toggle.
<code>ImageUploader</code>	FilePond wrapper. Drag-and-drop, multiple files, reordering, per-file progress, delete with confirmation, and the accepted-types and size limits from §11.7. Copy: "Drag & drop your files or Browse ". Uploads immediately on drop and reports server ids to the form; never holds base64 in form state.
<code>RichTextEditor</code>	TipTap. Toolbar: bold, italic, underline, H3, H4, bullet list, ordered list, link, blockquote, code block, table, image, undo, redo, clear formatting. Emits HTML constrained to the allowlist in §7.4.2, with a character counter and an HTML source view for administrators who want it.
<code>AttributeMatrix</code>	The variant editor described in §14.3.2 — dynamic attribute columns plus fixed Price and Stock columns, add/remove row, add/remove column, delete table, per-cell validation.
<code>ColorField</code>	Swatch, native colour input and a hex text field kept in sync. Used by the banner builder for its six colour fields.
<code>EntityPicker</code>	Modal search-and-select over a catalogue entity. Used by the invoice builder to add products.

Component	Responsibility
FormLayout	Standard admin form scaffold: section headings, two-column field grid, a sticky footer with Cancel and Submit, unsaved-changes guard, and a disabled submit until the form is dirty and valid.
PermissionGate	Renders children only when the signed-in staff member holds a permission; otherwise renders nothing, or a disabled control with an explanatory tooltip when <code>mode="disable"</code> .
AuditTrail	Compact reverse-chronological list of changes to one entity: who, when, what changed, before and after.

8.5 Contracts every component honours

- **Controlled by default.** Components take `value` and `onChange`. Uncontrolled variants exist only where React Hook Form needs an uncontrolled input for performance.
- **No data fetching in presentation components.** Fetching lives in route loaders and hooks. A component that fetches cannot be reused with different data and cannot be tested without a network mock.
- **Every interactive element has an accessible name.** Enforced by `eslint-plugin-jsx-a11y` and by an axe pass in the component test suite.
- **Loading, empty and error are props, not afterthoughts.** Any component that renders a collection must accept all three; the type system requires `emptyState`.
- **No hard-coded colour, spacing or font values.** Only tokens. Enforced by a stylelint rule.
- **No hard-coded user-facing strings.** All copy is resolved through the i18n layer (§19.8), even while only English ships.
- **Each component ships with a Storybook story per state.** The story set is the visual regression suite (§21.1).

Data Model — MySQL

9.1 Modelling principles

These rules are applied without exception across all 50 tables. Where a rule differs from what the reference platform does, the reason is given — the reference platform stores documents in MongoDB, and a direct transliteration of that shape into MySQL would inherit its defects.

Principle	Rule and rationale
Primary keys	<code>BIGINT UNSIGNED AUTO_INCREMENT</code> . Compact, ordered, and ideal for InnoDB's clustered index. Public identifiers are separate: slugs for catalogue entities, human-readable numbers for orders and invoices, and random tokens for shareable links. A primary key is never exposed in a URL (§6.6).
Public identifiers	Every entity that appears in a URL carries either a <code>slug VARCHAR(96)</code> with a unique index, or a formatted number such as <code>TS-2026-000148</code> . This is what makes defect R1 structurally impossible.
Money	<code>BIGINT</code> holding minor units — for TZS, whole shillings, since TZS has no subunit in practice. Never <code>FLOAT</code> or <code>DOUBLE</code> . Every money column is named with an <code>_amount</code> suffix and is accompanied by a <code>currency CHAR(3)</code> on the owning row wherever the value is customer-facing, so a future multi-currency change is additive.
Timestamps	<code>created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)</code> and <code>updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3)</code> on every table. All stored in UTC ; the application converts to Africa/Dar_es_Salaam (UTC+3) for display. <code>DATETIME</code> not <code>TIMESTAMP</code> , to avoid the 2038 limit.
Soft deletes	<code>deleted_at DATETIME(3) NULL</code> on every entity a customer or an order can reference. A deleted product must still render correctly inside a three-year-old order. Queries filter <code>deleted_at IS NULL</code> by default through a Prisma middleware, with an explicit escape hatch for the admin console's trash view.
Character set	<code>utf8mb4</code> with <code>utf8mb4_0900_ai_ci</code> collation, database-wide. Product names contain <code>@</code> , <code>μ</code> , <code>Ω</code> and emoji appear in reviews; <code>utf8</code> (three-byte) would truncate them.
Engine	InnoDB throughout. Foreign keys and transactional integrity are required by the order and stock logic in Part 12.
Enumerations	<code>VARCHAR</code> with a <code>CHECK</code> constraint, not MySQL's <code>ENUM</code> . Adding a value to a native <code>ENUM</code> rewrites the table; a check constraint is altered cheaply, and the values stay legible in a dump.
Booleans	<code>TINYINT(1) NOT NULL DEFAULT 0</code> . Named affirmatively — <code>is_active</code> , never <code>is_not_disabled</code> .
Nullability	<code>NOT NULL</code> is the default. <code>NULL</code> is used only where "unknown" is genuinely different from "empty" — a guest order's email is <code>NULL</code> ; a product with no description holds <code>''</code> .
Denormalised counters	<code>product_count</code> , <code>order_count</code> , <code>review_count</code> , <code>average_rating</code> are maintained in the same transaction as the fact that changes them, never recomputed on read. The reference platform does the same and it is right — but it must be transactional, or the counts drift.
Snapshots	Orders and invoices store a copy of the name, SKU, price and attributes at the moment of purchase, alongside a nullable reference to the live entity. An order must be reprintable after the product is renamed, repriced or deleted. This is not redundancy; it is the correct model for a financial record.

Principle	Rule and rationale
Naming	snake_case , plural table names, singular column names, foreign keys as <singular_table>_id . Indexes named idx_<table>_<cols> , uniques uq_ , foreign keys fk_<table>_<ref> .

9.2 Entity–relationship overview



9.3 Table-by-table specification

Columns are given as *name · type · constraints · notes*. Every table also carries `created_at` and `updated_at` as defined in §9.1; these are omitted below for brevity and are present in the DDL in Appendix E.

9.3.1 users

Column	Type	Notes
id	BIGINT UNSIGNED PK	
public_id	CHAR(26) UNIQUE	ULID. Used in admin URLs and support conversations, so the sequential primary key is never exposed.
email	VARCHAR(255) NULL UNIQUE	Nullable because a phone-only account is valid. Stored lowercase; uniqueness is case-insensitive by collation.
email_verified_at	DATETIME(3) NULL	Replaces the reference platform's boolean <code>validEmail</code> — knowing <i>when</i> is worth the same storage.

Column	Type	Notes
phone	VARCHAR(15) NULL UNIQUE	Stored canonically as 255XXXXXXXX , digits only, no plus.
phone_verified_at	DATETIME(3) NULL	For a future SMS OTP flow.
username	VARCHAR(80) NOT NULL	Display name. Not unique — two customers may both be "John Mushi".
password_hash	VARCHAR(255) NULL	Argon2id. NULL for OAuth-only accounts.
account_type	VARCHAR(16) NOT NULL DEFAULT 'customer'	CHECK IN ('customer','staff'). Fine-grained authority comes from roles.
status	VARCHAR(16) NOT NULL DEFAULT 'active'	CHECK IN ('active','suspended','closed').
profile_image_id	BIGINT UNSIGNED NULL FK → media_files	ON DELETE SET NULL.
marketing_opt_in	TINYINT(1) NOT NULL DEFAULT 0	Consent for campaigns. Absent from the reference platform, which mails every contact — a compliance risk.
last_login_at	DATETIME(3) NULL	
failed_login_count	SMALLINT UNSIGNED NOT NULL DEFAULT 0	Drives progressive lockout (§11.6).
locked_until	DATETIME(3) NULL	
deleted_at	DATETIME(3) NULL	

Constraints. CHECK (email IS NOT NULL OR phone IS NOT NULL) — an account must be reachable by at least one channel.

Indexes. uq_users_email , uq_users_phone , uq_users_public_id , idx_users_status_created , idx_users_account_type .

9.3.2 user_addresses

Column	Type	Notes
id	BIGINT UNSIGNED PK	
user_id	BIGINT UNSIGNED FK → users	ON DELETE CASCADE.
label	VARCHAR(40) NULL	"Home", "Office".
receiver_name	VARCHAR(120) NOT NULL	Often not the account holder.
email	VARCHAR(255) NULL	
phone	VARCHAR(15) NOT NULL	The courier's contact number.
country	VARCHAR(64) NOT NULL DEFAULT 'Tanzania'	
region	VARCHAR(64) NOT NULL	Denormalised name, not an FK — regions rarely change and the historical name must survive a rename.
district	VARCHAR(64) NOT NULL	
street_address	VARCHAR(255) NOT NULL	
postal_code	VARCHAR(20) NULL	

Column	Type	Notes
<code>is_default</code>	TINYINT(1) NOT NULL DEFAULT 0	Application-enforced: at most one default per user.
<code>deleted_at</code>	DATETIME(3) NULL	

9.3.3 `oauth_accounts`

`id` · `user_id` FK CASCADE · `provider` VARCHAR(20) CHECK IN ('google','facebook') · `provider_user_id` VARCHAR(191) · `email` VARCHAR(255) NULL · `raw_profile` JSON NULL · `linked_at` DATETIME(3). Unique on (`provider`, `provider_user_id`); index on `user_id`. A user may link both providers.

9.3.4 `sessions`

`id` · `user_id` FK CASCADE · `refresh_token_hash` CHAR(64) UNIQUE (SHA-256 — the raw token is never stored) · `user_agent` VARCHAR(255) · `ip_address` VARBINARY(16) · `expires_at` DATETIME(3) · `revoked_at` DATETIME(3) NULL · `replaced_by_id` BIGINT NULL (rotation chain, §11.2). Indexes on (`user_id`, `revoked_at`) and `expires_at` for the cleanup job.

9.3.5 `email_verification_tokens` and `password_reset_tokens`

Identical shape: `id` · `user_id` FK CASCADE · `token_hash` CHAR(64) UNIQUE · `expires_at` DATETIME(3) · `used_at` DATETIME(3) NULL · `requested_ip` VARBINARY(16). Verification tokens live 24 hours, reset tokens 60 minutes. Issuing a new token invalidates all outstanding ones for that user.

9.3.6 `roles`, `permissions`, `role_permissions`, `user_roles`

Table	Columns
<code>roles</code>	<code>id</code> · <code>key</code> VARCHAR(40) UNIQUE · <code>name</code> VARCHAR(80) · <code>description</code> VARCHAR(255) · <code>is_system</code> TINYINT(1) — system roles cannot be deleted.
<code>permissions</code>	<code>id</code> · <code>key</code> VARCHAR(60) UNIQUE (e.g. <code>product.create</code>) · <code>group</code> VARCHAR(40) · <code>description</code> VARCHAR(255).
<code>role_permissions</code>	<code>role_id</code> · <code>permission_id</code> , composite PK, both FK CASCADE.
<code>user_roles</code>	<code>user_id</code> · <code>role_id</code> , composite PK · <code>granted_by</code> BIGINT NULL · <code>granted_at</code> .

Seeded roles: **owner** (all permissions, undeletable, at least one must always exist), **manager** (everything except staff and settings), **catalogue** (products, categories, media), **fulfilment** (orders, invoices, customers read-only), **marketing** (campaigns, contacts, articles, banners), **support** (orders and customers read-only, comment and review moderation).

9.3.7 `categories`

`id` · `name` VARCHAR(120) NOT NULL · `slug` VARCHAR(96) UNIQUE · `sku_code` CHAR(2) UNIQUE (the two-digit prefix, §12.1) · `description` VARCHAR(500) NULL · `image_id` FK → `media_files` NULL · `position` INT NOT NULL DEFAULT 0 (manual ordering) · `subcategory_count` INT UNSIGNED NOT NULL DEFAULT 0 · `product_count` INT UNSIGNED NOT NULL DEFAULT 0 · `is_active` TINYINT(1) NOT NULL DEFAULT 1 · `deleted_at`. Indexes: `uq_categories_slug`, `uq_categories_sku_code`, `idx_categories_position`.

9.3.8 subcategories

id · category_id FK → categories RESTRICT · name VARCHAR(140) · slug VARCHAR(96) UNIQUE · sku_code CHAR(4) UNIQUE · description VARCHAR(500) NULL · image_id NULL · position INT · product_count INT UNSIGNED · is_active TINYINT(1) · deleted_at . Indexes: uq_subcategories_slug , uq_subcategories_sku_code , idx_subcategories_category .

WHY EXACTLY TWO LEVELS

A general tree with a self-referencing parent is more flexible and worse here. The SKU scheme is positional and assumes two levels; the mega-menu is a two-panel design that assumes two levels; and the observed taxonomy — 17 categories, 102 subcategories — has never needed a third. Two explicit tables give simpler queries, cheaper counts and stronger constraints. If a third level is ever required, it is an additive migration, not a redesign.

9.3.9 products (the product group)

Column	Type	Notes
id	BIGINT UNSIGNED PK	
subcategory_id	BIGINT UNSIGNED FK RESTRICT	A subcategory holding products cannot be hard-deleted.
name	VARCHAR(255) NOT NULL	Trimmed and internal whitespace collapsed on write — the direct fix for the dirty names observed in the reference data (R2).
slug	VARCHAR(96) NOT NULL UNIQUE	Generated once; never regenerated (\$6.6).
sku_prefix	CHAR(7) NOT NULL UNIQUE	The group's seven-digit code; variants extend it (\$12.1).
short_description	VARCHAR(500) NULL	Plain text, used for meta descriptions and card tooltips.
description_html	MEDIUMTEXT NOT NULL	Sanitised on write.
description_text	MEDIUMTEXT NOT NULL	Plain-text projection, maintained on write, used by the full-text index — searching HTML matches tag names.
brand	VARCHAR(120) NULL	
manufacturer_part_number	VARCHAR(80) NULL	Indexed. An electronics buyer searches by MPN as often as by name.
display_image_id	BIGINT UNSIGNED NULL FK	The card and hero image.
status	VARCHAR(16) NOT NULL DEFAULT 'draft'	CHECK IN ('draft','active','archived'). The reference platform has no draft state, so a half-entered product is immediately live.
total_stock	INT NOT NULL DEFAULT 0	Maintained sum across variants. Drives the in-stock facet without a join.
min_price_amount	BIGINT NOT NULL DEFAULT 0	Maintained. Drives "From TZS 200" and price-range filtering.
max_price_amount	BIGINT NOT NULL DEFAULT 0	Maintained.
order_count	INT UNSIGNED NOT NULL DEFAULT 0	Lifetime units sold; drives best-seller ordering.
view_count	INT UNSIGNED NOT NULL DEFAULT 0	

Column	Type	Notes
<code>review_count</code>	INT UNSIGNED NOT NULL DEFAULT 0	
<code>average_rating</code>	DECIMAL(3,2) NOT NULL DEFAULT 0	Maintained on review approval.
<code>meta_title</code> , <code>meta_description</code>	VARCHAR(70) / VARCHAR(160) NULL	SEO overrides; fall back to name and short description.
<code>published_at</code>	DATETIME(3) NULL	Drives "NEW" badging independently of <code>created_at</code> .
<code>deleted_at</code>	DATETIME(3) NULL	

Indexes. `uq_products_slug` , `uq_products_sku_prefix` , `idx_products_subcategory_status` , `idx_products_status_published` , `idx_products_order_count` , `idx_products_min_price` , `idx_products_mpn` , and `ft_products_search` — a `FULLTEXT` index on `(name, description_text, brand, manufacturer_part_number)` .

9.3.10 `product_slug_history`

`id` · `product_id` FK CASCADE · `slug` VARCHAR(96) UNIQUE · `retired_at` . Looked up on a 404 to serve a 301 to the current slug. Retired slugs are reserved forever and can never be claimed by another product.

9.3.11 `product_images`

`id` · `product_id` FK CASCADE · `media_file_id` FK RESTRICT · `variant_id` BIGINT NULL FK SET NULL (an image may belong to one specific variant) · `alt_text` VARCHAR(255) NULL · `position` INT NOT NULL DEFAULT 0. Unique on `(product_id, media_file_id)` ; index on `(product_id, position)` .

9.3.12 `product_attributes`

The attribute *columns* a product's variant matrix defines. `id` · `product_id` FK CASCADE · `name` VARCHAR(80) ("Resistance", "Capacity", "Colour") · `position` INT · `unit` VARCHAR(20) NULL ("Ω", "μF", "mm"). Unique on `(product_id, name)` .

9.3.13 `product_variants`

Column	Type	Notes
<code>id</code>	BIGINT UNSIGNED PK	
<code>product_id</code>	BIGINT UNSIGNED FK CASCADE	
<code>sku</code>	VARCHAR(20) NOT NULL UNIQUE	Group prefix plus a two-digit variant suffix (§12.1).
<code>price_amount</code>	BIGINT NOT NULL	<code>CHECK (price_amount > 0)</code> — this constraint alone makes defect R16 impossible.
<code>compare_at_amount</code>	BIGINT NULL	The "was" price, for showing a saving.
<code>cost_amount</code>	BIGINT NULL	Purchase cost. Never exposed to the storefront; drives margin reporting.
<code>currency</code>	CHAR(3) NOT NULL DEFAULT 'TZS'	
<code>stock_quantity</code>	INT NOT NULL DEFAULT 0	<code>CHECK (stock_quantity >= 0)</code> .

Column	Type	Notes
<code>reserved_quantity</code>	INT NOT NULL DEFAULT 0	Held by unpaid orders. Available = stock – reserved (\$12.4).
<code>low_stock_threshold</code>	INT NOT NULL DEFAULT 5	Drives the dashboard alert and the "Only n left" treatment.
<code>weight_grams</code>	INT NULL	For future weight-based shipping.
<code>barcode</code>	VARCHAR(64) NULL	
<code>position</code>	INT NOT NULL DEFAULT 0	Display order in the selector.
<code>is_active</code>	TINYINT(1) NOT NULL DEFAULT 1	
<code>deleted_at</code>	DATETIME(3) NULL	

Indexes. `uq_product_variants_sku`, `idx_variants_product_active`, `idx_variants_stock`, `idx_variants_price`.

EVERY PRODUCT HAS AT LEAST ONE VARIANT

A product with no options still gets a single variant row. Uniform structure removes every "does this product have variants?" branch from cart, order, stock and reporting code. The storefront simply hides the selector when the count is one. This is the structural fix for defect R2.

9.3.14 `variant_attribute_values`

`variant_id` FK CASCADE · `product_attribute_id` FK CASCADE · `value` VARCHAR(120) NOT NULL. Composite PK on (`variant_id`, `product_attribute_id`); index on (`product_attribute_id`, `value`) for future attribute faceting.

9.3.15 `tags` and `product_tags`

`tags`: `id` · `name` VARCHAR(60) · `slug` VARCHAR(60) UNIQUE. `product_tags`: `product_id` · `tag_id`, composite PK, both CASCADE. Tags feed search boosting and curated collections. The reference platform has a `tags` array on every product and never populates it; here it is at least usable.

9.3.16 `discounts`

`id` · `scope` VARCHAR(20) CHECK IN ('product','variant','subcategory','category') · `scope_id` BIGINT UNSIGNED · `type` VARCHAR(16) CHECK IN ('percentage','fixed') · `value` INT NOT NULL (percent 1–90, or minor units) · `starts_at` DATETIME(3) NOT NULL · `ends_at` DATETIME(3) NOT NULL · `is_active` TINYINT(1) · `created_by` FK → users. Index on (`scope`, `scope_id`, `is_active`, `starts_at`, `ends_at`). CHECK (`ends_at` > `starts_at`). Resolution order when several apply is defined in §12.3.

9.3.17 `media_files`

Column	Type	Notes
<code>id</code>	BIGINT UNSIGNED PK	
<code>public_id</code>	CHAR(26) UNIQUE	ULID; forms the served filename.
<code>storage_driver</code>	VARCHAR(20) NOT NULL	'local' 's3' 'cloudinary'.
<code>storage_key</code>	VARCHAR(512) NOT NULL	Path or key within the driver.
<code>original_filename</code>	VARCHAR(255)	Sanitised; never used as a path component.
<code>mime_type</code>	VARCHAR(80)	Determined by magic-byte sniffing, not by the client-supplied header (§11.7).

Column	Type	Notes
<code>byte_size</code>	INT UNSIGNED	
<code>width</code> , <code>height</code>	INT UNSIGNED NULL	Used to reserve layout space and prevent shift.
<code>checksum_sha256</code>	CHAR(64)	Indexed — identical uploads are deduplicated to one stored file.
<code>derivatives</code>	JSON NOT NULL	<code>{"sm":{...}, "md":{...}, "lg":{...}}</code> each with key, width, height and byte size.
<code>uploaded_by</code>	BIGINT UNSIGNED NULL FK SET NULL	
<code>reference_count</code>	INT UNSIGNED NOT NULL DEFAULT 0	Maintained. Zero-reference files older than 24 hours are swept (§15.4).
<code>deleted_at</code>	DATETIME(3) NULL	

9.3.18 `carts` and `cart_items`

Table	Columns
<code>carts</code>	<code>id</code> · <code>user_id</code> BIGINT NULL FK CASCADE · <code>guest_token</code> CHAR(32) NULL UNIQUE · <code>currency</code> CHAR(3) · <code>last_activity_at</code> DATETIME(3) · <code>converted_order_id</code> BIGINT NULL FK SET NULL. <code>CHECK (user_id IS NOT NULL OR guest_token IS NOT NULL)</code> . Unique on <code>user_id</code> where not null — one open cart per account.
<code>cart_items</code>	<code>id</code> · <code>cart_id</code> FK CASCADE · <code>product_variant_id</code> FK RESTRICT · <code>quantity</code> INT <code>CHECK (quantity > 0)</code> · <code>unit_price_amount</code> BIGINT — the price seen when added, kept only to <i>detect</i> a change, never to charge against · <code>added_at</code> . Unique on <code>(cart_id, product_variant_id)</code> so adding an existing line increments rather than duplicating.

Abandoned guest carts are purged after 30 days of inactivity; signed-in carts are kept indefinitely. Cart persistence server-side is what lets a customer start on a phone and finish on a laptop — the reference platform keeps carts only in `localStorage` and posts them at checkout, so that journey loses everything.

9.3.19 `wishlists` and `wishlist_items`

`wishlists`: `id` · `user_id` FK CASCADE UNIQUE · `guest_token` CHAR(32) NULL UNIQUE. `wishlist_items`: `id` · `wishlist_id` FK CASCADE · `product_id` FK CASCADE · `product_variant_id` BIGINT NULL FK SET NULL · `added_at` . Unique on `(wishlist_id, product_id, product_variant_id)` . No quantity column.

9.3.20 `shipping_methods`

Column	Type	Notes
<code>id</code>	BIGINT UNSIGNED PK	
<code>name</code>	VARCHAR(80) NOT NULL	"Savings", "Cash on delivery", "Office Pickup".
<code>slug</code>	VARCHAR(60) UNIQUE	
<code>description</code>	VARCHAR(500) NOT NULL	Shown beneath the option at checkout, not hidden in a tooltip.
<code>cost_amount</code>	BIGINT NOT NULL DEFAULT 0	<code>CHECK (cost_amount >= 0)</code> .

Column	Type	Notes
<code>requires_shipping_address</code>	TINYINT(1) NOT NULL DEFAULT 1	Office pickup does not.
<code>accepts_mobile_payment</code>	TINYINT(1) NOT NULL DEFAULT 1	Cash on delivery does not — this flag is what hides the payment block at checkout.
<code>is_cash_on_delivery</code>	TINYINT(1) NOT NULL DEFAULT 0	Explicit rather than inferred from the name, so the order flow does not depend on a string.
<code>free_over_amount</code>	BIGINT NULL	Free shipping above this subtotal. Not present on the reference platform; a cheap, effective lever.
<code>estimated_days_min/max</code>	SMALLINT NULL	Renders "2–5 working days".
<code>available_regions</code>	JSON NULL	<code>NULL</code> means everywhere. Lets an upcountry method be hidden for Dar es Salaam addresses.
<code>position</code> , <code>is_active</code> , <code>deleted_at</code>		

9.3.21 orders

Column	Type	Notes
<code>id</code>	BIGINT UNSIGNED PK	
<code>order_number</code>	VARCHAR(24) NOT NULL UNIQUE	<code>TS-YYYY-0000000</code> , sequence per year. Human-quotable over the phone.
<code>user_id</code>	BIGINT UNSIGNED NULL FK SET NULL	<code>NULL</code> for guest orders. The order survives account deletion.
<code>status</code>	VARCHAR(24) NOT NULL	CHECK against the state machine in §12.7.
<code>payment_status</code>	VARCHAR(24) NOT NULL	CHECK IN ('unpaid','pending_verification','paid','partially_refunded','refunded','failed').
<code>fulfilment_status</code>	VARCHAR(24) NOT NULL	CHECK IN ('unfulfilled','processing','shipped','delivered','returned').
<code>customer_name</code>	VARCHAR(120) NOT NULL	Snapshot. Never joined from <code>users</code> for display.
<code>customer_email</code>	VARCHAR(255) NULL	
<code>customer_phone</code>	VARCHAR(15) NOT NULL	The one field every order must have — it is how the store contacts the buyer.
<code>shipping_method_id</code>	BIGINT UNSIGNED NULL FK SET NULL	
<code>shipping_method_name</code>	VARCHAR(80) NOT NULL	Snapshot.
<code>subtotal_amount</code>	BIGINT NOT NULL	Sum of line totals.
<code>discount_amount</code>	BIGINT NOT NULL DEFAULT 0	
<code>shipping_amount</code>	BIGINT NOT NULL DEFAULT 0	

Column	Type	Notes
<code>tax_amount</code>	BIGINT NOT NULL DEFAULT 0	Zero at launch; the column exists so adding VAT is not a migration of every historical row.
<code>total_amount</code>	BIGINT NOT NULL	Stored, not computed on read — the total charged must never change because a formula changed.
<code>paid_amount</code>	BIGINT NOT NULL DEFAULT 0	Sum of settled payments. Supports partial payment.
<code>currency</code>	CHAR(3) NOT NULL	
<code>payment_method</code>	VARCHAR(24) NULL	CHECK IN ('lipa_namba','cash','aggregator').
<code>payment_number</code>	VARCHAR(15) NULL	The mobile-money number the buyer will pay from — the key to manual reconciliation (\$13.6).
<code>delegated_payer_email</code>	VARCHAR(255) NULL	Set when a third party will pay.
<code>payment_token_hash</code>	CHAR(64) NULL UNIQUE	Backs <code>/pay/:token</code> .
<code>payment_token_expires_at</code>	DATETIME(3) NULL	
<code>idempotency_key</code>	CHAR(36) NULL UNIQUE	Prevents duplicate orders (\$7.7.3).
<code>customer_note</code>	VARCHAR(500) NULL	
<code>internal_note</code>	TEXT NULL	Staff only; never returned to a customer-facing endpoint.
<code>cancelled_at</code> , <code>cancel_reason</code>	DATETIME(3) / VARCHAR(255) NULL	
<code>placed_at</code> , <code>paid_at</code> , <code>shipped_at</code> , <code>delivered_at</code>	DATETIME(3) NULL	Denormalised from the event log for cheap reporting.
<code>source</code>	VARCHAR(20) NOT NULL DEFAULT 'web'	'web' 'admin' 'phone'.

Indexes. `uq_orders_order_number`, `idx_orders_user_placed`, `idx_orders_status_placed`, `idx_orders_payment_status`, `idx_orders_customer_phone`, `idx_orders_placed_at`, `uq_orders_idempotency_key`, `uq_orders_payment_token_hash`.

THREE STATUSES, NOT ONE

The reference platform carries a single order status ("Completed", "Pending", "Cancelled"), which cannot express the ordinary case of an order that is paid but not yet shipped, or shipped but not yet paid for on delivery. Splitting lifecycle, payment and fulfilment into three independent axes is what makes the admin console usable and the reports truthful.

9.3.22 `order_items`

`id` · `order_id` FK CASCADE · `product_id` BIGINT NULL FK SET NULL · `product_variant_id` BIGINT NULL FK SET NULL · `product_name` VARCHAR(255) *snapshot* · `variant_sku` VARCHAR(20) *snapshot* · `variant_attributes` JSON *snapshot* — e.g. [{"name": "Resistance", "value": "10kΩ"}] · `image_url` VARCHAR(512) NULL *snapshot* · `unit_price_amount` BIGINT · `quantity` INT CHECK (> 0) · `discount_amount` BIGINT DEFAULT 0 · `line_total_amount` BIGINT. Index on `order_id` and on `product_id` for "who bought this" reporting.

9.3.23 order_shipping_addresses

One row per order, a frozen copy: `order_id` FK CASCADE UNIQUE · `receiver_name` · `email` · `phone` · `country` · `region` · `district` · `street_address` · `postal_code`. Absent for pickup orders. Kept in its own table rather than as columns on `orders` so that the order row stays narrow and the address can be omitted cleanly.

9.3.24 order_events

The append-only audit trail of an order's life. `id` · `order_id` FK CASCADE · `event_type` VARCHAR(40) ('placed', 'payment_recorded', 'payment_verified', 'status_changed', 'note_added', 'cancelled', 'refunded', 'shipped', 'delivered', 'email_sent', 'sms_sent') · `from_value`, `to_value` VARCHAR(40) NULL · `actor_type` VARCHAR(16) ('customer', 'staff', 'system', 'webhook') · `actor_user_id` BIGINT NULL FK SET NULL · `message` VARCHAR(500) NULL · `metadata` JSON NULL · `created_at`. Index on (`order_id`, `created_at`). **Never updated, never deleted.** This table is the source for the customer timeline (§7.9.2), the admin history panel and every dispute.

9.3.25 payments and payment_events

Table	Columns
payments	<code>id</code> · <code>order_id</code> FK RESTRICT · <code>method</code> VARCHAR(24) · <code>provider</code> VARCHAR(40) NULL ('manual', 'selcom', 'clickpesa', 'azampay', 'dpo') · <code>amount</code> BIGINT · <code>currency</code> CHAR(3) · <code>status</code> VARCHAR(24) CHECK IN ('initiated', 'pending', 'succeeded', 'failed', 'cancelled', 'refunded') · <code>payer_phone</code> VARCHAR(15) NULL · <code>payer_name</code> VARCHAR(120) NULL · <code>provider_reference</code> VARCHAR(120) NULL — the mobile-money transaction id · <code>merchant_reference</code> VARCHAR(64) — our own reference sent to the provider · <code>verified_by</code> BIGINT NULL FK — the staff member who confirmed a manual payment · <code>verified_at</code> · <code>failure_reason</code> VARCHAR(255) NULL · <code>raw_payload</code> JSON NULL. Unique on (<code>provider</code> , <code>provider_reference</code>) where both are non-null — the single constraint that makes double-crediting impossible.
payment_events	<code>id</code> · <code>payment_id</code> BIGINT NULL FK CASCADE · <code>provider</code> · <code>event_type</code> · <code>external_event_id</code> VARCHAR(120) UNIQUE — webhook idempotency · <code>signature_valid</code> TINYINT(1) · <code>payload</code> JSON · <code>processed_at</code> DATETIME(3) NULL · <code>processing_error</code> VARCHAR(500) NULL. Every inbound webhook is written here <i>before</i> it is acted upon, so a mis-processed callback can be replayed (§13.5).

9.3.26 stock_movements

Every change to stock, ever. `id` · `product_variant_id` FK RESTRICT · `delta` INT NOT NULL (signed) · `balance_after` INT NOT NULL · `reason` VARCHAR(32) CHECK IN ('purchase', 'sale', 'reservation', 'release', 'adjustment', 'return', 'damage', 'stocktake') · `reference_type` VARCHAR(32) NULL · `reference_id` BIGINT NULL · `actor_user_id` BIGINT NULL FK SET NULL · `note` VARCHAR(255) NULL · `created_at`. Index on (`product_variant_id`, `created_at`). Stock is never adjusted without a corresponding movement row, in the same transaction — which is what makes "why is this figure wrong?" an answerable question.

9.3.27 invoices and invoice_items

`invoices`: `id` · `invoice_number` VARCHAR(24) UNIQUE (INV-`{YYYY}-{000000}`) · `order_id` BIGINT NULL FK SET NULL — an invoice may be raised without an order, which is exactly the quotation case (persona Grace) · `client_name` VARCHAR(120) · `client_email` VARCHAR(255) · `client_phone` VARCHAR(15) NULL · `client_address` VARCHAR(500) NULL · `shipping_method_name` VARCHAR(80) NULL · `subtotal_amount` · `shipping_amount` · `tax_amount` · `total_amount` · `currency` · `status` VARCHAR(20) CHECK IN ('draft', 'issued', 'paid', 'void') · `issued_at` · `due_at` NULL · `paid_at` NULL · `share_token_hash` CHAR(64) UNIQUE NULL · `notes` TEXT NULL · `created_by` FK → users · `deleted_at`.

`invoice_items`: `id` · `invoice_id` FK CASCADE · `product_variant_id` BIGINT NULL FK SET NULL · `description` VARCHAR(255) · `sku` VARCHAR(20) NULL · `unit_price_amount` BIGINT · `quantity` INT · `line_total_amount` BIGINT ·

`position` INT. Free-text lines are permitted so an invoice can carry a service charge that is not a catalogue product.

9.3.28 Content tables

Table	Columns
<code>banners</code>	<code>id</code> · <code>title</code> VARCHAR(120) · <code>title_color</code> CHAR(7) · <code>subtitle</code> VARCHAR(300) NULL · <code>subtitle_color</code> CHAR(7) · <code>background_color</code> CHAR(7) · <code>image_id</code> FK RESTRICT · <code>link_url</code> VARCHAR(512) NULL · <code>button_text</code> VARCHAR(40) NULL · <code>button_background</code> CHAR(7) · <code>button_text_color</code> CHAR(7) · <code>placement</code> VARCHAR(20) DEFAULT 'home' · <code>position</code> INT · <code>starts_at</code> , <code>ends_at</code> DATETIME(3) NULL — scheduled banners, which the reference platform lacks · <code>is_active</code> · <code>deleted_at</code> .
<code>services</code>	<code>id</code> · <code>heading</code> VARCHAR(160) · <code>slug</code> UNIQUE · <code>excerpt</code> VARCHAR(500) NULL · <code>content_html</code> MEDIUMTEXT · <code>content_text</code> MEDIUMTEXT · <code>image_id</code> FK · <code>position</code> · <code>is_active</code> · <code>published_at</code> · <code>deleted_at</code> .
<code>partners</code>	<code>id</code> · <code>name</code> VARCHAR(120) · <code>logo_id</code> FK · <code>website_url</code> VARCHAR(512) · <code>position</code> · <code>is_active</code> · <code>deleted_at</code> .
<code>articles</code>	<code>id</code> · <code>title</code> VARCHAR(200) · <code>slug</code> UNIQUE · <code>excerpt</code> VARCHAR(500) · <code>content_html</code> MEDIUMTEXT · <code>content_text</code> MEDIUMTEXT · <code>cover_image_id</code> FK · <code>author_id</code> FK → users · <code>status</code> CHECK IN ('draft','published','archived') · <code>published_at</code> · <code>view_count</code> · <code>comment_count</code> · <code>meta_title</code> , <code>meta_description</code> · <code>deleted_at</code> . FULLTEXT on (<code>title</code> , <code>content_text</code>) .
<code>comments</code>	<code>id</code> · <code>article_id</code> FK CASCADE · <code>user_id</code> FK SET NULL · <code>parent_id</code> BIGINT NULL FK CASCADE (one level of replies) · <code>author_name</code> VARCHAR(120) <code>snapshot</code> · <code>body</code> TEXT · <code>status</code> CHECK IN ('pending','approved','rejected','spam') · <code>moderated_by</code> , <code>moderated_at</code> · <code>ip_address</code> VARBINARY(16) · <code>deleted_at</code> .
<code>reviews</code>	<code>id</code> · <code>product_id</code> FK CASCADE · <code>user_id</code> FK SET NULL · <code>order_item_id</code> BIGINT NULL FK SET NULL — presence of this is what earns the "Verified purchase" badge · <code>rating</code> TINYINT CHECK (<code>rating</code> BETWEEN 1 AND 5) · <code>title</code> VARCHAR(80) NULL · <code>body</code> TEXT · <code>status</code> as comments · <code>helpful_count</code> INT · <code>moderated_by</code> , <code>moderated_at</code> · <code>deleted_at</code> . Unique on (<code>product_id</code> , <code>user_id</code>) — one review per customer per product.
<code>faqs</code>	<code>id</code> · <code>question</code> VARCHAR(300) · <code>answer_html</code> TEXT · <code>group</code> VARCHAR(60) · <code>position</code> · <code>is_active</code> .
<code>site_settings</code>	<code>key</code> VARCHAR(80) PK · <code>value</code> JSON · <code>group</code> VARCHAR(40) · <code>description</code> VARCHAR(255) · <code>updated_by</code> FK. A typed key–value store, read once at boot and cached in Redis. Holds store identity, contact details, social links, page hero image ids, payment till numbers and instruction blocks, currency configuration, and feature flags.
<code>regions / districts</code>	<code>regions</code> : <code>id</code> · <code>name</code> VARCHAR(64) UNIQUE · <code>code</code> · <code>position</code> . <code>districts</code> : <code>id</code> · <code>region_id</code> FK CASCADE · <code>name</code> VARCHAR(64) · <code>position</code> . Unique on (<code>region_id</code> , <code>name</code>) . Seeded from Appendix F.

9.3.29 Marketing and analytics tables

Table	Columns
<code>contacts</code>	<code>id</code> · <code>user_id</code> BIGINT NULL FK SET NULL · <code>name</code> VARCHAR(120) · <code>email</code> VARCHAR(255) NULL · <code>phone</code> VARCHAR(15) NULL · <code>source</code> VARCHAR(24) ('registration','order','manual','import','contact_form') · <code>email_opt_in</code> , <code>sms_opt_in</code> TINYINT(1) · <code>unsubscribed_at</code> DATETIME(3) NULL · <code>tags</code> JSON · <code>deleted_at</code> . Unique on <code>email</code> and on <code>phone</code> where non-null. A marketing audience distinct from the user table, because a contact may come from a form or an import and never hold an account.
<code>campaigns</code>	<code>id</code> · <code>channel</code> CHECK IN ('sms','email') · <code>name</code> VARCHAR(160) — the subject line for email · <code>audience</code> CHECK IN ('all','specific','tag','segment') · <code>audience_filter</code> JSON NULL · <code>body_text</code> TEXT — SMS body or email plain-text part · <code>body_html</code> MEDIUMTEXT NULL · <code>additional_recipients</code> JSON NULL — ad-hoc numbers or addresses typed in the composer · <code>scheduled_for</code> DATETIME(3) NULL · <code>status</code> CHECK IN ('draft','scheduled','sending','sent','failed','cancelled') · <code>recipient_count</code> , <code>sent_count</code> , <code>failed_count</code> , <code>delivered_count</code> INT · <code>cost_amount</code> BIGINT NULL — SMS is metered, so the spend belongs on the record · <code>created_by</code> FK · <code>sent_at</code> · <code>deleted_at</code> .

Table	Columns
campaign_recipients	id · campaign_id FK CASCADE · contact_id BIGINT NULL FK SET NULL · destination VARCHAR(255) — the address or number actually used · status CHECK IN ('queued','sent','delivered','failed','bounced','unsubscribed') · provider_message_id VARCHAR(120) NULL · error VARCHAR(255) NULL · sent_at, delivered_at . Index on (campaign_id, status) .
search_logs	id · term VARCHAR(160) · normalised_term VARCHAR(160) — lowercase, trimmed, whitespace collapsed; this is what is aggregated · result_count INT · user_id BIGINT NULL · session_id CHAR(32) · clicked_product_id BIGINT NULL FK SET NULL — the click-through that makes success measurable · created_at . Index on (normalised_term, created_at) and on (result_count, created_at) to find zero-result searches fast.
contact_messages	id · name · email · subject VARCHAR(200) NULL · message TEXT · send_copy TINYINT(1) · status CHECK IN ('new','read','replied','archived','spam') · handled_by FK NULL · ip_address · user_agent .
notify_requests	id · product_variant_id FK CASCADE · email VARCHAR(255) · user_id BIGINT NULL · notified_at DATETIME(3) NULL. Unique on (product_variant_id, email) . Backs "Notify me when available" — and doubles as a demand signal for restocking.

9.3.30 audit_logs

id · actor_user_id BIGINT NULL FK SET NULL · actor_email VARCHAR(255) snapshot · action VARCHAR(60) ('product.create', 'order.status_change', 'user.suspend') · entity_type VARCHAR(60) · entity_id BIGINT NULL · entity_label VARCHAR(255) — a human-readable name captured at the time · before JSON NULL · after JSON NULL · ip_address VARBINARY(16) · user_agent VARCHAR(255) · request_id CHAR(26) · created_at . Indexes on (entity_type, entity_id, created_at), (actor_user_id, created_at) and (action, created_at) . Append-only, enforced by database privileges: the application's runtime user is granted `INSERT` and `SELECT` on this table and nothing else. Retained 24 months.

9.4 Indexing strategy

Indexes are specified for the queries that actually run, listed here with the query each serves. Anything not on this list is speculative and should not be created.

Query	Index
Catalogue grid, subcategory scope, sorted by newest	products(subcategory_id, status, published_at DESC)
Catalogue grid, sorted by best selling	products(status, order_count DESC)
Catalogue grid, price filter	products(status, min_price_amount)
Product detail by slug	products(slug) unique
Retired-slug redirect	product_slug_history(slug) unique
Keyword search	FULLTEXT products(name, description_text, brand, manufacturer_part_number) ; plus product_variants(sku) unique for the exact-SKU fast path
Variant lookup for a product	product_variants(product_id, is_active, position)
Low-stock dashboard alert	product_variants(stock_quantity) — scanned with a threshold predicate
Customer order history	orders(user_id, placed_at DESC)
Admin order queue by status	orders(status, placed_at DESC)

Query	Index
Payment reconciliation by phone	<code>orders(customer_phone) , payments(payer_phone)</code>
Revenue over a period	<code>orders(payment_status, paid_at)</code>
Order timeline	<code>order_events(order_id, created_at)</code>
Stock ledger for one variant	<code>stock_movements(product_variant_id, created_at DESC)</code>
Approved reviews for a product	<code>reviews(product_id, status, created_at DESC)</code>
Zero-result searches	<code>search_logs(result_count, created_at)</code>
Search-term aggregation	<code>search_logs(normalised_term, created_at)</code>
Campaign delivery report	<code>campaign_recipients(campaign_id, status)</code>
Audit history for one entity	<code>audit_logs(entity_type, entity_id, created_at DESC)</code>
Session validation and cleanup	<code>sessions(refresh_token_hash) unique, sessions(expires_at)</code>

FULL-TEXT CONFIGURATION

MySQL's default full-text minimum token length is 3, which silently drops two-character searches. Electronics part numbers are full of short tokens. Set `innodb_ft_min_token_size = 2` and add an `innodb_ft_user_stopword_table` that is empty, because the default stopwords list removes real terms. Verify after any server upgrade — a reverted setting fails silently and shows up only as poor search results.

9.5 Referential integrity and delete semantics

Relationship	On delete	Reasoning
category → subcategories	RESTRICT	A category with subcategories cannot be removed. This is the direct fix for defect R12: the reference platform cascades a category delete through its subcategories and into every one of their products.
subcategory → products	RESTRICT	Same reasoning. The admin console offers "move products to another subcategory" as the supported path.
product → variants, images, attributes	CASCADE	These have no meaning apart from their product.
variant → order_items	SET NULL	Historical orders survive; the snapshot columns keep them readable.
variant → cart_items	RESTRICT	Forces the deletion path to clear affected carts explicitly and notify their owners, rather than silently emptying a cart.
user → orders	SET NULL	Financial records outlive accounts. The snapshot fields preserve who bought what.
user → sessions, tokens, addresses, cart	CASCADE	Personal data goes with the account.
order → items, events, address	CASCADE	Only relevant if an order is ever hard-deleted, which the application never does.
order → payments	RESTRICT	An order with a recorded payment cannot be deleted, under any circumstances.
media_file → any reference	RESTRICT	A referenced file cannot be deleted; the reference count enforces it before the constraint has to.
article → comments	CASCADE	
campaign → recipients	CASCADE	

SOFT DELETE IS THE DEFAULT; HARD DELETE IS AN OPERATION

Every customer-facing and administrator-facing delete sets `deleted_at`. Rows are removed from the database only by a scheduled purge — 90 days for catalogue entities, never for anything with a financial or audit dimension. The admin console gains a "Trash" view per entity with restore. This one rule converts the reference platform's most dangerous action, a bulk category delete that destroys a swathe of catalogue with no recovery, into a reversible mistake.

9.6 Full DDL

The complete, executable `CREATE TABLE` script — all 50 tables with every column, constraint, index and foreign key — is **Appendix E**. It is written to run against a clean MySQL 8.0 schema in a single pass, ordered so that no foreign key references a table that does not yet exist.

REST API Specification

10.1 Conventions

Rule	Specification
Base path	<code>/api/v1</code> . Versioned from day one — the reference platform's unversioned <code>/api</code> leaves no way to change a response shape without breaking every cached client.
Content type	<code>application/json; charset=utf-8</code> , except upload endpoints which accept <code>multipart/form-data</code> .
Casing	<code>camelCase</code> in JSON; <code>snake_case</code> in the database. The mapping happens once, in the serialisation layer.
Identifiers	Public identifiers only — slugs, order numbers, ULIDs. Numeric primary keys are never returned.
Money	Returned as an object: <code>{"amount": 35000, "currency": "TZS", "formatted": "TZS 35,000"}</code> . The server formats, so a price never renders differently in two places.
Timestamps	ISO 8601 with an explicit offset: <code>2026-08-19T14:32:05.123Z</code> . Always UTC.
Status codes	<code>200</code> read or update · <code>201</code> create, with a <code>Location</code> header · <code>204</code> delete · <code>400</code> malformed · <code>401</code> unauthenticated · <code>403</code> unauthorised · <code>404</code> absent · <code>409</code> conflict · <code>410</code> gone · <code>422</code> validation · <code>429</code> rate limited · <code>500</code> server.
Success assertion	Clients must test <code>response.ok</code> — the 2xx class — never a specific code. Fixes defect R8.
Collections	Always an envelope, never a bare array: <code>{"data": [...], "meta": {"page": 1, "pageSize": 24, "total": 617, "pageCount": 26}, "links": {"next": "...", "prev": null}}</code> . The reference platform returns bare arrays, which means a paginated response can never be introduced without a breaking change.
Single resources	<code>{"data": {...}}</code> . Same reason.
Sparse fields	<code>?fields=id,name,price</code> on read endpoints for expensive lists.
Expansion	<code>?include=variants,images,reviews</code> . Each endpoint documents which expansions it accepts; unknown values are a 422, never silently ignored.
Sorting	<code>?sort=-createdAt,name</code> . A leading minus is descending. Only whitelisted fields.
Pagination	<code>?page=</code> and <code>?pageSize=</code> , default 24, maximum 100. Cursor pagination (<code>?cursor=</code>) additionally on <code>audit_logs</code> , <code>order_events</code> and <code>search_logs</code> , where offsets get expensive.
Idempotency	<code>Idempotency-Key</code> header honoured on <code>POST /orders</code> and <code>POST /payments</code> . Replays within 24 hours return the original response.
Correlation	Every response carries <code>X-Request-Id</code> , echoed into logs and shown on error screens.
Deprecation	Retiring an endpoint sends <code>Deprecation</code> and <code>Sunset</code> headers for at least 90 days before removal.

10.2 Error format

One shape for every error, at every status code.

```

{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Some fields need attention.",
    "requestId": "01JBQ7X2K9P3M4N5Q6R7S8T9V0",
    "details": [
      { "field": "phone", "code": "invalid_length",
        "message": "Phone number must be 12 characters long" },
      { "field": "items[2].quantity", "code": "insufficient_stock",
        "message": "Only 2 available", "meta": { "available": 2 } }
    ]
  }
}

```

Code	Status	Raised when
VALIDATION_FAILED	422	Zod parse failed. <code>details</code> lists every failing field, not just the first.
UNAUTHENTICATED	401	Missing, malformed or expired access token.
FORBIDDEN	403	Authenticated but lacking the required permission.
NOT_FOUND	404	No such resource, or one the caller may not see. Never distinguishes the two.
GONE	410	Soft-deleted resource that used to be public.
CONFLICT	409	Unique constraint or a state-machine violation.
PRICE_CHANGED	409	Checkout revalidation found a different price. <code>details</code> carries old and new per line.
INSUFFICIENT_STOCK	409	Checkout revalidation found too little stock. <code>details</code> carries the available quantity per line.
INVALID_STATE_TRANSITION	409	Order status change not permitted by §12.7.
TOKEN_INVALID	400	Verification, reset, payment or invoice token bad, used or expired — one code for all four cases, deliberately.
RATE_LIMITED	429	With a <code>Retry-After</code> header.
PAYLOAD_TOO_LARGE	413	Body or upload over the cap.
UNSUPPORTED_MEDIA_TYPE	415	Upload MIME type not on the allowlist.
INTERNAL_ERROR	500	Message is always generic. Details go to logs, never to the client.

10.3 Authentication endpoints

Verb	Path	Access	Behaviour
POST	/auth/register	PUBLIC	Body <code>{username, email?, phone?, password, acceptedTerms}</code> . At least one of email or phone required. Creates the user, issues tokens, queues a verification email. 201.
POST	/auth/login	PUBLIC	Body <code>{identifier, password}</code> where identifier is an email or a phone. Sets the refresh cookie, returns the access token and profile. Rate limited per §10.9.
POST	/auth/refresh	cookie	Rotates the refresh token, returns a new access token. Reuse of a rotated token revokes the whole family (§11.2).
POST	/auth/logout	TOKEN	Revokes the current session and clears the cookie. 204.
POST	/auth/logout-all	TOKEN	Revokes every session for the user.

Verb	Path	Access	Behaviour
GET	/auth/oauth/:provider	PUBLIC	302 to the provider with PKCE and a state parameter.
GET	/auth/oauth/:provider/callback	PUBLIC	Validates state, exchanges the code, links or creates the user, then 302 to the SPA with a one-time exchange code — never with a token in the URL, which would leak through referrers and history.
POST	/auth/oauth/exchange	PUBLIC	Swaps the one-time code for a session.
POST	/auth/email/verify/request	TOKEN	Re-sends verification. Limited to 3 per hour.
POST	/auth/email/verify	PUBLIC	Body {token} . Marks verified.
POST	/auth/password/forgot	PUBLIC	Always 200 with the same body, whether or not the address exists.
POST	/auth/password/reset	PUBLIC	Body {token, password} . Revokes all sessions.
PATCH	/auth/password	TOKEN	Body {currentPassword, newPassword} . Revokes other sessions.
GET	/me	TOKEN	Current profile, roles and permission keys.
PATCH	/me	TOKEN	Body {username?, email?, phone?, marketingOptIn?} . Changing email clears verification and re-sends.
POST	/me/avatar	TOKEN	Multipart, field file .
GET POST	/me/addresses	TOKEN	List and create.
PATCH DELETE	/me/addresses/:id	TOKEN	Update and soft-delete.

10.3.1 Login response

```

200 OK
Set-Cookie: ts_rt=<opaque>; HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth; Max-Age=2592000
{
  "data": {
    "accessToken": "eyJhbGciOi...",
    "expiresIn": 900,
    "user": {
      "id": "01JBQ7X2K9P3M4N5Q6R7S8T9V0",
      "username": "Amina Mushi",
      "email": "amina@example.com",
      "emailVerified": false,
      "phone": "255712345678",
      "accountType": "customer",
      "profileImage": { "sm": "...", "md": "...", "lg": "..."},
      "roles": [],
      "permissions": []
    }
  }
}

```

10.4 Catalogue endpoints

Verb	Path	Access	Behaviour
GET	/categories	PUBLIC	?include=subcategories , ?activeOnly=true . Cached 5 minutes at the edge and invalidated on write.
GET	/categories/:slug	PUBLIC	With its subcategories and counts.
GET	/subcategories	PUBLIC	?categorySlug= .
GET	/products	PUBLIC	The catalogue feed. Query: page , pageSize , sort (bestSelling newest priceAsc priceDesc random), categorySlug , subcategorySlug , minPrice , maxPrice , inStock , onOffer , tag , ids .
GET	/products/:slug	PUBLIC	Full detail with variants, attributes, images and review summary. 410 when soft-deleted; 301 hint when the slug is retired.
GET	/products/:slug/related	PUBLIC	Up to 12 from the same subcategory, by sales.
GET	/products/:slug/reviews	PUBLIC	Approved only, paginated, with the rating histogram.
POST	/products/:slug/reviews	TOKEN	Body {rating, title?, body} . Created as pending . 409 if the user has already reviewed.
PATCH DELETE	/reviews/:id	TOKEN	Own review only, or with review.moderate .
GET	/search	PUBLIC	?q= plus every catalogue filter. Logs the term with its result count. ?suggest=true returns the lightweight 8-item shape for the header dropdown and does not log.
GET	/locations/regions	PUBLIC	Cached 24 hours.
GET	/locations/regions/:region/districts	PUBLIC	Cached 24 hours.
GET	/shipping-methods	PUBLIC	?region= filters to methods available there.
GET	/settings/public	PUBLIC	Store name, contact details, social links, currency configuration, payment instruction blocks, feature flags. One request instead of the reference platform's several.

10.4.1 Product detail response

```
{
  "data": {
    "slug": "pzem-004t-ac-energy-meter-module",
    "name": "PZEM-004T AC Energy Meter Voltage Current Power Monitoring Module",
    "skuPrefix": "0510003",
    "brand": "Peacefair",
    "manufacturerPartNumber": "PZEM-004T-100A",
    "shortDescription": "Real-time voltage, current, power and energy monitoring over UART.",
    "descriptionHtml": "<p>...</p>",
    "subcategory": { "slug": "voltage", "name": "Voltage",
      "category": { "slug": "sensors-conditioning-boards",
        "name": "Sensors & Conditioning Boards" } },
    "images": [ { "id": "01J...", "alt": "...",
      "sm": "/media/01J...-sm.webp",
      "md": "/media/01J...-md.webp",
      "lg": "/media/01J...-lg.webp",
      "width": 1200, "height": 1200 } ],
    "attributes": [ { "name": "Current range", "unit": "A", "position": 0 } ],
    "variants": [
      { "sku": "051000301", "attributes": [ { "name": "Current range", "value": "100" } ],
        "price": { "amount": 35000, "currency": "TZS", "formatted": "TZS 35,000" },
        "compareAt": null, "discount": null,
        "stock": { "available": 10, "status": "in_stock", "lowThreshold": 5 },
        "imageId": null, "isActive": true }
    ],
    "priceRange": { "min": { "amount": 35000, ... }, "max": { "amount": 35000, ... } },
    "totalStock": 10,
    "reviewSummary": { "count": 0, "average": 0,
      "histogram": { "5": 0, "4": 0, "3": 0, "2": 0, "1": 0 } },
    "publishedAt": "2026-03-31T06:59:16.951Z"
  }
}
```

10.5 Cart, wishlist and orders

Verb	Path	Access	Behaviour
GET	/cart	token or guest cookie	Returns the cart with every line revalidated: current price, current availability, and a <code>changes[]</code> array describing anything that moved since the item was added.
POST	/cart/items	same	Body <code>{variantSku, quantity}</code> . Adding an existing line increments. 409 <code>INSUFFICIENT_STOCK</code> when the total would exceed availability.
PATCH	/cart/items/:variantSku	same	Body <code>{quantity}</code> . Zero removes.
DELETE	/cart/items/:variantSku	same	204.
DELETE	/cart	same	Empties the cart.
POST	/cart/merge	TOKEN	Called immediately after sign-in. Merges the guest cart into the account cart per \$12.5.4.
GET POST DELETE	/wishlist, /wishlist/items, /wishlist/items/:id	same	Same shape as the cart, without quantity.
POST	/orders	token or guest	Creates an order. Full contract below.

Verb	Path	Access	Behaviour
GET	/orders	TOKEN	The caller's orders, paginated, filterable by status.
GET	/orders/:orderNumber	owner or confirmation token	Full detail with items, address, totals and the event timeline.
POST	/orders/:orderNumber/cancel	owner	Body {reason?} . Permitted only from pending or awaiting_payment . Writes a transition, releases reservations. 409 otherwise.
POST	/orders/:orderNumber/reorder	owner	Adds available lines to the cart; returns {added[], unavailable[]}
GET	/invoices/shared/:token	token	Public printable invoice.
POST	/notify-requests	PUBLIC	Body {variantSku, email} . Back-in-stock registration.

10.5.1 Create order

```

POST /api/v1/orders
Idempotency-Key: 5f8c1e2a-...

{
  "customer": { "name": "Amina Mushi", "phone": "255712345678",
                "email": "amina@example.com" },
  "items": [ { "variantSku": "051000301", "quantity": 2 } ],
  "shippingMethodSlug": "savings",
  "shippingAddress": { "receiverName": "Amina Mushi", "phone": "255712345678",
                      "email": "amina@example.com", "country": "Tanzania",
                      "region": "DAR-ES-SALAAM", "district": "KINONDONI",
                      "streetAddress": "Plot 44, Mikocheni B", "postalCode": "14110" },
  "paymentMethod": "lipa_namba",
  "paymentNumber": "255712345678",
  "delegatedPayerEmail": null,
  "customerNote": null
}

201 Created
Location: /api/v1/orders/TS-2026-000148
{
  "data": {
    "orderNumber": "TS-2026-000148",
    "status": "awaiting_payment",
    "paymentStatus": "unpaid",
    "fulfilmentStatus": "unfulfilled",
    "items": [ ... ],
    "subtotal": { "amount": 70000, "currency": "TZS", "formatted": "TZS 70,000" },
    "shipping": { "amount": 15000, ... },
    "total": { "amount": 85000, ... },
    "paymentInstructions": {
      "method": "lipa_namba",
      "amount": { "amount": 85000, ... },
      "reference": "TS-2026-000148",
      "networks": [ { "name": "M-Pesa", "tillNumber": "...", "steps": [ "..." ] }, ... ]
    },
    "confirmationToken": "...", // lets a guest view their own confirmation
    "redirectUrl": null, // set when an aggregator hosts the payment
    "placedAt": "2026-08-19T14:32:05.123Z"
  }
}

```

Failure modes are `409 PRICE_CHANGED` and `409 INSUFFICIENT_STOCK`, each returning per-line detail so the client can update the summary in place rather than sending the customer back to the cart.

10.6 Payments

Verb	Path	Access	Behaviour
GET	/pay/:token	token	Order summary for the delegated payer. Deliberately omits the buyer's address, phone and email.
POST	/pay/:token	token	Body {paymentMethod, paymentNumber}. Records a payment as pending. Single-use: the token is consumed.
POST	/orders/:orderNumber/payments	owner	Records a payment attempt against the caller's own order — the "Pay now" path from order detail.
POST	/webhooks/payments/:provider	signature	Aggregator callback. Verifies the signature, writes a payment_events row, then processes idempotently by external_event_id. Always returns 200 once persisted, even if processing is deferred — a provider that receives a non-200 will retry forever.
POST	/admin/orders/:orderNumber/payments/:id/verify	ADMIN	Marks a manual payment settled. Body {providerReference, amount, note?}. Transitions the order and fires notifications.
POST	/admin/orders/:orderNumber/refunds	ADMIN	Body {amount, reason}. Records a refund and restores stock when the order was already fulfilled.

10.7 Content endpoints

Verb	Path	Access	Behaviour
GET	/banners	PUBLIC	?placement=home. Only active banners inside their schedule window.
GET	/partners	PUBLIC	
GET	/services, /services/:slug	PUBLIC	
GET	/articles, /articles/:slug	PUBLIC	Published only.
GET POST	/articles/:slug/comments	public / token	Approved only on read. Writes enter moderation.
DELETE	/comments/:id	TOKEN	Own comment, or with comment.moderate.
GET	/faqs	PUBLIC	Grouped and ordered.
POST	/contact-messages	PUBLIC	Body {name, email, subject?, message, sendCopy}. Honeypot and timing checks; rate limited.

10.8 Admin endpoints

Every admin route is under `/api/v1/admin` and requires a staff account plus the named permission. All follow the same CRUD shape, so only the resource names and their special operations are listed.

Resource	Permission prefix	Beyond standard CRUD
<code>/admin/products</code>	<code>product.*</code>	<code>POST /:id/variants</code> , <code>PATCH /:id/variants/:sku</code> , <code>DELETE /:id/variants/:sku</code> , <code>POST /:id/images</code> , <code>PATCH /:id/images/order</code> , <code>POST /bulk-delete</code> , <code>PATCH /bulk-status</code> , <code>POST /:id/duplicate</code> , <code>GET /export</code> , <code>POST /import</code>
<code>/admin/categories</code> , <code>/admin/subcategories</code>	<code>category.*</code>	<code>PATCH /order</code> for drag-reordering, <code>POST /:id/move-products</code>
<code>/admin/orders</code>	<code>order.*</code>	<code>PATCH /:n/status</code> , <code>PATCH /:n/fulfilment</code> , <code>POST /:n/notes</code> , <code>POST /:n/resend-confirmation</code> , <code>GET /:n/events</code> , <code>GET /export</code>
<code>/admin/invoices</code>	<code>invoice.*</code>	<code>POST /:id/share</code> issues a token, <code>DELETE /:id/share</code> revokes it, <code>POST /:id/send</code> emails it
<code>/admin/customers</code>	<code>customer.*</code>	<code>GET /:id/orders</code> , <code>POST /:id/suspend</code> , <code>POST /:id/unsuspend</code> , <code>POST /:id/password-reset</code> , <code>GET /export</code>
<code>/admin/shipping-methods</code>	<code>shipping.*</code>	<code>PATCH /order</code>
<code>/admin/banners</code>	<code>banner.*</code>	<code>PATCH /:id/image</code> , <code>PATCH /order</code>
<code>/admin/services</code> , <code>/admin/partners</code> , <code>/admin/articles</code> , <code>/admin/faqs</code>	<code>service.*</code> <code>partner.*</code> <code>article.*</code> <code>faq.*</code>	<code>PATCH /order</code> ; articles add <code>POST /:id/publish</code> and <code>POST /:id/unpublish</code>
<code>/admin/comments</code> , <code>/admin/reviews</code>	<code>*.moderate</code>	<code>POST /:id/approve</code> , <code>POST /:id/reject</code> , <code>POST /bulk-moderate</code>
<code>/admin/campaigns</code>	<code>campaign.*</code>	<code>POST /:id/send</code> , <code>POST /:id/schedule</code> , <code>POST /:id/cancel</code> , <code>POST /:id/test</code> , <code>GET /:id/recipients</code> , <code>GET /estimate</code> returns recipient count and SMS cost before sending
<code>/admin/contacts</code>	<code>campaign.read</code>	<code>POST /import</code> , <code>GET /export</code> , <code>POST /:id/unsubscribe</code>
<code>/admin/stock</code>	<code>stock.*</code>	<code>POST /adjust</code> with reason and note, <code>GET /movements</code> , <code>GET /low-stock</code>
<code>/admin/discounts</code>	<code>discount.*</code>	Standard CRUD plus <code>POST /bulk</code>
<code>/admin/media</code>	<code>media.*</code>	<code>POST /</code> upload, <code>DELETE /:id</code> , <code>GET /orphans</code>
<code>/admin/settings</code>	<code>settings.*</code>	<code>GET /</code> , <code>PATCH /</code> accepting a partial map of keys
<code>/admin/staff</code> , <code>/admin/roles</code>	<code>staff.manage</code>	<code>POST /:id/roles</code> , <code>DELETE /:id/roles/:roleId</code> , <code>POST /invite</code>
<code>/admin/analytics</code>	<code>analytics.view</code>	<code>GET /summary</code> , <code>GET /sales?period=</code> , <code>GET /orders?period=</code> , <code>GET /best-sellers?region=</code> , <code>GET /customers</code> , <code>GET /searches</code> , <code>GET /zero-result-searches</code>
<code>/admin/audit-logs</code>	<code>audit.view</code>	Read only. No create, update or delete route exists at all.

10.9 Rate limits

Scope	Limit	Keyed by
Global, unauthenticated	300 / 5 min	IP
Global, authenticated	1,200 / 5 min	user
POST /auth/login	5 / 15 min, then exponential lockout	IP + identifier
POST /auth/register	3 / hour	IP
POST /auth/password/forgot	3 / hour	IP + email
POST /auth/email/verify/request	3 / hour	user
POST /orders	10 / hour	user or IP
POST /pay/:token	5 / hour	token
POST /contact-messages	3 / hour	IP
GET /search	60 / min	IP
Upload endpoints	30 / hour	user
Webhooks	not limited	signature-verified; unsigned requests are dropped before the limiter

Every limited response carries `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset`. Counters live in Redis so limits hold across application instances.

Authentication, Authorisation and Security

11.1 Identity model

One `users` table holds both customers and staff, distinguished by `account_type` and by the roles attached to them. Two tables would duplicate authentication, password reset, session handling and profile management for no benefit, and would make "promote this customer to staff" a data migration.

Concept	Definition
Identifier	A user is reachable by email, by phone, or by both. Login accepts either in a single field, as the reference platform does; the server decides which by shape. At least one must exist, enforced by a check constraint.
Credential	A password hash, an OAuth link, or both. A user who registered with Google and later sets a password holds both and may use either.
Guest	No account. Identified within a session by a <code>guest_token</code> cookie that owns a cart and a wishlist. A guest may place an order; the order stores their name and phone directly.
Staff	<code>account_type = 'staff'</code> plus at least one role. Staff sign in through the same modal; the admin console appears in their account menu.
Role	A named bundle of permissions. Users may hold several; effective permissions are the union.
Permission	An atomic capability string, <code>resource.action</code> . This is the unit checked at every guard.

11.2 Token strategy

THE SINGLE MOST IMPORTANT DIVERGENCE FROM THE REFERENCE PLATFORM

The reference platform stores its bearer token in `localStorage` (defect R9). Any script that executes on the page — an injected third-party tag, a compromised dependency, a stored XSS — can read it and exfiltrate a full session. TechStar splits the session in two so that no single compromise is sufficient.

Token	Specification
Access token	JWT, 15-minute lifetime, signed <code>HS256</code> with a 32-byte secret (or <code>RS256</code> if the API is ever split across services). Held in memory only — a JavaScript variable inside the auth store, never in <code>localStorage</code> , <code>sessionStorage</code> or a cookie. Claims: <code>sub</code> (user public id), <code>typ</code> , <code>roles</code> , <code>perms</code> , <code>sid</code> (session id), <code>iat</code> , <code>exp</code> , <code>jti</code> . Sent as <code>Authorization: Bearer</code> .
Refresh token	32 bytes of cryptographic randomness, base64url, stored server-side as a SHA-256 hash. Delivered as a cookie: <code>HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth; Max-Age=2592000</code> (30 days). Unreadable from JavaScript by construction.
Rotation	Every call to <code>/auth/refresh</code> issues a new refresh token and marks the old one replaced. Refresh tokens are strictly single-use.
Reuse detection	Presenting a refresh token that has already been replaced means it was stolen. The entire session family is revoked immediately, the user is signed out everywhere, and a security email is sent. This is the mechanism that limits the damage of a stolen cookie to a single refresh cycle.

Token	Specification
Silent refresh	The client refreshes at 80% of the access token's life, and once on application boot to restore a session after a reload. Concurrent 401s are queued behind a single refresh, then replayed.
Revocation	Sign-out revokes the current session; password change and reset revoke all others. Revoked session ids are held in a Redis set until their natural expiry, so an unexpired access token cannot outlive its revocation.
Socket authentication	The Socket.IO handshake carries the access token and is verified server-side. Fixes defect R9's second half: the reference platform authenticates sockets with a raw user id, which anyone can supply.

11.3 OAuth

Google and Facebook, both authorization-code flow with PKCE. Implicit flow is not used under any circumstances.

```

1 Client → GET /api/v1/auth/oauth/google
2 Server generates code_verifier + state, stores both against a short-lived
  cookie, redirects to the provider
3 Provider → GET /auth/oauth/google/callback?code=...&state=...
4 Server validates state (CSRF), exchanges code + verifier for tokens,
  fetches the profile
5 Account resolution
  a. oauth_accounts row for (provider, providerUserId) exists → sign in
  b. no row, but a verified user has that email → link, then sign in
  c. no row, no match → create the user with email_verified_at set,
    since the provider has already verified it
6 Server issues its own session, redirects to
  /auth/oauth/callback?code=<one-time>
7 Client → POST /auth/oauth/exchange { code } → session established

```

WHY THE EXTRA EXCHANGE STEP

Redirecting with a token in the URL leaks it into browser history, the `Referer` header of any subsequent request, and server access logs. The one-time exchange code is valid for 60 seconds, single-use, and bound to the originating browser. Case (b) links only against a *verified* email — otherwise anyone could claim an account by registering its address at a provider.

11.4 Email verification, password reset, password rules

11.4.1 Email verification

Registration queues a verification email carrying a 32-byte token, hashed at rest, valid 24 hours, single-use. Until verified, the account can browse, buy and pay — verification is not a gate on commerce, which would cost orders — but it is required to write reviews or comments, and the persistent banner (§7.1.5) keeps asking. Requesting a new link invalidates all outstanding ones. Rate limited to 3 per hour.

11.4.2 Password reset

Token of 32 bytes, hashed at rest, valid 60 minutes, single-use. The request endpoint always returns the same response whether or not the address exists, and takes the same measured time either way — a fast negative is as good an oracle as a different message. On success, every session is revoked and a confirmation email is sent to the address on file, so an unauthorised reset is visible to the real owner.

11.4.3 Password rules

Rule	Specification
Length	Minimum 10 characters, maximum 128. Length is the only requirement that reliably increases strength.
Composition	No mandatory character classes. Forced classes push users toward <code>Password1!</code> ; they measurably reduce entropy.
Blocklist	Rejected against the 100,000 most common passwords and against the user's own email, username and phone.
Breach check	Optional k-anonymity lookup against Have I Been Pwned's range API — only the first five characters of the SHA-1 hash leave the server. Fails open if the service is unreachable.
Strength meter	<code>zxcvbn</code> , shown live during registration and change. Advisory, not blocking, beyond the hard rules above.
Hashing	Argon2id, memory 19 MiB, iterations 2, parallelism 1 — the OWASP baseline. Parameters are stored with the hash so they can be raised later and rehashed on next successful login.
Comparison	Constant-time. On an unknown identifier, a dummy hash is still verified so that response time does not reveal whether the account exists.

11.5 Authorisation — the RBAC matrix

Permissions are checked on the server for every request. Route guards in the client are convenience only (\$6.5).

Permission group	Keys
Catalogue	<code>product.read create update delete publish</code> · <code>category.read create update delete</code> · <code>discount.read create update delete</code> · <code>stock.read adjust</code> · <code>media.read upload delete</code>
Commerce	<code>order.read update cancel refund export</code> · <code>invoice.read create update delete send</code> · <code>shipping.read create update delete</code> · <code>payment.verify refund</code>
Customers	<code>customer.read update suspend export</code>
Content	<code>banner.* service.* partner.* article.* faq.*</code> (read create update delete) · <code>comment.moderate</code> · <code>review.moderate</code>
Marketing	<code>campaign.read create send delete</code> · <code>contact.read import export</code>
Platform	<code>settings.read update</code> · <code>staff.manage</code> · <code>audit.view</code> · <code>analytics.view</code> · <code>dashboard.view</code>

Role	Catalogue	Orders	Customers	Content	Marketing	Platform
owner	full	full	full	full	full	full
manager	full	full	full	full	full	analytics, audit
catalogue	full	read	—	media only	—	dashboard
fulfilment	read, stock	full	read	—	—	dashboard
marketing	read	read	read	full	full	analytics
support	read	read, cancel	read	moderate	—	dashboard

STRUCTURAL RULES

At least one active `owner` must exist at all times — the system refuses to remove the last one. Nobody may edit their own roles. `payment.verify` and `order.refund` are never granted to `support`; separating who takes the order from who confirms the money is basic segregation of duties. Every permission change is written to the audit log with before and after.

11.6 Input validation

Layer	Specification
Schema	Every endpoint declares Zod schemas for <code>params</code> , <code>query</code> and <code>body</code> in <code>packages/contracts</code> . The middleware replaces the raw request values with the parsed, typed result, so a controller cannot accidentally read an unvalidated field.
Strictness	Objects are strict — an unexpected property is a 422, not silently dropped. This catches typos in client code and blocks mass-assignment.
Coercion	Explicit only. Query strings coerce to number or boolean where the schema says so; nowhere else.
Normalisation	Email lowercased and trimmed. Phone stripped to digits and canonicalised to <code>255XXXXXXXX</code> , accepting <code>+255...</code> , <code>0...</code> and <code>255...</code> as input. Names trimmed with internal whitespace collapsed. Slugs generated per §6.6.
HTML	Rich text is sanitised with an allowlist on write <i>and</i> on render. Two independent passes, because a single point of sanitisation is a single point of failure.
SQL	Prisma parameterises everything. The few raw queries in Part 18 use <code>\$queryRaw</code> with tagged-template parameters; string concatenation into SQL is forbidden and caught by lint rule.
Mass assignment	Controllers pass explicit field lists to services. No <code>...req.body</code> spread into an update, ever — that is how a customer sets <code>account_type = 'staff'</code> .
Ownership	Every resource read or write verifies the caller owns the row or holds the permission. Never trust an id in a URL to belong to its bearer.
Enumeration	Sequential ids never appear in URLs (§9.1), so resources cannot be walked.

11.7 Upload security

Control	Specification
Accepted types	<code>image/jpeg</code> , <code>image/png</code> , <code>image/webp</code> , <code>image/avif</code> . SVG is rejected — it is a script-execution vector, and vector logos should be converted to PNG on the way in.
Type determination	Magic bytes, not the client's <code>Content-Type</code> and not the file extension. A mismatch between claimed and actual type is a rejection, not a correction.
Size	3 MB per file, 30 MB per request, 100 files per product. Enforced by the multipart parser before anything touches disk.
Dimensions	Maximum 8000 × 8000 px. Larger images are rejected rather than resized, which stops decompression-bomb attacks.
Re-encoding	Every accepted image is decoded and re-encoded by <code>sharp</code> . The original bytes are never served. This strips any embedded payload and, incidentally, all EXIF including GPS coordinates.
Naming	Stored under a generated ULID with a canonical extension. The user's filename is retained as metadata only, and never used as a path component.
Storage location	Outside the web root. Served through a handler that sets <code>Content-Type</code> , <code>Content-Disposition: inline</code> , <code>X-Content-Type-Options: nosniff</code> and long-lived immutable cache headers.

Control	Specification
Execution	The upload directory is configured with execution disabled at the web-server level, belt and braces.
Authorisation	Only staff with <code>media.upload</code> may upload, except the customer avatar endpoint, which is limited to one file, 2 MB, 1024 px.
Rate	30 uploads per hour per user.

11.8 Security checklist

Risk	Control
Broken access control	Server-side permission check on every route; ownership check on every resource; no sequential ids in URLs; strict schemas prevent mass assignment; automated tests assert that each admin route rejects a customer token.
Cryptographic failures	TLS 1.2+ only, HSTS with a one-year max-age and preload. Argon2id for passwords, SHA-256 for token storage. Secrets from the environment, never in the repository. Database at rest on an encrypted volume. Backups encrypted with a separately managed key.
Injection	Parameterised queries throughout; HTML sanitised twice; no shell interpolation anywhere; JSON body size capped.
Insecure design	Order state machine rejects illegal transitions server-side; stock changes are transactional with a ledger row; payment verification is a separate permission from order handling; idempotency keys on order and payment creation.
Security misconfiguration	Config validated at boot and fails fast; production build strips development hosts (fixing R10); <code>X-Powered-By</code> removed; directory listing off; error responses carry no stack traces; distinct credentials per environment.
Vulnerable components	<code>npm audit</code> and Dependabot in CI; the build fails on a high or critical advisory; a monthly patch window; lockfiles committed.
Authentication failures	Rate limits and progressive lockout; identical responses for unknown and wrong credentials; refresh rotation with reuse detection; sessions listed and revocable by the user.
Data integrity failures	Webhook signatures verified before processing; every inbound event persisted before it is acted on; Subresource Integrity on the few external assets; CI publishes build provenance.
Logging failures	Structured logs with a request id on every line; authentication events, permission denials, payment verifications and admin mutations all logged; passwords, tokens, card-like numbers and full phone numbers redacted by a logger serializer; audit log is insert-only at the database-privilege level.
SSRF	No endpoint fetches a user-supplied URL. Outbound calls go only to a fixed allowlist of provider hosts.
XSS	React escapes by default; <code>dangerouslySetInnerHTML</code> permitted in exactly one component (§8.2), enforced by lint; CSP without <code>unsafe-inline</code> , using nonces for the few inline scripts; access token never in a readable store.
CSRF	Refresh cookie is <code>SameSite=Strict</code> and scoped to <code>/api/v1/auth</code> ; all other authentication is a bearer header, which CSRF cannot forge; state-changing requests require <code>Content-Type: application/json</code> , which blocks simple form posts.
Clickjacking	<code>frame-ancestors 'none'</code> plus <code>X-Frame-Options: DENY</code> .

Risk	Control
Enumeration	Identical responses and timing for unknown accounts, expired tokens and invalid tokens throughout.
Denial of service	Rate limits at several scopes; body and upload caps; query complexity bounded by a maximum page size; slow-query logging with alerting; Nginx connection limits.

11.8.1 Response headers

```

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Content-Security-Policy: default-src 'self';
  script-src 'self' 'nonce-{random}' https://www.googletagmanager.com;
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: blob: https://*.tile.openstreetmap.org;
  font-src 'self';
  connect-src 'self' https://www.google-analytics.com wss://{host};
  frame-ancestors 'none'; base-uri 'self'; form-action 'self';
  object-src 'none'; upgrade-insecure-requests
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=(), payment=()
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin

```

`style-src 'unsafe-inline'` is required because banners apply administrator-chosen colours as inline styles. The alternative — emitting a nonce per banner — is achievable and preferable, and is listed as a hardening item for Phase 5.

11.8.2 Personal data

Obligation	Implementation
Minimisation	Only what an order needs: name, phone, and an address when the shipping method requires one. Date of birth, gender and national identifiers are never collected.
Consent	<code>marketing_opt_in</code> is off by default and captured explicitly. Campaign sending excludes anyone without consent or with an unsubscribe timestamp — the reference platform mails every contact, which is a live compliance exposure.
Unsubscribe	One-click, honoured within seconds, present in every campaign email and every marketing SMS.
Access and portability	A customer may request an export of their personal data; the admin console produces a JSON bundle of profile, addresses, orders, reviews and comments.
Erasure	Account closure anonymises the user row — name replaced with "Deleted user", email and phone nulled, avatar removed — while orders are retained with their snapshot fields for statutory record-keeping. This is the only way to satisfy both obligations at once.
Retention	Guest carts 30 days · search logs 24 months · audit logs 24 months · sessions until expiry · orders and invoices retained per Tanzanian record-keeping requirements, minimum 7 years.

Business Rules and Domain Logic

This part is the contract the server enforces. Every rule here is checked server-side regardless of what the client does. Where a rule exists to correct a defect in the reference platform, the defect number is given.

12.1 SKU scheme

Inherited from the reference platform because it is genuinely good: a positional code makes a physical component store tractable, since a technician can read a bin label and know exactly where it sits in the taxonomy.

Category	2 digits	00–99	assigned on category creation
Subcategory	4 digits	CCSS	category code + 2-digit sequence within it
Product	7 digits	CCSSPPP	subcategory code + 3-digit sequence within it
Variant	9 digits	CCSSPPPVV	product code + 2-digit sequence within it
05	Sensors & Conditioning Boards		
0510	Sensors > Voltage		
0510003	Sensors > Voltage > PZEM-004T AC Energy Meter		
051000301	...the 100 A variant		

Rule	Specification
Assignment	Automatic. The sequence is the lowest unused number within the parent's scope, allocated inside the creating transaction with a row lock on the parent so two concurrent creations cannot collide.
Manual override	An administrator may type a specific code when creating a category or subcategory — this is how an existing physical numbering scheme is preserved. Uniqueness is still enforced.
Immutability	A SKU never changes. Moving a product to a different subcategory keeps its original SKU and records the move in the audit log. Renumbering would invalidate every printed bin label, every past invoice and every supplier order.
Reuse	Never. A deleted product's code is retired permanently, so a historical invoice can always be traced.
Capacity	100 categories, 100 subcategories each, 1,000 products each, 100 variants each — 10 ⁹ variants. Ample. If a subcategory ever exceeds 1,000 products, the correct response is to split it, which is also the correct merchandising response.
Search	The full SKU and any left-anchored prefix are searchable. Typing <code>0510</code> returns everything in Voltage.

12.2 Product grouping and variants

THE DEFECT BEING FIXED (R2)

The reference platform creates one document per attribute combination and reconstructs the product at read time by grouping on the *name string*. Consequences observed in live data: names carrying double spaces and trailing whitespace; grouping that silently splits on any typographical difference; a hard prohibition on two distinct products ever sharing a name; and a product URL that is a name, so a rename breaks every link. All of it follows from one modelling choice.

Rule	Specification
Structure	A <code>product</code> row is the group. Each purchasable configuration is a <code>product_variant</code> row. Every product has at least one variant, even when it has no options.

Rule	Specification
Attributes	A product declares its attribute columns in <code>product_attributes</code> — ordered, optionally with a unit. Each variant supplies one value per attribute. A variant missing a value for a declared attribute is invalid and rejected on save.
Uniqueness	Within one product, no two variants may share the same complete set of attribute values. Enforced by an application check plus a generated hash column with a unique index.
Name hygiene	Names are trimmed and internal whitespace collapsed on write. Duplicate names across products are permitted, since identity is the slug, not the name.
Which variant is default	The lowest <code>position</code> among active, in-stock variants; if all are out of stock, the lowest <code>position</code> overall. The product page preselects it.
Aggregates	<code>total_stock</code> , <code>min_price_amount</code> and <code>max_price_amount</code> are recomputed on the product row inside any transaction that changes a variant's price, stock or active flag. They are never computed on read — the catalogue grid must not join to variants.
Deleting a variant	Soft delete. Refused if it is the last active variant — delete the product instead. Refused with a warning if it appears in any open cart, listing how many.
Adding an attribute later	Adding a column to a product that already has variants requires a value for each existing variant. The admin console prompts for them in one screen rather than silently writing blanks.

12.3 Pricing and discounts

Rule	Specification
Storage	Integer minor units on the variant. Never a float, never a string.
Floor	<code>CHECK (price_amount > 0)</code> . A zero-price product cannot exist — fixing defect R16, where a live product sells for TZS 0. Genuinely free items are modelled as a 100%-discounted line on an order, never as a zero price.
Compare-at	Optional <code>compare_at_amount</code> , which must exceed <code>price_amount</code> when set. Renders as a struck-through "was" price. This is a permanent list price, distinct from a time-boxed discount.
Discount scopes	A discount targets a variant, a product, a subcategory or a category. Type is percentage (1–90) or a fixed amount.
Resolution	When several discounts apply, the most specific scope wins : variant, then product, then subcategory, then category. Within one scope, the one that saves the customer more wins. Discounts never stack — stacking is the source of most pricing incidents in retail systems.
Window	<code>starts_at</code> and <code>ends_at</code> are mandatory and compared in UTC. A discount with no end date is not permitted; a permanent price cut is a price change.
Floor after discount	The discounted price is floored at 1 minor unit. A fixed discount larger than the price is clamped, never negative.
Rounding	Percentage discounts round to the nearest whole shilling, half away from zero. Computed once, server-side, and returned formatted — the client never calculates a price.
Price at order time	The price is resolved and frozen into <code>order_items.unit_price_amount</code> when the order is created. A discount expiring one minute later does not change an existing order.

Rule	Specification
Currency	TZS with zero fraction digits, per the runtime currency setting. Formatting uses <code>Intl.NumberFormat</code> with that configuration, on the server.

12.4 Stock and reservation

THE DEFECT BEING FIXED (R7)

The reference platform displays stock on the product page and then never checks it again. Two customers can buy the last unit; nothing prevents it. For a store that holds one or two of many parts, this is a routine failure, not an edge case.

Rule	Specification
Two counters	Each variant carries <code>stock_quantity</code> (physically on hand) and <code>reserved_quantity</code> (committed to unpaid orders). Available = stock – reserved . Availability is what the storefront shows and what every check uses.
Reservation	Created when an order is placed. Reserved stock is not deducted from <code>stock_quantity</code> — the goods are still on the shelf — but it is not sellable.
Expiry	Reservations for unpaid orders expire after 48 hours . A scheduled job releases them, moves the order to <code>expired</code> and notifies the customer with a link to reorder. Cash-on-delivery orders are exempt, since payment happens at handover.
Deduction	<code>stock_quantity</code> is decremented and the reservation released at the moment payment is confirmed — or, for cash on delivery, when the order is marked shipped. Deducting earlier makes the figure lie about what is on the shelf.
Restoration	Cancellation releases the reservation. A refund after fulfilment restores <code>stock_quantity</code> only when the goods physically return, which the administrator confirms explicitly.
Concurrency	Every stock mutation runs inside a transaction that takes <code>SELECT ... FOR UPDATE</code> on the variant row, in ascending variant-id order across the whole order to prevent deadlock. The check and the write are never separated.
Ledger	Every change writes a <code>stock_movements</code> row in the same transaction, with reason, reference and actor. No exceptions — an adjustment with no ledger row is a bug.
Manual adjustment	Requires <code>stock.adjust</code> , a reason from the fixed list, and a note when the delta exceeds 10 units. Fully audited.
Backorder	Not supported at launch. An out-of-stock variant cannot be added to a cart. "Notify me when available" captures the demand instead.
Low stock	Crossing <code>low_stock_threshold</code> downward raises a dashboard alert and, once daily, a digest email to staff holding <code>stock.read</code> .

12.5 Cart rules

12.5.1 Composition

- A cart line is `(variant, quantity)`. Adding an existing variant increments rather than duplicating, enforced by a unique index.

- Quantity is at least 1 and at most the available stock. Requesting more clamps to available and reports it.
- A cart holds at most 100 distinct lines and 999 units per line — bounds that prevent abuse without troubling a real bulk buyer.

12.5.2 Persistence

A guest cart is server-side, keyed by a `guest_token` cookie, and mirrored to `localStorage` for instant first paint. A signed-in cart is server-side against the user. The server is always authoritative; local storage is a cache, not a source of truth. This is what lets a customer add on a phone and check out on a laptop — a journey the reference platform cannot support, since its cart never leaves the browser until checkout.

12.5.3 Revalidation

Every cart read, and the checkout submit, revalidates each line against live data. This is the fix for defects R6 and R7.

Condition	Response
Product or variant soft-deleted	Line removed; notice "This item is no longer available."
Variant deactivated	Same.
Price increased	Line updated to the new price; amber notice "Price changed from X to Y." Checkout requires an explicit re-confirmation.
Price decreased	Line updated silently, with a positive notice. Nobody objects to paying less, and no confirmation is required.
Available < quantity	Quantity clamped to available; notice "Only n left — quantity reduced."
Available = 0	Line marked unavailable and excluded from the total, but kept visible so the customer knows why the total changed.

12.5.4 Merge on sign-in

```

for each line in guestCart:
  if accountCart has the same variant:
    quantity = max(guest.quantity, account.quantity)  # never sum -
    # summing silently doubles an order the customer did not intend
  else:
    add the line
clamp every line to available stock
delete the guest cart, clear the guest cookie
report a summary: "3 items added to your cart"

```

The wishlist merges by union, with duplicates dropped.

12.6 Shipping

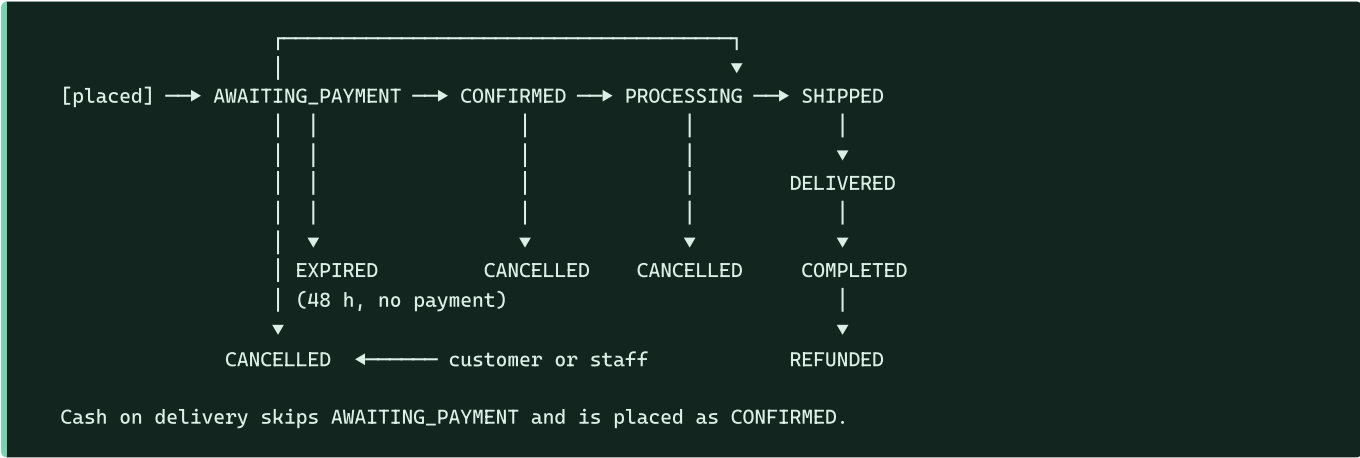
Rule	Specification
Model	Flat cost per method, as the reference platform does. Not weight- or distance-based. The columns for weight exist (\$9.3.13) so a future change is additive.
Availability	A method may declare <code>available_regions</code> . Methods not available for the chosen region are hidden once a region is selected. Before a region is chosen, all are shown.

Rule	Specification
Address requirement	<code>requires_shipping_address</code> drives whether the address block appears. Office pickup does not require one.
Payment coupling	<code>accepts_mobile_payment</code> drives whether the payment block appears. A cash-on-delivery method sets it false and <code>is_cash_on_delivery</code> true.
Free threshold	When <code>free_over_amount</code> is set and the subtotal reaches it, the cost renders as "FREE" with the original struck through, and the order records zero shipping.
Cost freezing	The cost is copied into the order at placement. A later change to the method's price does not alter existing orders.
Default	No method is preselected. Preselecting the cheapest looks helpful and produces orders shipped by a method the customer never chose.

12.7 Order state machine

Three independent axes. Lifecycle answers "where is this order"; payment answers "have we been paid"; fulfilment answers "have we shipped". Collapsing them into one field, as the reference platform does, makes the common case — paid but not yet shipped — inexpressible.

12.7.1 Lifecycle



State	Meaning and permitted transitions
<code>AWAITING_PAYMENT</code>	Placed, stock reserved, money not yet received. → <code>CONFIRMED</code> (payment verified) · <code>CANCELLED</code> (customer or staff) · <code>EXPIRED</code> (48-hour job).
<code>CONFIRMED</code>	Paid, or cash on delivery accepted. Stock deducted for paid orders. → <code>PROCESSING</code> · <code>CANCELLED</code> (staff only, with reason; triggers a refund when money was taken).
<code>PROCESSING</code>	Being picked and packed. → <code>SHIPPED</code> · <code>CANCELLED</code> (staff only).
<code>SHIPPED</code>	Handed to the courier or ready for pickup. For cash on delivery, stock is deducted here. → <code>DELIVERED</code> · No cancellation.
<code>DELIVERED</code>	Received by the customer. → <code>COMPLETED</code> automatically after 7 days · <code>REFUNDED</code> (returns window).
<code>COMPLETED</code>	Terminal. Unlocks review eligibility.
<code>CANCELLED</code>	Terminal. Reservations released; refund raised if money was taken.

State	Meaning and permitted transitions
EXPIRED	Terminal. Reservations released. Distinguished from CANCELLED because it is a system action, and the analytics meaning is different.
REFUNDED	Terminal. Full or partial; stock restored only on confirmed physical return.

ENFORCEMENT

Transitions are validated against an explicit table in the order service. Anything not listed returns 409 INVALID_STATE_TRANSITION. Every accepted transition writes an order_events row with actor, timestamp, from and to. No code path anywhere may write orders.status directly — the state machine is the only door.

12.7.2 Payment and fulfilment axes

Payment: unpaid → pending_verification → paid, with failed, partially_refunded and refunded as branches.

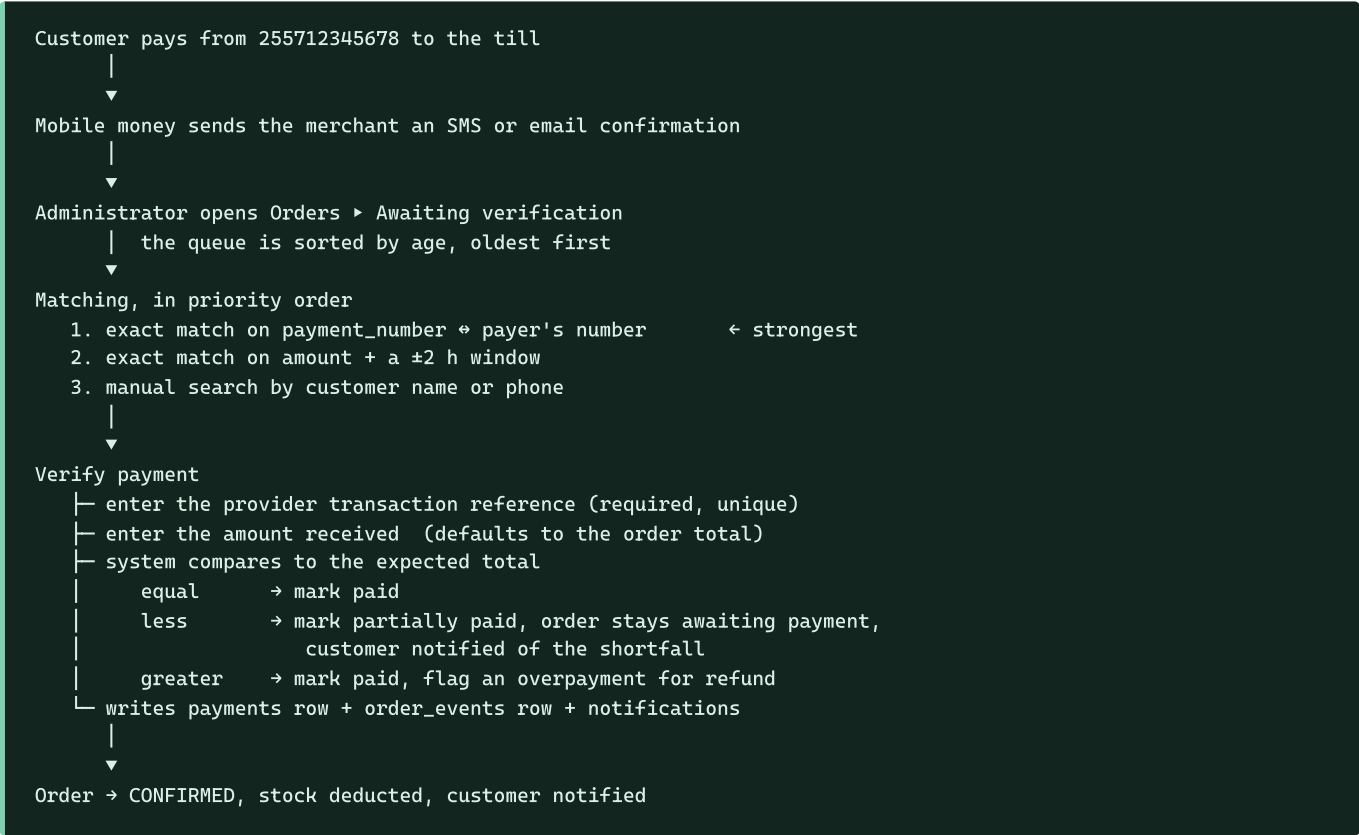
Fulfilment: unfulfilled → processing → shipped → delivered, with returned as a branch. Each axis moves independently; the lifecycle state is derived from the pair and never contradicts it.

12.7.3 Notifications per transition

Transition	Customer	Staff
Order placed	Email + SMS with the order number and payment instructions	Realtime toast and a bell notification in the admin console
Payment recorded	Email "We have received your payment details"	Realtime alert flagging an order awaiting verification
Payment verified	Email + SMS "Payment confirmed"	—
Shipped	Email + SMS with any tracking reference	—
Delivered	Email, plus a review invitation after 3 days	—
Cancelled	Email with the reason	Notification when the customer cancelled
Expiring soon	SMS at 36 hours: "Your order expires in 12 hours"	—
Expired	Email with a one-click reorder link	Daily digest

12.8 Payment reconciliation

With manual Lipa Namba, the money arrives at the till outside the system. Reconciliation is the process of tying an arrival to an order, and it is where a manual flow either works or quietly loses money.



Control	Specification
Reference uniqueness	Unique on (provider, provider_reference) . The same mobile-money transaction can never be credited to two orders — the most likely and most costly manual error.
Separation of duties	payment.verify is a distinct permission, not granted to the support role.
Audit	Every verification records who, when, from where, and the reference. Reversal is possible but never silent, and always leaves both records.
Ageing	Orders awaiting verification for more than 2 hours are highlighted amber on the dashboard; more than 24 hours, red.
Automation path	An aggregator (§13.3) replaces steps 1–3 with a signed webhook carrying the reference and amount. The verification record is written identically, with actor_type = 'webhook' , so the audit trail and the reports do not change shape when automation arrives.

12.9 Delegated payment

The "friend payment" feature, redesigned around a token. Persona Grace — a teacher whose school pays — is the primary case, and it is common enough in this market to be a first-class flow rather than a workaround.

Step	Specification
1. Nominate	At checkout the buyer expands "Someone else is paying" and supplies the payer's email. The order is created as AWAITING_PAYMENT with stock reserved as normal.

Step	Specification
2. Token	A 32-byte token is generated, stored hashed on the order, valid 14 days , single-use. It is the <i>only</i> credential for the payment page — no order id appears anywhere, which is what closes the reference platform's exposure of using a raw order identifier in the URL.
3. Notify	The payer receives an email naming the requester by first name and last initial, stating the amount and what it is for, with the link. The buyer receives a copy confirming it was sent.
4. Pay	The payer opens <code>/pay/:token</code> , sees only the line items and totals, picks a payment method, enters their payment number and submits. No account, no sign-in.
5. Confirm	Verification is identical to §12.8. Both payer and buyer are notified on success.
6. Expiry	An unpaid order expires at 48 hours like any other; the token is invalidated with it. The buyer may re-issue a new link from their order detail page, which resets both clocks.
Privacy	The payer never sees the delivery address, phone number or email of the buyer, nor their order history. The buyer never sees the payer's phone number beyond the last four digits.

12.10 Invoicing

Rule	Specification
Two origins	An invoice may be generated from an order, or created standalone by an administrator. The standalone case is the quotation a school or company needs before it will release funds, and is the reason the module exists.
Numbering	<code>INV-YYYY-000000</code> , gapless within a year, allocated under a row lock. Gaps in an invoice sequence raise questions in an audit.
Immutability	A draft invoice is editable. Once issued it is frozen; corrections are made by voiding and re-issuing, never by editing. Voided invoices keep their number.
Line items	Catalogue lines carry a variant reference and snapshot fields; free-text lines are allowed for services and charges outside the catalogue.
Sharing	"Share" mints a token backing <code>/invoice/:token</code> . Revocable at any time. The page is noindex and never appears in the sitemap.
Printing	A print stylesheet renders A4 with the TechStar letterhead, registration details, client block, line table, totals, payment instructions and standard terms. No browser headers or footers.
Standard terms	Three lines carried on every invoice, as the reference platform does — goods shipped on confirmation of full payment; terms and conditions apply to handling, processing and shipping; payment only through TechStar's designated payment methods. Wording is administrator-editable in settings.
Stock	Issuing an invoice does not reserve or deduct stock. Only an order does. This must be explicit, because it is the assumption most often got wrong.

12.11 Returns and refunds

Rule	Specification
Window	7 days from delivery for a fault; 48 hours for an incorrect item shipped. Both are configurable in settings and must be reflected in the published Terms.
Eligibility	Components sold in opened anti-static packaging are non-returnable unless faulty — a necessary rule in this trade and one that must be stated plainly at the point of sale, not only in the Terms.
Process	The customer contacts support; staff raise a return against the order; goods are received and inspected; a refund is recorded; stock is restored only when the item is confirmed resaleable.

Rule	Specification
Refund method	Mobile money back to the paying number, which is why <code>payments.payer_phone</code> is captured. Cash refunds are recorded with the receiving staff member named.
Partial refunds	Supported per line. The order moves to <code>partially_refunded</code> and both amounts remain visible.
Audit	Every refund records actor, amount, reason and reference, and appears on the order timeline for the customer to see.

Payments in Tanzania

Payment is where an e-commerce specification written for one market fails in another. Card penetration is low, mobile money is near-universal, and the dominant flow is a customer pushing funds to a merchant till number from a USSD menu — an action the website cannot observe. Everything in this part follows from that fact.

13.1 Landscape

Channel	Characteristics and implications for TechStar
Mobile money — customer-initiated push (Lipa Namba / merchant till)	The customer dials a USSD string or uses their wallet app, selects the merchant-payment option, enters the till number and the amount, and confirms with a PIN. The merchant receives an SMS or email notification. <i>No API is involved.</i> This is what the reference platform implements, it works without any commercial integration, and it is what TechStar launches with. The cost is manual reconciliation (\$12.8).
Mobile money — merchant-initiated pull (USSD push / STK)	The merchant's system asks the network to prompt the customer's handset; the customer approves with a PIN and the merchant is notified over a webhook within seconds. This requires an aggregator or a direct operator integration, and it removes reconciliation entirely. It is the target state, specified as Phase 4.
Cash on delivery	Widely expected, especially for first-time buyers, and materially raises conversion. The cost is delivery risk and a cash-handling process. The reference platform charges a lower shipping fee for it, which is a sensible way to price the risk down.
Bank transfer	Relevant for institutional buyers — schools, universities, companies — who pay against an invoice. Handled by the invoice module plus manual reconciliation; no integration needed.
Cards	Low volume domestically, useful for diaspora and corporate purchases. Available through the same aggregators. Out of scope for v1 (\$3.5).

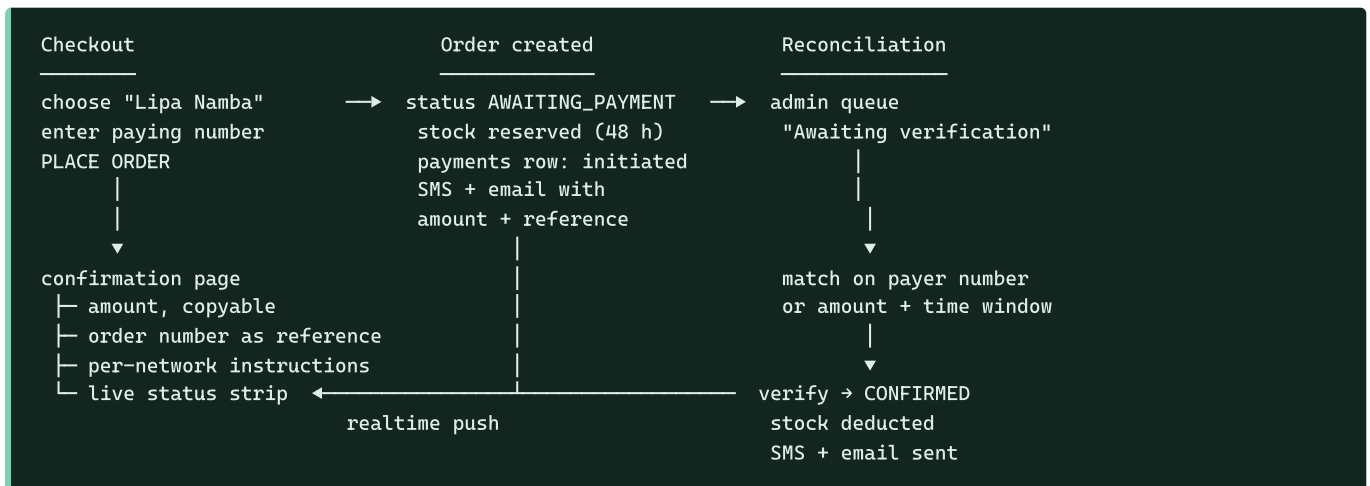
13.1.1 Networks to support

Network	Operator	Notes
M-Pesa	Vodacom	Largest by subscriber base. Wallet app supports scanning a merchant QR code, which is why the payment panel carries one.
Mixx by Yas	Yas (formerly Tigo)	The reference platform still labels this "Tigo Pesa". Show both names during the transition — a customer whose phone menu says one thing and whose screen says another will hesitate.
Airtel Money	Airtel	
HaloPesa	Halotel	Smaller, but the "other networks" instruction block covers it, since all Tanzanian wallets interoperate.
T-Pesa	TTCL	Covered by the same interoperability path.

VERIFY BEFORE YOU PUBLISH

USSD strings, menu positions and till-number formats change, and they differ between consumer and merchant products. Every instruction block must be verified against the live network — by an actual test payment — before launch, and re-verified whenever an operator changes its menus. That is precisely why the instruction content is stored as editable settings (§14.16) rather than compiled into the front end: correcting a wrong menu step must take two minutes, not a release.

13.2 Manual Lipa Namba flow



Design point	Rationale
Capture the paying number	The single strongest reconciliation key. Required, validated as 12 digits beginning 255 . It is what turns matching from a search into a lookup.
Order number as reference	Shown prominently and copyable. Some wallets allow a reference; where they do, it makes matching exact.
Exact amount	"Pay exactly TZS 85,000." Round-number habits produce under- and over-payments, and each one costs a phone call.
Reservation window	48 hours, communicated at placement and again by SMS at 36 hours. Long enough for someone who pays on payday; short enough that stock is not held indefinitely.
Live status	The confirmation page subscribes to the order's realtime channel, so verification updates the page the customer is already looking at. It is a small thing that removes a large share of "have you received my payment?" contacts.
No payment proof upload	Deliberately omitted. Screenshots are trivially forged and create a queue of items requiring judgement. The merchant's own notification is the source of truth.

13.3 Aggregator integration

Automation is introduced behind one interface, so that the choice of provider is a configuration decision and never a rewrite. Nothing outside the adapter knows which provider is in use.

```

interface PaymentProvider {
  readonly key: string // 'selcom' | 'clickpesa' | ...
  readonly capabilities: {
    mobileMoneyPush: boolean // prompt the customer's handset
    hostedCheckout: boolean // redirect to a provider page
    cards: boolean
    refunds: boolean
    statusQuery: boolean
  }

  createPayment(input: {
    orderNumber: string
    amount: Money
    customer: { name: string; phone: string; email?: string }
    method: 'mobile_money' | 'card'
    network?: 'mpesa' | 'mixx' | 'airtel' | 'halopesa'
    returnUrl: string
    webhookUrl: string
    idempotencyKey: string
  }): Promise<{
    providerReference: string
    status: 'pending' | 'succeeded' | 'failed'
    redirectUrl?: string // hosted checkout
    instructions?: string // e.g. "approve the prompt on your phone"
  }>

  verifyWebhook(headers, rawBody): { valid: boolean; event: ProviderEvent }
  queryStatus(providerReference): Promise<PaymentStatus>
  refund(providerReference, amount: Money, reason: string): Promise<RefundResult>
}

```

Candidate	What to evaluate at selection time
Selcom	Established Tanzanian aggregator with broad wallet coverage and a merchant network. Evaluate settlement period, per-transaction pricing and webhook reliability.
ClickPesa	Developer-oriented, positioned at exactly this use case. Evaluate onboarding time and sandbox quality.
AzamPay	Local aggregator with multi-wallet collection. Evaluate documentation depth and support responsiveness.
DPO Group / Pesapal	Pan-African, card plus mobile money, hosted checkout available. Evaluate pricing on low-value transactions — a TZS 200 resistor order must not cost more in fees than it earns.
Direct operator	Vodacom and Airtel offer merchant APIs directly. Better unit economics at volume; heavier onboarding and one integration per network.

SELECTION CRITERIA, IN PRIORITY ORDER

- (1) Coverage of all four major wallets through one integration.
- (2) Reliable, signed webhooks with documented retry semantics.
- (3) A sandbox that behaves like production.
- (4) Settlement within two business days.
- (5) Per-transaction cost that is viable on a TZS 5,000 basket.
- (6) Refund support through the API. Providers that fail (2) create more manual work than they remove.

13.4 Cash on delivery

Rule	Specification
Order state	Placed directly as CONFIRMED — payment is not pending in any meaningful sense. Payment status stays unpaid until the courier settles.
Stock	Reserved at placement, deducted at SHIPPED , not at confirmation.

Rule	Specification
Reservation expiry	Does not apply. The order is confirmed, so there is nothing to expire.
Basket cap	Configurable maximum order value for cash on delivery, default TZS 300,000. Above it the method is hidden with an explanation. Unpaid delivery risk scales with basket size and should be bounded.
Verification call	Staff telephone the customer to confirm before dispatch. Recorded as an order event, so a repeat no-answer is visible.
Settlement	The courier records collection; staff mark the payment received with the collecting person named. This creates the <code>payments</code> row with method <code>cash</code> .
Refusal on delivery	Order moves to <code>CANCELLED</code> with reason "refused on delivery"; stock is restored on return to the store. Repeat refusals from one phone number are surfaced to staff on that customer's record.

13.5 Webhooks and idempotency

An aggregator's webhook is the only moment where money and system state meet automatically. It must be impossible to double-credit, and impossible to lose an event.

```
POST /api/v1/webhooks/payments/:provider
```

- 1 Read the RAW body – never the parsed one. Signature verification over re-serialised JSON fails intermittently and inexplicably.
- 2 Verify the signature with the provider's secret, constant-time compare. Fail → log, return 401, do not process.
- 3 Check the source IP against the provider's published range when one exists.
- 4 INSERT the event into `payment_events`, keyed by `external_event_id`. Duplicate key → this is a retry. Return 200 immediately. Do nothing else.
- 5 Return 200 NOW. Everything below runs on a queue.
A provider that does not get a prompt 200 will retry, sometimes forever.
- 6 Worker:
 - └ resolve the order by `merchant_reference`
 - └ compare the amount to the expected total
 - └ update the `payments` row inside a transaction
 - └ run the order state transition through the state machine
 - └ deduct stock with a ledger row
 - └ write `order_events` with `actor_type = 'webhook'`
 - └ queue customer notifications
 - └ mark `payment_events.processed_at`
- 7 Processing failure → record `processing_error`, alert staff, leave the event replayable from the admin console. The event is never lost.

Guarantee	Mechanism
At-most-once crediting	Unique index on <code>payment_events.external_event_id</code> , and on <code>payments(provider, provider_reference)</code> . Two independent constraints, because this is the one thing that must not fail.
No lost events	Persist before processing. Every webhook body is stored whether or not it can be handled.
Out-of-order delivery	Events carry a provider timestamp; an event older than the payment's current state is recorded and ignored.
Unknown order	Recorded with <code>processing_error = 'order_not_found'</code> and raised to staff. Never silently dropped — an unmatched payment is money that arrived.
Amount mismatch	Never auto-confirms. Flagged for manual review with both figures shown.
Replay	Staff with <code>payment.verify</code> can replay a stored event from the admin console after a bug fix, without asking the provider to resend.

Guarantee	Mechanism
Reconciliation sweep	A scheduled job queries provider status for every payment stuck in <code>pending</code> beyond 30 minutes, catching webhooks that never arrived.

13.6 The reconciliation ledger

Three questions must be answerable at any moment, from the database alone, without a spreadsheet:

1. **What have we been paid?** Sum of `payments` in status `succeeded` over a period, grouped by method and provider.
2. **What are we owed?** Sum of `orders.total_amount - paid_amount` where the order is not cancelled, expired or refunded.
3. **What arrived that we cannot place?** `payment_events` with a processing error, and payments whose amount does not match their order.

Report	Definition
Daily settlement	Every payment verified in the day, with order number, method, provider reference, amount and verifier. Exportable to CSV; reconciled against the mobile-money statement.
Outstanding	Orders awaiting payment, aged into 0–2 h, 2–24 h and over 24 h buckets. Drives the dashboard alert.
Exceptions	Amount mismatches, unmatched events, duplicate reference attempts, overpayments awaiting refund. Should normally be empty; a persistently non-empty exceptions list is the signal to move to an aggregator.
Refund register	All refunds with reason, amount, destination number and authorising staff member.
Method mix	Share of orders and of revenue by payment method over time — the evidence base for deciding whether cash on delivery is worth its risk and whether automation is worth its fee.

FINANCIAL INVARIANT

For every order: `paid_amount = SUM(payments.amount WHERE status = 'succeeded') - SUM(refunds)`. A nightly job asserts this across all orders and alerts on any row that fails. An invariant that is checked is a control; one that is merely intended is a hope.

Admin Console Specification

The brief asks for "full CRUD ability to manipulate anything in the platform". This part delivers that literally: every entity in the data model has a create, read, update and delete path in the console, plus the bulk operations, filters and safeguards that make those paths usable at scale. §14.19 states the universal contract that every module below conforms to, so the individual sections describe only what differs.

14.1 Shell and navigation

Region	Specification
Sidebar	Fixed, 240 px, on <code>--ts-ink</code> , collapsible to a 64 px icon rail with the state persisted per user. TechStar wordmark at the top. Items are grouped with small uppercase muted labels. The active item carries a 3 px primary-green left rule and a lighter background. Each item renders only if the signed-in staff member holds the governing permission — an empty group header is never shown.
Groups	Overview — Dashboard. Catalogue — Products, Categories, Subcategories, Stock, Discounts, Media. Sales — Orders, Invoices, Payments, Customers, Shipping. Content — Banners, Articles, Comments, Reviews, Services, Partners, FAQs. Marketing — Campaigns, Contacts, Searches. System — Currency, Settings, Staff & Roles, Audit Log. Then Sign out.
Badges	Live counts beside items needing attention: Orders (awaiting verification), Comments (pending), Reviews (pending), Stock (below threshold). Updated over the realtime channel.
Top bar	64 px, white, bottom border. Left: breadcrumb, and the sidebar collapse toggle. Centre: global search — an omnibox over orders (by number or phone), products (by name or SKU), customers (by name, email or phone) and invoices, opened with <code>Ctrl+K</code> . Right: a "View store" link opening the storefront in a new tab; a notification bell with a dropdown of recent events; the account menu.
Content	<code>--ts-surface-alt</code> background, 24 px padding, max width 1600 px. Every page opens with a <code>PageHeader</code> : title, optional description, and a right-aligned action cluster.
Responsive	Below <code>lg</code> the sidebar becomes an off-canvas behind a hamburger. Tables scroll horizontally inside their own container. The console is expected to be used on a tablet in the store; it is not designed for a phone, and says so if opened on one.
Notifications	Bell dropdown lists the last 20 events: new order, payment recorded, low stock, new comment, new review, campaign completed, webhook failure. Each links to the relevant record. Unread count badge; marking all read is one action.

14.2 Dashboard

Purpose. Answer "what needs my attention right now" and "how are we doing" in one screen, without scrolling past the fold for the first of those.

14.2.1 Attention strip

The top row, and the reason the page is opened. Each tile is a filtered link; a tile with a zero count renders muted rather than disappearing, so its absence is never mistaken for a missing feature.

Tile	Contents and threshold colouring
Awaiting payment verification	Count of orders with payment status <code>pending_verification</code> . Amber past 2 hours, red past 24. Links to the filtered order queue.
Orders to fulfil	<code>CONFIRMED</code> and <code>PROCESSING</code> counts.
Low stock	Variants at or below their threshold. Links to the stock report sorted ascending.
Pending moderation	Comments plus reviews awaiting a decision.
Failed webhooks	Unprocessed <code>payment_events</code> . Red whenever above zero — this tile should always read zero.

14.2.2 Metric tiles

Four `StatCard`s, each with a period selector shared across the row (Today · Last 7 Days · Last 30 Days · Last 12 Months) and each showing the value, the change against the immediately preceding period of equal length, and a sparkline.

- **Sales** — sum of `total_amount` for orders paid in the period.
- **Orders** — count of orders placed in the period.
- **Average order value** — sales ÷ orders.
- **Customers** — new registrations in the period, clickable to a breakdown modal by region.

14.2.3 Charts and lists

Panel	Specification
Sales analytics	Bar chart of revenue by bucket — day for 7 and 30 days, month for 12 months. Primary green bars. Tooltip shows the amount and the order count. A "View as table" toggle exposes the same figures accessibly.
Orders analytics	Line chart of order count over the same buckets, on the same period selector.
Best selling	Top 10 products by units sold in the period, with thumbnail, name, units and revenue. A region filter answers "what sells upcountry versus in Dar", which is the reference platform's own <code>best_selling?region=</code> feature and a genuinely good one.
Recent orders	Last 10, with number, customer, total, status pill and time. Rows link to detail.
Top searches	Top 10 search terms in the period with their result counts. Terms returning zero are highlighted red — this list is the catalogue's own to-do list.
Empty state	Before any orders exist, the charts are replaced by a short onboarding checklist: add a category, add a product, configure shipping, set the payment till number, place a test order.

14.3 Products

14.3.1 Product list

Aspect	Specification
Columns	Selection checkbox · 48 px thumbnail · Name (with SKU prefix in monospace beneath) · Subcategory · Price (range when multi-variant) · Stock (total, coloured by threshold) · Variants (count) · Status pill (Draft / Active / Archived) · Sold (lifetime units) · Updated · Actions.
Search	Name, SKU, MPN and brand. Debounced, reflected in the URL.
Filters	Category, subcategory, status, stock state (in stock / low / out), discounted, created-date range.
Sort	Name, price, stock, units sold, created, updated.

Aspect	Specification
Row actions	Edit · Duplicate · Set discount · View on storefront · Archive · Delete.
Bulk actions	Delete · Archive · Activate · Set discount · Move to subcategory · Export CSV. All bulk actions follow §14.19.4.
Header actions	"New product" primary · "Import" · "Export".
Empty	"No products yet" with "Add your first product" and a link to the import guide.

14.3.2 Product create and edit — the variant matrix

This is the most important screen in the console: catalogue growth is bounded by how fast a product can be entered, and the target is under three minutes (§3.6). The reference platform's attribute-matrix design is good and is kept, with its rough edges removed.

Section	Fields and behaviour
1. Classification	Subcategory — a searchable select grouped by category, required. Choosing it derives and displays the SKU prefix immediately, so the administrator sees the code the product will receive before saving.
2. Details	Product name (required, 3–255, trimmed and whitespace-collapsed on blur, with a live duplicate-name warning that informs but does not block) · Short description (plain text, 500, used for meta description with a live SEO preview) · Description (rich text, required) · Brand · Manufacturer part number · Tags (create-as-you-type).
3. Photos	Drag-and-drop uploader, up to 100 files, 3 MB each, JPEG/PNG/WebP/AVIF. Reorderable by drag; the first is the display image unless another is starred. Each accepts alt text. Upload begins on drop with per-file progress. Derivatives are generated asynchronously and the UI shows a "processing" state rather than a broken thumbnail.
4. Pricing & stock — simple	When no attributes are defined: Price (required, > 0) · Compare-at price (optional, must exceed price) · Cost (optional, staff-only) · Available stock (required, ≥ 0) · Low-stock threshold (default 5) · Barcode . One variant is created behind the scenes.
5. Pricing & stock — variants	Introduced by "Add attributes", with the explanatory line the reference platform uses: <i>"Add specifications to your product here. Skip this step if the product does not have any special specifications."</i> The matrix then appears as a table whose columns are the attributes plus fixed Price and Stock . "+" adds an attribute column, prompting for its name and optional unit. "Add new values" adds a row. Each cell validates on blur. A row may be removed; "Delete table" clears the matrix back to the simple form after confirmation. Every row becomes one variant with an automatically assigned SKU shown read-only in a leading column.
6. Visibility	Status — Draft / Active / Archived, default Draft, so a half-entered product is never live. Publish date (optional, schedules activation). Featured flag.
7. SEO	URL slug — auto-generated, editable at creation. On edit it is read-only with a "Change URL" control that warns about link breakage and offers the 301 redirect described in §6.6. Meta title (70) and meta description (160) with counters and a search-result preview.
Footer	Sticky. "Cancel" · "Save as draft" · "Save and add another" · "Save" primary. Disabled until dirty and valid. Leaving with unsaved changes prompts.

Validation	Message
Subcategory missing	"Subcategory is required"
Name missing	"Product name is required"
Description missing	"Description is required"

Validation	Message
No images	"At least one image is required"
Price $\leq$ 0	"Price must be greater than zero"
Compare-at $\leq$ price	"Compare-at price must be higher than the price"
Stock negative	"Stock cannot be negative"
Attribute value blank	"Every variant needs a value for each attribute"
Duplicate attribute set	"Two variants have the same combination of values"
Attribute name duplicated	"Attribute names must be unique within a product"

IMPROVEMENTS OVER THE REFERENCE IMPLEMENTATION

A draft status, so nothing goes live half-finished. A visible SKU before saving. A duplicate-name warning. Editable alt text. Variant SKUs shown in the matrix. A "Save and add another" path for bulk entry sessions. And the matrix keeps its state when validation fails — the reference platform's implementation loses unsaved rows on a failed submit, which is the single most expensive small defect in the whole console.

14.3.3 Stock management

A dedicated screen at `/admin/stock`, listing variants rather than products: thumbnail, product name, attribute values, SKU, on hand, reserved, available, threshold, last movement. Filterable to low or out of stock; sortable by available ascending, which is its default. Inline "Adjust" opens a dialog taking a signed delta or an absolute count, a reason from the fixed list (purchase, adjustment, damage, return, stocktake), and a note required for deltas above 10. Each variant has a "Movements" drawer showing its full ledger. Bulk adjustment by CSV upload — SKU, delta, reason — is provided for stocktakes, with a dry-run preview before commit.

14.3.4 Discounts

Reachable from a product row action and from a dedicated list. The modal takes **Discount (%)** or a fixed amount, **Discount start date** and **Discount end date** — the reference platform's three fields — plus scope (this variant / this product / this subcategory / this category) and an active toggle. A live preview shows the resulting price for the affected items and warns if the discount would take any price below one shilling. The list view shows all discounts with scope, value, window, affected item count and status, filterable to active, scheduled or expired.

14.4 Categories and subcategories

Aspect	Specification
Category list	Columns: selection · Name · SKU code · Subcategories (count) · Products (count) · Position (drag handle) · Status · Created · Actions. Rows are drag-reorderable, which writes <code>position</code> and immediately changes the storefront mega-menu order.
Create / edit	A modal, as the reference platform does — it is the right weight for four fields. Category name (required, unique) · SKU (2 digits) (required, digits only, unique; auto-suggested as the next free code) · Description (optional) · Image (optional) · Active .
Validation	"Name is required" · "SKU is required" · "SKU should only contain digits" · "SKU must be exactly 2 digits" · "That SKU is already in use" · "A category with this name already exists".

Aspect	Specification
Subcategory list	Columns: selection · Name · Category · SKU code · Products (count) · Position · Status · Actions. Filterable by category.
Subcategory create / edit	Category (searchable select, required) · Sub-category name · SKU (4 digits) , auto-suggested as the parent's code plus the next free sequence, with the parent's two digits shown fixed and only the last two editable — which makes an invalid code impossible to type.
Delete	Refused while children exist, per §9.5. The dialog states the blocking count and offers the two supported paths: "Move products to another subcategory" or "Delete the products first". This is the fix for defect R12 — the reference platform instead offers to cascade-delete every product in the tree.
Move products	A dedicated action: choose a destination subcategory, review the count, confirm. SKUs are preserved (§12.1) and the move is audited.

14.5 Orders

14.5.1 Order queue

Aspect	Specification
Status tabs	All · Awaiting payment · Awaiting verification · Confirmed · Processing · Shipped · Delivered · Completed · Cancelled · Expired — each with a live count. The reference platform offers only All / Completed / Pending / Cancelled, which cannot express the working day.
Columns	Order number (monospace) · Date · Customer name · Phone · Items (count) · Total · Payment status pill · Fulfilment status pill · Shipping method · Region · Actions.
Search	Order number, customer name, phone, email, payment number, or a product SKU within the order.
Filters	Date range, payment method, shipping method, region, amount range, has-delegated-payer.
Default sort	Placed date descending — except on the "Awaiting verification" tab, where it is ascending, oldest first, because that is a work queue and the oldest item is the most urgent.
Bulk actions	Update fulfilment status · Print packing slips · Export CSV. Deliberately no bulk cancel and no bulk delete — an order is a financial record.
Realtime	A new order inserts at the top with a brief highlight and an audible chime, subject to a per-user mute. This is what the reference platform's Socket.IO connection exists for.

14.5.2 Order detail

Two columns. Left (8 of 12): status bar, items, payment panel, fulfilment panel, timeline. Right (4 of 12): customer card, shipping card, totals card, internal notes.

Panel	Specification
Status bar	Order number, placed date, and the three status pills. A "Change status" control offering <i>only</i> the transitions legal from the current state (§12.7) — an illegal transition is not merely rejected, it is not offered. Each change requires a confirmation and accepts an optional note.
Items	Thumbnail, name (linking to the product), attributes, SKU, unit price, quantity, line total. Items whose product has since been deleted render from their snapshot with a muted "no longer in catalogue" marker.

Panel	Specification
Payment	Method, the number the customer said they would pay from, expected total, amount received. Recorded payments listed with status, provider reference, amount, verifier and timestamp. A "Verify payment" action opens the dialog from \$12.8; a "Record manual payment" action covers cash. "Issue refund" is present when the order is paid, behind the <code>payment.refund</code> permission.
Fulfilment	Shipping method and cost, delivery address with a copy-all control, a tracking reference field, "Print packing slip", "Print invoice", and "Mark shipped" / "Mark delivered".
Timeline	The full <code>order_events</code> stream, newest first: actor, action, before and after, timestamp, and any note. Notifications sent appear here too, so "did we tell the customer?" is answerable.
Customer	Name, email, phone, and whether the account is registered or guest. For registered customers: total lifetime orders and value, and a link to their record. A "Contact" control offers WhatsApp, call and email with the order number prefilled.
Internal notes	Threaded, timestamped, attributed. Staff only, never exposed on any customer-facing endpoint.
Actions	Resend confirmation · Regenerate payment link · Cancel order (with reason) · Duplicate as a new order · Create invoice from this order.

14.6 Invoices

Screen	Specification
List	Columns: Invoice number · Order (linked, when derived from one) · Client name · Total · Status pill (Draft / Issued / Paid / Void) · Issued date · Due date · Actions. Search by number, client name or email. Filter by status and date range.
Builder	Client details — name (required), email (required, valid), phone, address. Line items — a product search field with the reference platform's own placeholder, "Please search here ...", adding catalogue lines with SKU, price and an editable quantity; plus "Add custom line" for free-text charges. Shipping method (optional). Notes . A live totals panel on the right.
Validation	"Client name is required" · "Email is required" · "Enter a valid email" · "Add at least one line item" · "Quantity must be at least 1".
Detail	Rendered as the printable document, with an action bar above: Print · Download PDF · Email to client · Share link (mint, copy, revoke) · Mark paid · Void · Delete (drafts only).
Print	A4, TechStar letterhead, registration details, client block, line table, totals, payment instructions and the three standard terms (\$12.10).

14.7 Customers

Entirely new — the reference platform has a "Customers" sidebar entry and no screen behind it (defect R13). Without this, staff cannot answer a customer's question about their own account.

Screen	Specification
List	Columns: Avatar · Name · Email (with a verified tick) · Phone · Orders (count) · Lifetime value · Last order · Registered · Status pill. Search by name, email or phone. Filters: verified, has ordered, region, registration date, marketing consent, status.
Detail — profile	All fields, with edit for name, email, phone and marketing consent. Shows verification state with a "Resend verification" action, and account status with suspend and reactivate.
Detail — orders	Full order history with totals, linking through to each order.
Detail — addresses	Saved addresses, read-only.

Screen	Specification
Detail — activity	Reviews and comments written, wishlist contents, last sign-in, active sessions with a "Revoke all sessions" action.
Detail — actions	Send password reset · Suspend (with reason; suspended users can sign in but cannot order, and are told why) · Export personal data (§11.8.2) · Anonymise account.
Guardrails	Staff cannot read or set a password. Every view of a customer record is written to the audit log — access to personal data is itself an auditable event.

14.8 Shipping methods

Aspect	Specification
List	Columns: drag handle · Name · Cost · Description (truncated) · Requires address · Accepts mobile payment · Cash on delivery · Free over · Regions · Active · Actions. Drag-reordering sets the order shown at checkout.
Create / edit modal	Shipping method (name, required) · Shipping cost (required, ≥ 0 , may be zero for pickup) · Description (required — it is shown to the customer at checkout and is how they choose) · Estimated delivery days min and max · Free over (optional) · Available regions (multi-select; empty means everywhere) · three checkboxes with the reference platform's own explanatory copy: <i>"Check the box if this shipping method requires the client to provide shipping details."</i> , <i>"Check the box if this shipping method can receive mobile payments."</i> , and <i>"This method is cash on delivery."</i> · Active .
Validation	"Shipping method is required" · "Shipping cost is required" · "Please enter a valid shipping cost" · "Description is required" · "A method with this name already exists".
Delete	Soft delete. A method referenced by existing orders is never hard-deleted; deactivating hides it from checkout while past orders keep their snapshot name and cost. Deleting the last active method is refused — checkout would have nothing to offer.

14.9 Banners

Aspect	Specification
List	Columns: drag handle · Thumbnail · Title · Placement · Schedule (start–end, or "Always") · Status · Created · Actions. Reordering sets carousel order.
Builder	A two-column screen: form on the left, a live preview on the right rendering the banner exactly as the storefront will, updating on every keystroke. Fields: Title and Title colour · Subtitle and Subtitle colour · Background colour · Image (required) · Link — "Where does the banner redirect to" · Button text , Button background , Button text colour · Placement · Start and End date-times (both optional) · Active .
Colour fields	Each is a <code>ColorField</code> — swatch, picker and hex input in sync. A contrast checker beside the title and subtitle fields warns when the chosen text colour fails 4.5:1 against the background, showing the computed ratio. Advisory, not blocking; the reference platform has no such check and its banners rely on the author's eye.
Image guidance	Recommended 1200 × 800, transparent PNG or WebP for product cut-outs. The uploader warns when a file's aspect ratio would crop badly at the banner's dimensions.
Link validation	Must be a valid absolute or site-relative URL. Internal links are offered as a picker over products, categories and pages, so a typo cannot produce a dead banner.
Preview modal	"Preview" opens the banner full width at desktop and mobile widths side by side, matching the reference platform's preview modal.
Image replacement	Editing text and replacing the image are separate operations, as on the reference platform — replacing an image is a re-upload, and coupling it to a text save makes both slower.

14.10 Services · 14.11 Partners

Module	Specification
Services	Card grid list — image, heading, excerpt, edit and delete. Composer: Heading (required, maximum 15 words, with the reference platform's own rule "Heading should not exceed 15 words") · Excerpt · Content (rich text, required) · Image (required, "Drag & drop your image") · Position · Active . Validation: "Service heading is required" · "Heading should not exceed 15 words" · "Service content is required" · "Image is required".
Partners	Card grid — logo, name, website. Composer: Partner name (required) · Partner's website link (required, valid URL) · Logo (required; square recommended, transparent background preferred) · Position · Active . Validation: "Partner name is required" · "Link to partner's website is required" · "Image is required".

14.12 Blog, comments and reviews

Module	Specification
Articles	List as cards or a table, filterable by status and author. Composer: Heading (required, ≤ 15 words) · Description — the excerpt, required · Content (rich text, required, placeholder "Compose an epic article...") · Cover image (required) · Tags · Status · Publish date · SEO title and description with counters. Actions: Save draft · Preview (opens the storefront article route in a draft-token mode) · Publish · Unpublish.
Comments	Moderation queue with tabs Pending · Approved · Rejected · Spam. Columns: Author · Article · Excerpt · Submitted · Status. A detail drawer shows the full body with the article for context and Approve / Reject / Mark spam / Delete. Bulk moderation supported. Approving fires no notification to the commenter; rejecting is silent by design.
Reviews	Same queue shape. Columns: Product · Rating · Author · Excerpt · Verified-purchase badge · Submitted · Status. Approving recalculates the product's <code>review_count</code> and <code>average_rating</code> in the same transaction. Reviews are never edited by staff — only approved, rejected or deleted, with the reason recorded.
FAQs	Simple list with drag ordering and a grouping field. Composer: Question · Answer (rich text) · Group · Position · Active .

14.13 Campaigns and contacts

14.13.1 Campaign list

Four counters across the top — Total campaigns · Total contacts · SMS sent · Emails sent — then a table: Name · Channel · Audience · Scheduled · Status · Recipients · Sent · Delivered · Failed · Cost · Created by · Actions. Header actions "New SMS campaign" and "New email campaign". A detail drawer shows the full message, audience definition, schedule and the per-recipient delivery report.

14.13.2 SMS composer

Field	Behaviour
Campaign name	Required. Internal label only.
Send to	"All contacts" or "Specific contacts". Choosing specific reveals a multi-select over the contact list with search.
Additional contacts	Free entry of numbers, space or comma separated, with the reference platform's own hint: "Press the space key to add additional numbers, e.g. 255729XXXXX, 255729YYYYY". Each is validated as 12 digits beginning <code>255</code> ; invalid entries are rejected inline with "Invalid phone number. Please enter a 12-digit number."
Message	Required. A live counter in the reference platform's format — "Chars. Remaining (0) 0/160" — that also reports the number of SMS segments and switches to the 70-character limit when a non-GSM character is typed, since one accented character silently triples the cost of a campaign.

Field	Behaviour
When to send	"Immediately" or "Later" with a date-time picker. Validation: "Date and time are required for sending the campaign later"; the time must be in the future.
Estimate	Before sending, a panel states recipient count, segments per message, total segments and estimated cost. Sending requires confirming that figure. The reference platform sends without any cost estimate, which on a metered channel is a real risk.
Test send	"Send test" delivers to one number without recording a campaign send.
Consent	Recipients without <code>sms_opt_in</code> , or with an unsubscribe timestamp, are excluded automatically and the exclusion count is shown. This is not optional.

14.13.3 Email composer

The same shape with **Subject** in place of campaign name, a rich-text body with a variable palette (`{{name}}` , `{{firstName}}`), a desktop and mobile preview, and a test send. Additional recipients accept addresses with the hint "Press the space key to add additional emails, e.g. *person1@example.com, person2@example.com*". Every email carries a one-click unsubscribe link and a physical postal address, both required for deliverability as much as for compliance.

14.13.4 Contacts

Table: Name · Email · Phone · Source · Email consent · SMS consent · Tags · Orders · Joined. Search across name, email and phone. Filters by source, consent and tag. Actions: import from CSV with a column-mapping step and a duplicate-handling choice; export; edit tags in bulk; unsubscribe. Customers are synchronised into contacts on registration and on order placement, carrying their consent flags with them.

14.14 Search analytics

Two tables. **Top searches**: Search query · Number of searches · Average result count · Click-through rate · Last searched. **Zero-result searches**, shown first and highlighted, since it is the actionable one: term, count, last searched, and a "Create product" shortcut that opens the product form with the term prefilled as the name. Both are filterable by date range and exportable. This turns the reference platform's read-only search table into a merchandising instrument.

14.15 Currency

A small screen matching the reference platform's: two cards showing the current **Display currency** and **Fraction digits**, and an "Update currency format" modal taking **Currency type** (ISO 4217 select) and **Maximum fraction digits** (0–4). A live preview shows a sample amount formatted with the pending settings before saving. Changing the display currency does **not** convert stored prices — a warning states this plainly, because the opposite assumption is the obvious one and would be catastrophic.

14.16 Settings

Grouped into tabs. Every field writes to `site_settings` and is audited.

Tab	Fields
Store	Store name · Legal entity name · Tagline · Logo · Favicon · Registration country · Registration agency · TIN · Certificate of incorporation number · Registered address · P.O. box.

Tab	Fields
Contact	Primary and secondary phone · Email · WhatsApp number and default message · Physical address · Map latitude and longitude · Opening hours.
Social	Facebook · Instagram · X · LinkedIn · YouTube · TikTok. Empty entries are hidden from the footer rather than rendered as dead icons.
Payments	M-Pesa till number · Mixx by Yas till number · Airtel Money till number · Other-network instructions · Merchant QR image · Per-network instruction blocks in Swahili and English, each a rich-text field · Cash-on-delivery maximum order value · Payment reservation window in hours.
Page images	Hero images for About, Privacy Policy and Terms & Conditions, with a preview each — the reference platform's own settings screen, kept. Validation: "About Us page image is required" and equivalents.
Email	From name · From address · Reply-to · SMTP host, port, user, password, TLS mode · a "Send test email" action that reports the exact failure when it fails.
SMS	Provider · API credentials · Sender ID · a "Send test SMS" action · a low-balance warning threshold.
Catalogue	Products per page · Default sort · "New" badge window in days · Low-stock default threshold · Whether to hide empty subcategories in navigation (default on, \$6.4.2).
Policies	Returns window in days · Warranty statement · The three standard invoice terms · Cancellation policy text.
Features	Toggles: reviews · comments · wishlist · guest checkout · delegated payment · back-in-stock notifications · maintenance mode with a message.

14.17 Staff and roles

Screen	Specification
Staff list	Name · Email · Roles · Last sign-in · Status · Created. Header action "Invite staff member".
Invite	Email plus role selection. Sends a single-use invitation valid 7 days; the recipient sets their own password. Staff accounts are never created with a password chosen by someone else.
Detail	Profile, roles (add and remove), effective permission list computed from the union of roles, active sessions with revoke, and that member's audit history.
Roles	List of roles with member counts. A role editor showing the full permission catalogue as a grouped checkbox matrix. System roles are visible but locked.
Guardrails	Nobody may edit their own roles. The last <code>owner</code> cannot be removed, demoted or suspended. Every change is audited with before and after permission sets. Granting <code>staff.manage</code> requires the <code>owner</code> role.

14.18 Audit log

Read-only. Columns: Timestamp · Actor · Action · Entity type · Entity · IP address · Request id. Filterable by actor, action, entity type, entity and date range; cursor-paginated because it grows without bound. Expanding a row shows a field-level diff of `before` and `after`. Exportable to CSV for a compliance request. Every entity's own detail screen carries an `AuditTrail` panel showing just its own history, which is where it is actually consulted.

What is logged: every create, update and delete on every entity; order status transitions; payment verifications and refunds; stock adjustments; permission and role changes; settings changes; customer record views; data exports; sign-in, sign-out and failed sign-in for staff accounts; and campaign sends. What is not logged: read access to non-personal data, which would swamp the useful signal.

14.19 The universal CRUD contract

Every module above conforms to this. It is what makes "full CRUD on anything" a coherent promise rather than fifteen slightly different screens.

14.19.1 Create

- Reached from a primary "New {entity}" button in the page header, or from a "+" in an empty state.
- Modal for entities with 6 or fewer fields; a full page for anything larger.
- Required fields marked with an asterisk and an explanatory legend. Optional fields are labelled "(optional)" rather than left ambiguous.
- Validation on blur, then live once a field has errored, then in full on submit. Submit is disabled until dirty and valid.
- On success: a toast "{Entity} created", and navigation either to the new record or back to the list with the new row highlighted for three seconds.
- "Save and add another" wherever bulk entry is realistic — products, categories, subcategories, FAQs.

14.19.2 Read

- Every list is a `DataTable` (§8.4.1) with search, filters, sort, pagination and density, all reflected in the URL so a filtered view is shareable with a colleague.
- Every list has a specified empty state naming the primary action.
- Every detail screen carries created and updated timestamps, the creating and last-updating actor, and an audit panel.
- Every list offers CSV export of the current filtered set, capped at 50,000 rows and delivered as a queued job with an email link when large.

14.19.3 Update

- Same form as create, prefilled, with the entity name in the heading.
- Only changed fields are sent; the payload is a partial.
- Optimistic concurrency: the form carries the record's `updatedAt`; a mismatch returns 409 and the UI offers "Reload and reapply my changes" with a field-level diff of what someone else altered.
- Navigating away with unsaved changes prompts.
- Inline editing is offered only for single scalar fields where it is genuinely faster — position, active flag, stock.

14.19.4 Delete

Rule	Specification
Soft by default	Sets <code>deleted_at</code> . Recoverable from the entity's Trash view for 90 days.
Confirmation	Always. The dialog names the specific record — "Delete <i>Arduino Uno R3</i> ?" — never "Delete this item?".
Consequences stated	The dialog lists what else is affected and the counts: "This product has 3 variants and appears in 2 open carts."
Blocked deletes	Where §9.5 says RESTRICT, the dialog explains why and offers the supported alternative rather than a bare refusal.
Bulk deletes	Require typing the exact count to confirm — "Type 14 to confirm" — using <code>ConfirmDialog</code> 's <code>requireTyping</code> mode. Slow enough to interrupt an automatic click, which is the point.
Undo	The success toast offers "Undo" for 8 seconds on single soft deletes.
Hard delete	Available only from the Trash view, only to <code>owner</code> , and never for orders, payments, invoices or audit records.

Rule	Specification
Audit	Deletion writes the full prior state to <code>audit_logs.before</code> , so a hard-deleted record can still be described even when it can no longer be restored.

14.19.5 Cross-cutting behaviours

Behaviour	Applies to
Permission gating	Every action control is wrapped in <code>PermissionGate</code> . A missing permission hides the control entirely; a conditionally unavailable action is disabled with a tooltip explaining why. Server-side enforcement is independent and mandatory.
Optimistic UI	Toggles and reordering apply immediately and revert with an error toast on failure.
Keyboard	Ctrl+K global search · / focuses the list search · N new record · Esc closes any overlay · Ctrl+S saves a form.
Deep links	Every filtered list state, every drawer and every modal is addressable by URL.
Loading	Skeletons at final dimensions, never a full-page spinner over content that is already on screen.
Errors	Field errors inline; request errors as a dismissible banner above the form carrying the request id; nothing is ever lost from the form on failure.
Timestamps	Relative for anything under 7 days ("2 hours ago") with the absolute value on hover, absolute beyond that. Always in Africa/Dar_es_Salaam.

14.19.6 Coverage matrix

The literal answer to "full CRUD on anything in the platform". **C** create, **R** read, **U** update, **D** delete, **B** bulk operations, **X** export.

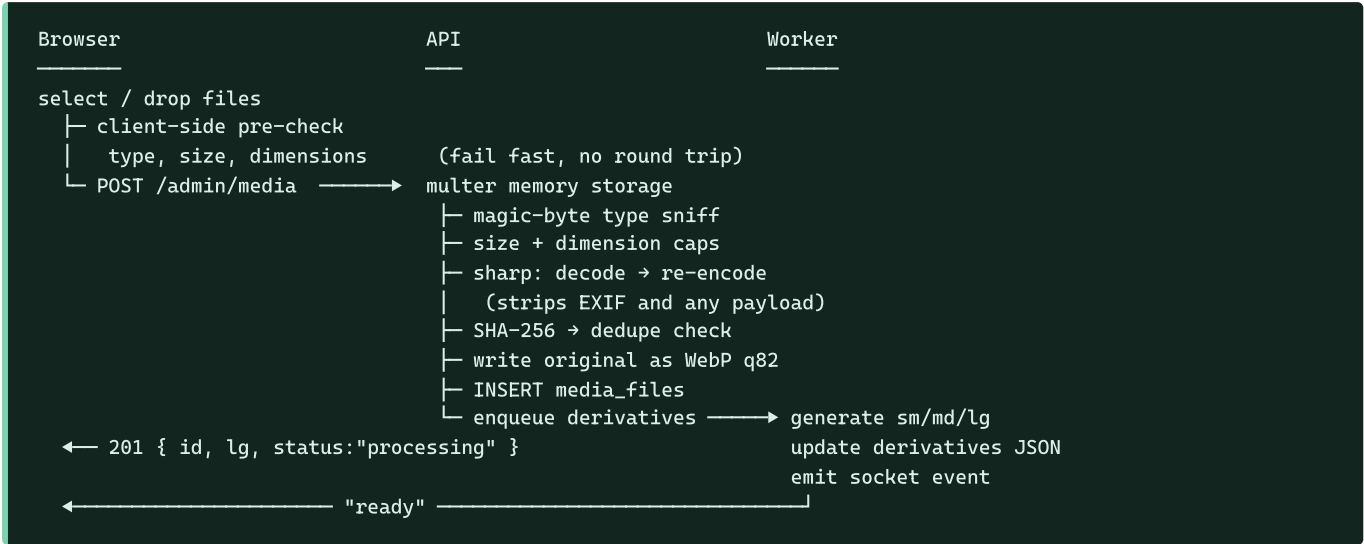
Entity	C	R	U	D	B	X	Notes
Products	✓	✓	✓	✓	✓	✓	Import too
Product variants	✓	✓	✓	✓	✓	✓	Via the matrix
Product images	✓	✓	✓	✓	✓	—	Reorder, alt text
Categories · Subcategories	✓	✓	✓	✓	✓	✓	Delete restricted by children
Discounts	✓	✓	✓	✓	✓	—	
Stock	—	✓	✓	—	✓	✓	Adjustments only; ledger is immutable
Orders	✓	✓	✓	—	✓	✓	Created by staff for phone orders; cancelled, never deleted
Order items	✓	✓	✓	✓	—	—	Only while CONFIRMED or earlier
Payments	✓	✓	✓	—	—	✓	Verify, refund; never deleted
Invoices	✓	✓	✓	✓	—	✓	Delete only while draft
Customers	✓	✓	✓	✓	—	✓	Delete anonymises
Addresses	—	✓	—	—	—	—	Customer-owned; staff read-only
Shipping methods	✓	✓	✓	✓	—	—	
Banners	✓	✓	✓	✓	✓	—	
Services · Partners · FAQs	✓	✓	✓	✓	✓	—	
Articles	✓	✓	✓	✓	✓	—	

Entity	C	R	U	D	B	X	Notes
Comments · Reviews	—	✓	✓	✓	✓	✓	Moderation only; staff never author
Campaigns	✓	✓	✓	✓	—	✓	Edit only while draft or scheduled
Contacts	✓	✓	✓	✓	✓	✓	Import and export
Media files	✓	✓	✓	✓	✓	—	Delete blocked while referenced
Search logs	—	✓	—	✓	✓	✓	Delete is a retention purge
Contact messages	—	✓	✓	✓	✓	✓	Status changes
Regions · Districts	✓	✓	✓	✓	—	—	Seeded; rarely touched
Settings	—	✓	✓	—	—	✓	Keys are fixed by the schema
Currency	—	✓	✓	—	—	—	
Staff · Roles	✓	✓	✓	✓	—	✓	System roles locked
Audit logs	—	✓	—	—	—	✓	Insert-only at the database level

Where a cell reads "—", it is a deliberate refusal with a stated reason, not a missing feature. Financial and audit records are never editable or deletable, because a system in which they are is not a system anyone can rely on.

Media Pipeline

15.1 Upload flow



The response returns before derivatives exist so the interface is never blocked by image processing. The uploader shows a "processing" shimmer on the thumbnail until the ready event arrives, or until a poll confirms it.

15.2 Derivative sizes

Key	Width	Fit	Used by
sm	200 px	contain, white pad	Cart and wishlist rows, gallery thumbnails, admin table rows
md	600 px	contain, white pad	Product cards in grids and carousels
lg	1400 px max	inside, no upscale	Product detail hero, lightbox, banners

All derivatives are WebP at quality 82, with a JPEG fallback generated only if analytics ever show meaningful traffic from browsers without WebP support — as of now, none does. Names are content-addressed: `{publicId}-{key}-{checksum8}.webp`, so a re-upload can never be served from a stale cache. Cache headers are `public, max-age=31536000, immutable`.

TWO CORRECTIONS TO THE REFERENCE PLATFORM'S PIPELINE

It applies a *face-detection* crop (`g_face`) to photographs of circuit boards, which is meaningless and occasionally crops the subject out. TechStar uses `contain` with white padding, so a component photographed on a white background is never cut. And its three sizes carry inconsistent version segments, so they can drift apart after a re-upload; content-addressed names make that impossible.

15.3 Storage adapters

```
interface StorageAdapter {
  put(key: string, body: Buffer, contentType: string): Promise<void>
  get(key: string): Promise<Buffer>
  delete(key: string): Promise<void>
  exists(key: string): Promise<boolean>
  publicUrl(key: string): string
}
```

Adapter	Behaviour
local	Writes under <code>MEDIA_ROOT</code> , outside the web root, sharded two levels by the ULID's first four characters so no directory grows unbounded. Served by Nginx with execution disabled. The default.
s3	Any S3-compatible endpoint. Objects private; served through CloudFront or an equivalent, or by signed URL.
cloudinary	For parity with the reference platform's hosting, should it ever be wanted. Derivatives are still generated locally so the URL contract does not change.

15.4 Cleanup and orphans

- `media_files.reference_count` is incremented and decremented in the same transaction as the referencing row.
- A nightly job soft-deletes files with zero references older than 24 hours — the delay covers an upload whose form was never submitted.
- A weekly job hard-deletes soft-deleted files older than 30 days and removes their bytes.
- A monthly reconciliation compares storage keys against database rows in both directions and reports discrepancies rather than acting on them.
- Deleting a referenced file is refused by the API before the foreign key has to intervene, with a message naming what references it.

Search, Filtering, Sorting, Pagination

16.1 Engine choice

MYSQL FULL-TEXT AT LAUNCH; A DEDICATED ENGINE ONLY WHEN MEASUREMENT DEMANDS IT

At 617 products — and at ten times that — MySQL's InnoDB full-text index with the tuning in §9.4 returns results in single-digit milliseconds. Adding Elasticsearch or Meilisearch on day one buys typo tolerance and synonyms at the cost of a second datastore to run, secure, back up and keep in sync. The trigger for revisiting is explicit: the catalogue passes 20,000 products, or the zero-result rate exceeds 15% and is traceable to matching rather than to genuine catalogue gaps. The search service is written behind an interface so the swap is contained.

16.2 Query grammar and ranking

A search runs four passes and returns them in this order, deduplicated. Ordering by relevance means ordering by which pass matched.

Pass	Match	Rationale
1	Exact SKU, or exact manufacturer part number	Someone typing <code>74HC14</code> or <code>0510003</code> wants that part and nothing else. This pass short-circuits: a hit here returns immediately.
2	Name prefix (<code>name LIKE 'term%'</code>)	Prefix matches are near-certain intent.
3	Name contains	Catches mid-string matches like "uno" in "Arduino Uno R3".
4	<code>MATCH ... AGAINST</code> in boolean mode over name, description, brand and MPN	Full-text relevance, with name weighted 4×, brand and MPN 2×, description 1×.

Rule	Specification
Normalisation	Lowercase, trim, collapse whitespace, strip punctuation except <code>-</code> , <code>.</code> and <code>/</code> , which carry meaning in part numbers.
Multi-word	All terms required (boolean <code>+</code>), then relevance-ordered. "arduino uno" must not return every Arduino.
Short tokens	<code>innodb_ft_min_token_size = 2</code> with an empty stopword table, so "IC", "DC" and "5V" are searchable.
Suggestions	Spelling suggestions come from a term dictionary built nightly from product names, matched by trigram similarity within an edit distance of 2, and only offered when the original returned nothing.
Synonyms	A small hand-maintained table for this trade: resistor ↔ resistance, capacitor ↔ cap, LED ↔ light emitting diode, pot ↔ potentiometer, uC ↔ microcontroller, PSU ↔ power supply. Editable in the admin console.
Filters	Applied as SQL predicates after matching, never in the application layer, so pagination totals stay correct.
Highlighting	Performed client-side on the returned names, so the server does not have to escape and return HTML.

16.3 Facets

Facet counts are computed in one additional aggregate query against the same filtered set, so a count never contradicts the grid beneath it. Price bands are derived from the actual distribution in scope — quartiles rounded to readable boundaries — rather than fixed bands that would be empty for most subcategories.

16.4 Search analytics

Signal	Captured how, used for what
Term and result count	Written once per distinct term per session, on submit — not per keystroke, which is what makes the reference platform's search table too noisy to act on.
Zero results	Surfaced first in the admin console with a "Create product" shortcut. This is the highest-value report in the system: it is customers telling you exactly what to stock.
Click-through	<code>clicked_product_id</code> written when a result is opened. Terms with many searches and no clicks indicate matching that finds the wrong things.
Trends	Week-over-week movement per term, to catch seasonal and course-driven demand.

16.5 Pagination contracts

Context	Strategy
Catalogue, search, admin lists	Offset pagination with a total count. Page size default 24, maximum 100. Predictable page numbers matter more than raw speed at this scale. Fixes defect R17.
Audit log, order events, search logs	Cursor pagination on <code>(created_at, id)</code> . These grow without bound and a deep offset scan on them is a slow query waiting to happen.
Exports	Streamed in 1,000-row chunks; never materialised in memory.
Carousels	Fixed 24-item fetch, no pagination.

Messaging: Email, SMS, Realtime, Campaigns

17.1 Transactional email

Trigger	Contents
Welcome / verify email	Verification link, 24-hour validity stated.
Order placed	Order number, items, totals, delivery method and address, payment instructions for the chosen method.
Payment received (recorded)	Acknowledgement that details were received and verification is pending.
Payment confirmed	Receipt with the transaction reference.
Order shipped	Tracking reference where available, estimated delivery window.
Order delivered	Thank-you, plus a review invitation sent separately 3 days later.
Order cancelled / expired	Reason, and a one-click reorder link.
Delegated payment request	To the nominated payer: who asked, what for, how much, the payment link.
Password reset · Password changed	Reset link; and a security notice to the address on file after any change.
Back in stock	To everyone on <code>notify_requests</code> for that variant.
Invoice sent	PDF attached plus the share link.
Staff invitation	Single-use link, 7-day validity.
Low stock digest	Daily to staff holding <code>stock.read</code> .

Templates are React Email components rendering to both HTML and plain text, sharing one layout: TechStar header, content, footer with contact details and, for marketing mail only, an unsubscribe link. Transactional mail carries no unsubscribe, because a receipt is not marketing. SPF, DKIM and DMARC must be configured before launch; without them, delivery to Gmail addresses is unreliable regardless of code quality.

17.2 SMS

SMS matters more than email in this market — a Tanzanian customer will see an SMS within minutes and an email possibly never. Sent for: order placed, payment confirmed, order shipped, order ready for pickup, and the 36-hour expiry warning. Kept under 160 GSM-7 characters so each is one segment. The adapter interface:

```
interface SmsProvider {
  send(to: string, message: string, opts?: { senderId?: string })
    : Promise<{ messageId: string; segments: number; cost?: Money }>
  sendBulk(messages: { to: string; message: string }[])
    : Promise<BulkResult>
  deliveryStatus(messageId: string): Promise<DeliveryStatus>
  balance(): Promise<Money>
}
```

Implementations for Beem Africa and NextSMS, plus a console stub for development. Numbers are canonicalised to `255XXXXXXX` before sending. A sender ID must be registered with the operators in advance — allow two to three weeks

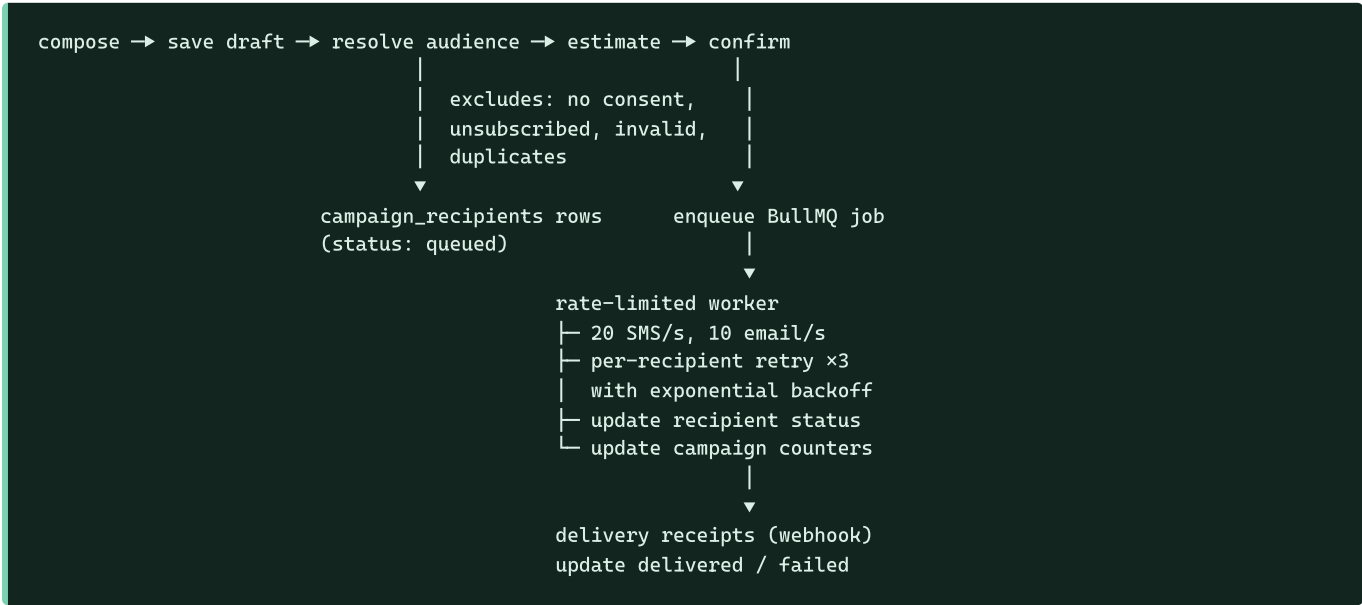
for this in the launch plan, because it cannot be hurried and blocks go-live.

17.3 Realtime events

Channel	Events
admin:orders	order.placed , order.payment_recorded , order.cancelled — drives the queue insertion and the chime.
admin:alerts	stock.low , webhook.failed , comment.pending , review.pending — drives the bell badge.
order: {orderNumber}	status.changed , payment.confirmed — drives the live strip on the confirmation page.
user:{publicId}	cart.updated for multi-tab and multi-device consistency.

Every connection is authenticated with the access token at handshake and re-authenticated on refresh; channel subscription is authorised per channel. This closes defect R9's second half. Redis is used as the Socket.IO adapter so events reach every application instance.

17.4 Campaign engine



A scheduled campaign is a delayed job; cancelling before it fires removes it. A campaign in `sending` can be paused, which stops the worker taking new recipients without losing those already sent. Every send is audited with actor, audience size and cost.

17.5 Queues

Queue	Jobs · concurrency · retry
email	Transactional sends · 5 · 3 attempts, exponential backoff
sms	Transactional sends · 5 · 3 attempts
campaign	Bulk fan-out · 2 · per-recipient retry
media	Derivative generation · 3 · 2 attempts

Queue	Jobs · concurrency · retry
webhook	Payment event processing · 5 · 5 attempts
scheduled	Reservation expiry (every 5 min) · stock digest (daily 07:00) · media cleanup (nightly) · session purge (nightly) · search dictionary rebuild (nightly) · payment status sweep (every 30 min) · financial invariant check (nightly)

Every queue has a dead-letter queue and an admin view listing failed jobs with their error and a retry action. A failed job that nobody can see is a silent outage.

Analytics and Reporting

18.1 Metric definitions

Ambiguity in a metric definition is worse than not having the metric. These are normative.

Metric	Definition
Revenue	Sum of <code>total_amount</code> for orders whose <code>paid_at</code> falls in the period. Recognised on payment, not on placement — an unpaid order is not revenue.
Orders	Count of orders whose <code>placed_at</code> falls in the period, excluding <code>EXPIRED</code> . Cancelled orders are included, because cancellation rate is itself a signal.
Average order value	Revenue ÷ paid order count.
Conversion	Orders placed ÷ distinct sessions that added at least one item to a cart.
Units sold	Sum of <code>order_items.quantity</code> for paid orders.
New customers	Users whose <code>created_at</code> falls in the period.
Repeat rate	Customers with two or more paid orders ÷ customers with at least one, over a trailing 12 months.
Search success	Searches with at least one result and a click ÷ all searches.
Payment lag	Median minutes from <code>placed_at</code> to <code>paid_at</code> for Lipa Namba orders.
Fulfilment lead time	Median hours from <code>paid_at</code> to <code>shipped_at</code> .

18.2 Aggregation approach

Dashboard queries run against a small set of hand-written SQL statements using window functions and CTEs — this is where the ORM is set aside deliberately. Two things keep them fast: a date-spine CTE so that periods with no orders still produce a zero row (otherwise charts silently skip days), and the denormalised `paid_at`, `shipped_at` and `delivered_at` columns on `orders`, which avoid joining the event log for every chart.

```
-- Revenue and order count by month, last 12 months, with zero-filled gaps
WITH RECURSIVE spine AS (
  SELECT DATE_FORMAT(DATE_SUB(CURDATE(), INTERVAL 11 MONTH), '%Y-%m-01') AS d
  UNION ALL
  SELECT DATE_ADD(d, INTERVAL 1 MONTH) FROM spine
  WHERE d < DATE_FORMAT(CURDATE(), '%Y-%m-01')
)
SELECT DATE_FORMAT(s.d, '%Y-%m') AS bucket,
       COALESCE(SUM(o.total_amount), 0) AS revenue,
       COUNT(o.id) AS orders
FROM   spine s
LEFT JOIN orders o
  ON o.paid_at >= s.d
  AND o.paid_at < DATE_ADD(s.d, INTERVAL 1 MONTH)
  AND o.payment_status = 'paid'
GROUP BY bucket
ORDER BY bucket;
```

Results are cached in Redis for 5 minutes, keyed by period and filter. Above roughly 100,000 orders, a nightly rollup into a `daily_metrics` table replaces the live aggregation; the API contract does not change when that happens.

18.3 Charts

Chart.js, following the house rules: one hue family for a single series; the categorical palette only when series are genuinely unrelated; axes always labelled with units; the y-axis always starting at zero for bar charts; no more than six series on one chart; and a "View as table" toggle on every chart so the data is reachable without sight. Money on axes is abbreviated (TZS 1.2M) and shown in full in tooltips.

18.4 Exports

Export	Columns
Orders	Number, date, customer, phone, email, region, district, method, payment method, payment status, fulfilment status, subtotal, shipping, total, paid, items count
Order lines	Order number, date, SKU, product, attributes, unit price, quantity, line total
Products	SKU, name, category, subcategory, brand, MPN, price, compare-at, cost, stock, reserved, status, units sold, created
Stock movements	Date, SKU, product, delta, balance after, reason, reference, actor, note
Customers	Name, email, phone, verified, region, orders, lifetime value, first order, last order, consent flags
Payments	Date, order, method, provider, reference, amount, status, payer phone, verified by
Searches	Term, searches, average results, click-through rate, last searched

UTF-8 with a BOM so Excel opens Swahili and symbol characters correctly, comma-separated, dates in ISO 8601, money as plain integers without a currency symbol so spreadsheets can sum them. Exports over 5,000 rows run as a queued job and are delivered by an emailed download link valid 24 hours.

Frontend Architecture

19.1 Folder structure

apps/web/src/	
├─ main.tsx	entry, providers, error boundary
├─ router.tsx	route tree, guards, lazy boundaries
├─ app/	
│ ├─ providers/	query client, auth, toast, currency, i18n
│ └─ layouts/	StorefrontLayout, AdminLayout, AccountLayout
├─ features/	vertical slices – the primary organising axis
│ ├─ catalogue/	components, hooks, api, types
│ └─ product/	
│ ├─ cart/	
│ ├─ checkout/	
│ ├─ orders/	
│ ├─ account/	
│ ├─ auth/	
│ ├─ content/	
│ └─ admin/	
│ ├─ products/	orders/ customers/ categories/ banners/
│ └─ campaigns/	settings/ staff/ analytics/
│ └─ shared/	DataTable, FormLayout, PermissionGate
├─ components/	cross-feature composites
├─ hooks/	useDebounce, useMediaQuery, useLocalStorage...
├─ lib/	apiClient, formatters, validators, analytics
├─ stores/	zustand slices
├─ styles/	tokens.scss, bootstrap overrides, globals
└─ locales/	en.json, sw.json

Features are vertical slices, not horizontal layers. Everything about checkout lives in `features/checkout`. A file is promoted to `components/` only when a second feature imports it — moving code up when it is shared beats guessing what will be shared.

19.2 State management

State	Where	Why
Server data	TanStack Query	Products, orders, customers, settings. Cache keyed by request, invalidated by mutation. Directly fixes defects R4 and R5.
Cart, wishlist	Zustand + server sync	Needs optimistic local mutation and cross-tab sync; the server remains authoritative.
Auth session	Zustand, memory only	Access token never leaves memory (§11.2).
UI state	Zustand	Off-canvas open, modal mode, sidebar collapsed, toast queue.
Filters, pagination, tabs	URL query string	Shareable, bookmarkable, back-button correct. Never component state.
Form state	React Hook Form	Uncontrolled inputs, minimal re-render, Zod resolver.
Preferences	localStorage	Grid density, recently viewed, dismissed banners.

```
// stores/cart.ts – the shape ported from the reference platform's Pinia store,
// with the increment defect (R3) removed and server sync added.
interface CartStore {
  lines: CartLine[]
  notices: CartNotice[]
  isSyncing: boolean
  add(variantSku: string, quantity?: number): Promise<void>
  setQuantity(variantSku: string, quantity: number): Promise<void>
  remove(variantSku: string): Promise<void>
  clear(): Promise<void>
  hydrate(): Promise<void> // localStorage first paint, then server truth
  merge(): Promise<MergeResult> // after sign-in, §12.5.4
  get subtotal(): Money
  get itemCount(): number
}
```

19.3 Data fetching

Concern	Rule
Query keys	Structured arrays: <code>['products', {subcategory, sort, page}]</code> . Built by a keys factory per feature so no two call sites can disagree.
Stale times	Categories and settings 30 min · products 5 min · cart 0 (always fresh) · orders 1 min · admin lists 0.
Invalidation	Every mutation declares the key prefixes it invalidates. A mutation with no invalidation list fails review.
Optimistic updates	Cart quantity, wishlist toggle, admin active toggles and reordering. Always with a rollback path.
Prefetch	Product detail is prefetched on card hover or focus after a 150 ms delay; the next catalogue page is prefetched when the current one is 80% scrolled.
Retry	Twice with exponential backoff for GET; never for POST, PATCH or DELETE, which is what idempotency keys are for instead.
Error handling	A single API client maps error codes to typed errors; 401 triggers one queued refresh and a replay; 403 and 5xx surface through the error boundary.

19.4 Forms and validation

One Zod schema per form, imported from `packages/contracts` — the same object the server validates against. Mode is `onTouched` then `onChange` once a field has errored. Server-side field errors from a 422 are mapped back onto form fields by name so they appear in the same place as client-side ones. Submit is disabled while pending and while the form is pristine. Every form guards against navigation with unsaved changes.

19.5 Error boundaries

Three levels: a root boundary rendering the 500 page with the request id; a route-level boundary so one failing route does not blank the shell; and section-level boundaries around each independently loaded home-page rail (§7.2.2), so one failed request degrades one section. Boundaries report to the error tracker with the request id, the route and a redacted breadcrumb trail.

19.6 Code splitting and performance

- Route-level lazy loading for every route; the admin console is a single separate chunk group that a customer never downloads.
- Heavy libraries are loaded only where used: the rich-text editor, the chart library, FilePond and Leaflet are all dynamic imports.
- Initial storefront bundle budget: **180 KB gzipped** of JavaScript. The reference platform ships a 2 MB vendor bundle on first paint; the largest single contributor is a date library that has a 20× smaller replacement.
- Images lazy below the fold, eager for the banner and the product hero, always with explicit dimensions.
- Fonts self-hosted, preloaded, subset, `font-display: swap`.
- `react-window` for any list beyond 200 rows — the admin product table at scale.

19.7 Accessibility

WCAG 2.1 AA is the target and is testable. Landmark regions on every page; one `h1` per page with no skipped levels; a visible skip link; focus never lost or trapped except deliberately in modals and off-canvas panels; a visible 2 px focus ring everywhere; all interactive elements reachable and operable by keyboard; 44 px minimum touch targets; live regions for cart changes, search result counts and form errors; no information conveyed by colour alone — every status pill carries text; all images have meaningful alt text or are explicitly decorative; forms use real labels, never placeholders as labels. `eslint-plugin-jsx-a11y` runs in CI and axe runs against every Storybook story.

19.8 Internationalisation

`react-i18next`, English shipped, Swahili scaffolded. All copy resolved through `t()` from day one — retrofitting this into a finished application costs many times what it costs now. Interpolation for names and counts; ICU plurals; dates and money through `Intl` with the Africa/Dar_es_Salaam zone and the configured currency. The payment instruction blocks ship in both languages immediately (\$7.8.1), because that is where a mistranslation costs an order.

Backend Architecture

20.1 Folder structure

apps/api/src/	
├─ server.ts	http server, socket gateway, graceful shutdown
├─ app.ts	express app, middleware pipeline (§4.3)
├─ config/	zod-validated environment, fails fast at boot
├─ modules/	one folder per bounded context
│ ├─ auth/	
│ │ ├─ catalogue/	
│ │ │ ├─ cart/	
│ │ │ │ ├─ orders/	
│ │ │ │ │ └─ payments/	
│ │ │ │ └─ invoices/	
│ │ │ │ ├─ customers/	
│ │ │ │ │ ├─ shipping/	
│ │ │ │ │ │ ├─ content/	
│ │ │ │ │ │ │ └─ campaigns/	
│ │ │ │ │ └─ media/	
│ │ │ │ ├─ search/	
│ │ │ │ │ ├─ analytics/	
│ │ │ │ │ │ ├─ settings/	
│ │ │ │ │ │ └─ audit/	
│ │ │ │ └─ each:	routes.ts · controller.ts · service.ts · repository.ts
│ │ │ │ └─ schemas.ts	
│ │ │ │ └─ events.ts	
│ │ │ │ └─ __tests__/	
├─ middleware/	auth, rbac, validate, rateLimit, requestId, error
├─ infra/	
│ └─ db/	prisma client, transaction helper
│ └─ cache/	redis
│ └─ queue/	bullmq queues and workers
│ └─ storage/	storage adapters (§15.3)
│ └─ mail/	
│ │ └─ sms/	provider adapters
│ └─ payments/	payment provider adapters (§13.3)
│ └─ realtime/	socket.io gateway
├─ domain/	pure logic: state machines, pricing, SKU, stock
├─ jobs/	scheduled job definitions
└─ lib/	logger, errors, money, phone, slug, ids

20.2 Layering

Strictly one direction: `routes` → `controller` → `service` → `repository` → `Prisma`. Controllers know HTTP and nothing about business rules. Services own invariants and transaction boundaries and know nothing about HTTP. Repositories know Prisma and nothing about rules. `domain/` is pure and has no imports from any other layer, which is what makes the order state machine, the pricing resolver and the SKU allocator testable without a database. Violations are caught by an import-boundary lint rule.

20.3 Transactions

Operation	Must be atomic across
Place order	Order · items · address · reservations on every variant · stock movements · order event · cart clear · idempotency record
Verify payment	Payment · order payment status and lifecycle · stock deduction · movements · order event · notification enqueue
Cancel order	Order status · reservation release · movements · order event
Create product	Product · attributes · variants · images · SKU allocation · counter updates on subcategory and category
Delete category	Precondition check · soft delete · counter updates · audit record
Approve review	Review status · product <code>review_count</code> and <code>average_rating</code> · audit record

Rules: lock rows in a consistent order — always ascending by primary key — to avoid deadlock; keep transactions short and never perform network I/O inside one; enqueue side effects rather than executing them, so a failed email never rolls back a paid order; and set an explicit 5-second timeout with a retry for deadlock victims.

20.4 Error taxonomy

```
AppError (base: code, httpStatus, message, details?, isOperational)
├─ ValidationError          422    from the Zod middleware
├─ UnauthenticatedError     401
├─ ForbiddenError          403
├─ NotFoundError           404
├─ GoneError               410
├─ ConflictError           409
├─   └─ PriceChangedError
├─   └─ InsufficientStockError
├─   └─ InvalidStateTransitionError
├─ RateLimitError          429
└─ ExternalServiceError     502    provider unreachable or malformed
```

Anything else is non-operational: logged with a full stack, reported to the error tracker, and returned to the client as a generic 500 with a request id.

20.5 Logging and observability

Structured JSON via `pino`. Every line carries `requestId`, `userId` where known, route, method, status and duration. A serializer redacts `password`, `token`, `authorization`, `refreshToken`, `paymentNumber` and any key matching `/secret|key|hash/i`. Levels: `error` for failures needing attention, `warn` for handled anomalies such as a rejected webhook signature, `info` for business events (order placed, payment verified, campaign sent), `debug` off in production. OpenTelemetry traces span HTTP, database and queue work, so a slow checkout can be attributed to a specific query.

20.6 Caching

Cached	TTL	Invalidated by
Category and subcategory tree	30 min	Any category or subcategory write
Public settings	30 min	Settings write
Shipping methods	30 min	Shipping write
Home page rails	5 min	Product or banner write
Product detail	5 min	Write to that product or its variants
Analytics aggregates	5 min	Time only
Regions and districts	24 h	Manual

Cart, orders, customers, payments and anything user-specific are never cached. Cache keys are namespaced with a schema version so a deployment that changes a response shape does not serve stale structures.

20.7 Configuration

One `config/index.ts` parses `process.env` through a Zod schema at boot and exports a frozen, typed object. A missing or malformed variable stops the process with a message naming the variable — never a runtime `undefined` three hours later. No other file may read `process.env`, enforced by lint. The full variable list is Appendix D.

Quality: Testing, Performance, SEO

21.1 Test strategy

Level	Target	Scope
Unit — domain	95% of domain/	Order state machine (every legal and illegal transition), pricing and discount resolution, SKU allocation, stock arithmetic, phone canonicalisation, slug generation, money formatting. Pure functions, no database, milliseconds to run.
Unit — components	80%	Every component in every state: loading, empty, error, populated. Plus an axe assertion per story.
Integration — API	Every endpoint	Supertest against a real MySQL schema in Docker. Each endpoint tested for success, validation failure, unauthenticated, unauthorised, and not-found.
Integration — data	Key invariants	Concurrent add-to-cart on the last unit; double webhook delivery; duplicate idempotency key; cascade and restrict behaviour; the financial invariant of \$13.6.
End-to-end	7 journeys	Playwright, Chromium and mobile Chrome: guest purchase with Lipa Namba · signed-in purchase with saved address · delegated payment · admin creates a product with variants · admin verifies a payment and ships · search to purchase · registration, verification and password reset.
Visual	All stories	Storybook snapshots at three widths, diffed in CI.
Load	Before launch	k6: 200 concurrent browsers on the catalogue; 20 orders per minute sustained; the search endpoint at 100 requests per second.
Security	Each release	Dependency audit, an automated ZAP baseline scan, and a manual authorisation matrix test asserting that a customer token is rejected by every admin route.

Test data comes from typed factories, never from fixtures copied between tests. Each test creates only what it asserts on. The database is truncated between tests, not re-migrated.

21.2 Performance budgets

Metric	Budget	Measured
Largest Contentful Paint, mobile 3G	< 2.5 s	Lighthouse CI on every pull request
Interaction to Next Paint	< 200 ms	Field data
Cumulative Layout Shift	< 0.1	Lighthouse CI
Initial JS, gzipped	< 180 KB	Bundle analysis, CI fails on regression
API p95, reads	< 200 ms	Server metrics
API p95, order creation	< 800 ms	Server metrics
Search p95	< 150 ms	Server metrics
Slow query threshold	> 100 ms	MySQL slow log with alerting

21.3 SEO

The catalogue is the acquisition channel, so the client-rendered decision (D3) must be paid for deliberately. Public routes — home, catalogue, every product, every article, every content page — are **prerendered at build time and revalidated on content change**, producing static HTML that crawlers and slow devices receive immediately while React hydrates over it. Beyond that: unique title and meta description per page; canonical URLs; Open Graph and Twitter cards with the product image; JSON-LD for Product, Offer, BreadcrumbList, Review, Organisation and FAQPage; a generated `sitemap.xml` segmented by type and refreshed nightly; a `robots.txt` excluding `/admin`, `/account`, `/checkout`, `/pay` and `/invoice`; 301s from retired slugs; and a PWA manifest with offline support for previously viewed pages.

Deployment and Operations

22.1 Topology

Production (single VPS is adequate to ~50k monthly sessions)

Nginx	:443	TLS (Let's Encrypt, auto-renew)
/		→ SPA static build, immutable cache
/api		→ 127.0.0.1:4000
/socket.io		→ 127.0.0.1:4000 (upgrade)
/media		→ /var/lib/techstar/media (exec off)

PM2:	api ×2 (cluster)	·	worker ×1	·	scheduler ×1
------	------------------	---	-----------	---	--------------

MySQL 8	(local socket, 2 GB buffer pool)
Redis 7	(maxmemory 512 MB, allkeys-lru for cache DB, noeviction for the queue DB – never evict a job)

Scaling path, in order: move MySQL to its own host → add a read replica for analytics → move media to S3 + CDN → run the API behind a load balancer.

22.2 CI/CD

on pull request	typecheck · lint · unit · integration · build bundle-size check · Lighthouse CI · axe npm audit (fail on high/critical)
on merge to main	build → staging → migrate → smoke → e2e
on tag v*	build → manual approval → production ├ backup database ├ prisma migrate deploy ├ pm2 reload (zero downtime) ├ smoke tests └ rollback on failure

The production build strips all non-production configuration — no development hostnames, no localhost ports (fixing R10), no source maps served publicly, though they are uploaded to the error tracker. Secrets come from the deployment environment and never from the repository.

22.3 Migrations

Prisma Migrate, forward-only, reviewed in the pull request that needs them. Rules: additive first — add a column nullable, backfill in a job, then enforce `NOT NULL` in a later release; never drop a column in the same release that stops writing to it; every destructive migration is preceded by a verified backup; and migrations run automatically on deploy with an advisory lock so two instances cannot race.

22.4 Backups and disaster recovery

Item	Policy
Database	<code>mysqldump --single-transaction</code> nightly, plus binary logs for point-in-time recovery. Encrypted, off-site, 30 daily / 12 monthly / 7 yearly.
Media	Nightly incremental sync to off-site object storage.
Restore drill	Quarterly, mandatory. Restore to a scratch host and verify order counts and totals. An untested backup is not a backup.
RPO / RTO	24 hours / 4 hours for a full rebuild from backup; 15 minutes / 1 hour with binary logs available.

22.5 Monitoring and runbooks

Uptime checks on `/`, `/api/v1/health` and `/api/v1/health/deep` (which verifies MySQL, Redis and storage). Alerts on: 5xx rate above 1% for 5 minutes; p95 latency above 1 second; queue depth above 1,000; failed webhook count above zero; disk above 80%; MySQL connections above 80% of maximum; SMS balance below the threshold; a failed nightly backup; and any financial-invariant violation. Written runbooks cover payment provider outage, SMS provider outage, database restore, stuck queue, high traffic event, and a suspected security incident — each naming who to call and what to say to customers.

22.6 Local development on XAMPP

The workstation already runs XAMPP, and its MySQL is a perfectly good development database. Start MySQL from the XAMPP control panel, create the `techstar` schema with `utf8mb4`, point `DATABASE_URL` at `mysql://root@127.0.0.1:3306/techstar`, and run migrations and the seed. phpMyAdmin at `localhost/phpmyadmin` is then available for inspection. Apache is not needed — Vite serves the front end and Express serves the API. Redis is the one thing XAMPP does not provide: run it under Docker, or under WSL, or use the in-memory queue driver, which is supported for development only.

Delivery Plan

23.1 Phases

Phase	Outcome	Contents
0	Foundations	Repository, workspaces, tooling, CI. Prisma schema and migrations for all 50 tables. Seed: taxonomy, regions, districts, roles, permissions, settings. Design tokens and the primitive component set with Storybook. Auth end to end. <i>Exit: a developer can clone, install, migrate, seed and sign in.</i>
1	Catalogue	Categories, subcategories, products with variants, images and media pipeline. Admin CRUD for all of it. Storefront home, catalogue, product detail, search. <i>Exit: the full reference catalogue shape is enterable and browsable.</i>
2	Commerce	Cart, wishlist, checkout, shipping methods, order creation with revalidation and reservation, order state machine, manual Lipa Namba, cash on delivery, admin order queue and payment verification, transactional email and SMS. <i>Exit: a real order can be placed, paid and shipped.</i>
3	Accounts & content	Profile, addresses, order history, reviews, banners, articles, comments, services, partners, FAQs, content pages, moderation queues, invoices, delegated payment. <i>Exit: feature parity with the reference platform.</i>
4	Operations & growth	Dashboard analytics, search analytics, campaigns, contacts, staff and roles, audit log, exports, aggregator payment adapter, prerendering and SEO, PWA. <i>Exit: the business can be run and measured from the console.</i>
5	Hardening & launch	Load testing, security review, accessibility audit, restore drill, runbooks, staff training, content population, sender-ID registration, soft launch, launch. <i>Exit: live.</i>

23.2 Estimate

Ranges for one full-stack engineer working steadily, and for a team of three (two full-stack, one front-end-leaning). They assume the design system is built once in Phase 0 and reused, which is where most of the compression comes from.

Phase	Solo	Team of three	Critical path
0	2–3 weeks	1–1.5 weeks	Schema decisions; everything downstream depends on them
1	4–5 weeks	2–2.5 weeks	The variant matrix; it is the hardest single screen
2	5–6 weeks	2.5–3 weeks	Order state machine, stock concurrency, payment reconciliation
3	4–5 weeks	2–2.5 weeks	Breadth rather than depth; highly parallelisable
4	4–5 weeks	2–2.5 weeks	Campaigns and the aggregator integration, both with external dependencies
5	2–3 weeks	1.5–2 weeks	Sender-ID registration and content population are not engineering work and must start in Phase 3
Total	21–27 weeks	11.5–14 weeks	

23.3 Acceptance criteria

A phase is complete only when all of the following hold, not when the screens look finished.

- Every endpoint in scope has integration tests covering success, validation, unauthenticated, unauthorised and not-found.
- Every screen in scope has its loading, empty, error and populated states implemented and covered by a Storybook story.
- Domain logic in scope is at 95% unit coverage.
- Lighthouse and axe pass at the budgets in §21.2 with no new violations.
- No high or critical dependency advisory.
- Every admin action in scope writes an audit record.
- The relevant end-to-end journeys pass on Chromium and mobile Chrome.
- Any deviation from this specification is written down, with its reason, in a decision record.

23.4 Risk register

Risk	Impact	Mitigation
SMS sender-ID registration takes weeks	High	Start in Phase 3, not Phase 5. It blocks launch and cannot be accelerated. Have an email-only fallback path ready.
Manual reconciliation does not scale past ~50 orders/day	High	Aggregator adapter designed in Phase 2, implemented in Phase 4. Watch the payment-lag metric; it is the trigger.
Catalogue population is slower than expected	High	CSV import in Phase 1, not later. Product entry time is a tracked metric (§3.6).
Payment provider onboarding is slow or is refused	Medium	Launch does not depend on it. Approach two providers in parallel.
Mobile money USSD menus change	Medium	Instructions are editable settings, not code (§14.16). Re-verify quarterly.
Legal copy not ready at launch	Medium	Assign in Phase 0. Privacy and Terms need legal review, which has its own lead time.
Product photography inconsistent	Medium	Agree a shooting standard — white background, square, 1200 px minimum — before Phase 1 bulk entry, not after.
Scope creep into marketplace or multi-currency	Medium	§3.5 states the non-goals. Changes go through a written decision record.
Single-server topology becomes a bottleneck	Low	The scaling path in §22.1 is ordered and requires no re-architecture.

Catalogue Taxonomy

The recommended launch taxonomy for a STEM and electronics retailer: **17 categories** and **102 subcategories**, each with its positional SKU code per §12.1. This is a structural classification of the electronics component trade, not proprietary content — it is the right starting shape, and TechStar should add, rename and reorder freely once real stock is known. Codes are assigned so that a subcategory code always begins with its parent category code. Gaps in the sequence are intentional and reserved.

Code	Category	Subs	Subcategories
00	Components	9	0000 Connectors & sockets · 0001 Prototyping boards & PCBs · 0002 Displays · 0003 Switches & keypads · 0004 Passive devices · 0005 Circuit protection · 0006 Semiconductors · 0007 Electromechanics · 0008 Wires & cables
01	Hardware Development Tools & Platforms	4	0100 Compatible Arduino® boards & accessories · 0101 Programmers and debuggers · 0102 Development kits · 0111 Raspberry Pi boards & accessories
02	Integrated Circuits	9	0200 4000 Series · 0201 7400 Series · 0202 OpAmps · 0203 A/D & D/A Converters · 0204 Memory chips · 0205 Timers · 0206 Drivers · 0207 Programmable devices · 0208 Special function
03	Microcontrollers & Microprocessors	5	0300 AVR · 0301 PIC · 0302 ARM · 0303 8051 · 0304 MSP
04	Power Supply & Switching Boards	3	0400 AC--DC · 0401 DC--DC · 0402 Switching
05	Sensors & Conditioning Boards	17	0500 Liquid · 0501 Pressure · 0502 Vibration · 0503 Sound · 0504 Temperature · 0505 Touch · 0506 Gas · 0507 Biometric · 0508 Object & Metal · 0509 Current · 0510 Voltage · 0511 Motion · 0512 Dust · 0513 Distance · 0514 Weight/Stress · 0515 Light & Radiation · 0516 Temperature/Humidity
06	Tools & consumables	14	0600 Screw drivers · 0601 Fasteners · 0602 Soldering tools & cleaners · 0603 PCB processing tools · 0604 Strippers, crimpers & cutters · 0605 Pliers · 0606 Drilling machines, rotary drivers & accessories · 0607 Tweezers · 0608 Hot glue gun & sticks · 0609 Helping hands · 0610 Magnifying glass · 0611 Anti-static and ESD tools · 0612 Bench vise · 0613 Toolbox
07	Lab Equipment	8	0700 Power supplies · 0701 Function generators · 0702 Frequency counters · 0703 Oscilloscopes · 0704 Multimeters · 0705 Ammeters · 0706 Voltmeters · 0707 Galvanometers
08	Energy	4	0800 Solar panels · 0801 Solar charge controllers · 0802 Inverters · 0803 Batteries & accessories
09	Toys & Hobbies	4	0900 Aircraft & accessories · 0901 Car & accessories · 0902 Drones · 0903 Robots
10	Communication	6	1000 USB-to-TTL · 1001 USB-to-UART · 1002 UART-to-USB · 1003 Radio Frequency · 1004 Laser · 1005 Infrared
11	Computers & Accessories	7	1100 Single Board Computers & Accessories · 1101 Networking devices · 1102 Data storage devices · 1103 Calculators · 1104 Presenters · 1105 Keyboards · 1106 Mice & accessories
12	Data & Signal Processing	4	1200 Audio amplifiers · 1201 Voice recognition boards · 1202 SD Card Readers · 1203 Decoder Modules
13	Security & Surveillance	2	1300 CCTV Cameras · 1301 Access control systems

Code	Category	Subs	Subcategories
14	Industrial Control & Automation	1	1400 Programmable Logic Controllers
16	Heating, Cooling & Air Quality	4	1600 Humidifiers · 1601 Dehumidifiers · 1604 Heat sinks · 1605 Heat shrink tubing & heat sleeves
17	Electronics Projects	1	1700 Automatic Irrigation Pump

A.1 Notes on applying this taxonomy

- **Depth is fixed at two levels** by the SKU scheme and by the mega-menu design (§9.3.8). Resist a third level; split a subcategory instead.
- **Reserve codes as you go.** Leaving gaps — using 00, 10, 20 for the first three subcategories — lets related items be inserted later without renumbering, and SKUs are immutable once assigned.
- **Empty subcategories are hidden from storefront navigation** (§6.4.2) but remain fully manageable in the console, so the tree can be built out ahead of stock arriving.
- **Name for the buyer, not the engineer.** "Passive devices" is correct and opaque; consider "Resistors, capacitors & inductors" for the STEM half of the audience, with the technical term as a tag.
- **Seed order matters.** Categories must be created before subcategories, and both before products, because SKU allocation reads the parent (Appendix F).

Validation Message Catalogue

Every validation message in the product, in one place, so that wording stays consistent and translation is a single pass. Messages marked *(source)* are carried over verbatim from the reference platform because they are already clear; the rest are new or rewritten. Keys are the i18n identifiers.

Key	Message	Applies to
<code>validation.required</code>	{Field} is required	Generic fallback
<code>auth.username.required</code>	Username is required <i>(source)</i>	Register, checkout, profile
<code>auth.email.required</code>	Email is required <i>(source)</i>	Register, checkout, contact
<code>auth.email.invalid</code>	Enter a valid email <i>(source)</i>	Everywhere an email is taken
<code>auth.phone.required</code>	Phone is required <i>(source)</i>	Register, checkout, profile
<code>auth.phone.numeric</code>	Phone must be a number <i>(source)</i>	Any phone field
<code>auth.phone.length</code>	Phone number must be 12 characters long <i>(source)</i>	Any phone field
<code>auth.phone.prefix</code>	Enter your number in the form 255XXXXXXXXX	Any phone field
<code>auth.password.required</code>	Password is required	Register, login, reset
<code>auth.password.short</code>	Password must be at least 10 characters	Register, reset, change
<code>auth.password.common</code>	That password is too common. Choose something less predictable.	Register, reset, change
<code>auth.password.mismatch</code>	Passwords do not match	Register, reset, change
<code>auth.terms.required</code>	Please accept the Terms & Conditions to continue	Register
<code>auth.credentials.invalid</code>	Email, phone or password is incorrect	Login — never says which
<code>auth.locked</code>	Too many attempts. Please try again in {minutes} minutes.	Login
<code>auth.token.invalid</code>	This link is no longer valid	Verify, reset, pay, invoice
<code>checkout.shipping.required</code>	Please select a shipping method	Checkout
<code>checkout.receiver.required</code>	Receiver's name is required <i>(source)</i>	Shipping details
<code>checkout.region.required</code>	Region is required <i>(source)</i>	Shipping details
<code>checkout.district.required</code>	District is required <i>(source)</i>	Shipping details
<code>checkout.street.required</code>	Street address is required <i>(source)</i>	Shipping details
<code>checkout.payment.required</code>	Please select a payment method <i>(source)</i>	Checkout, delegated payment
<code>checkout.paymentNumber.required</code>	Enter a payment number <i>(source)</i>	Lipa Namba
<code>checkout.paymentNumber.length</code>	Payment number must be 12 characters long <i>(source)</i>	Lipa Namba

Key	Message	Applies to
<code>cart.stock.insufficient</code>	Only {n} available — quantity adjusted	Cart, checkout
<code>cart.price.changed</code>	Price changed from {old} to {new}	Cart
<code>cart.item.unavailable</code>	This item is no longer available	Cart
<code>cart.empty</code>	Your cart is empty	Checkout guard
<code>product.subcategory.required</code>	Subcategory is required (<i>source</i>)	Admin product form
<code>product.name.required</code>	Product name is required (<i>source</i>)	Admin product form
<code>product.description.required</code>	Description is required (<i>source</i>)	Admin product form
<code>product.image.required</code>	At least one image is required	Admin product form
<code>product.price.positive</code>	Price must be greater than zero	Admin product form
<code>product.compareAt.higher</code>	Compare-at price must be higher than the price	Admin product form
<code>product.stock.negative</code>	Stock cannot be negative	Admin product form
<code>product.attribute.valueRequired</code>	Every variant needs a value for each attribute	Variant matrix
<code>product.attribute.duplicateSet</code>	Two variants have the same combination of values	Variant matrix
<code>product.attribute.duplicateName</code>	Attribute names must be unique within a product	Variant matrix
<code>category.name.required</code>	Name is required (<i>source</i>)	Category, subcategory
<code>category.sku.required</code>	SKU is required (<i>source</i>)	Category, subcategory
<code>category.sku.digits</code>	SKU should only contain digits (<i>source</i>)	Category, subcategory
<code>category.sku.length2</code>	SKU must be exactly 2 digits	Category
<code>category.sku.length4</code>	SKU must be exactly 4 digits	Subcategory
<code>category.sku.taken</code>	That SKU is already in use	Category, subcategory
<code>category.parent.required</code>	Category is required (<i>source</i>)	Subcategory
<code>shipping.name.required</code>	Shipping method is required (<i>source</i>)	Shipping form
<code>shipping.cost.required</code>	Shipping cost is required (<i>source</i>)	Shipping form
<code>shipping.cost.invalid</code>	Please enter a valid shipping cost (<i>source</i>)	Shipping form
<code>shipping.description.required</code>	Description is required (<i>source</i>)	Shipping form
<code>banner.title.required</code>	Banner title is required (<i>source</i>)	Banner builder
<code>banner.image.required</code>	An image is required (<i>source</i>)	Banner builder
<code>banner.contrast.low</code>	This text colour may be hard to read on that background ({ratio}:1)	Banner builder, advisory
<code>service.heading.required</code>	Service heading is required (<i>source</i>)	Service form
<code>service.heading.tooLong</code>	Heading should not exceed 15 words (<i>source</i>)	Service, partner, article
<code>service.content.required</code>	Service content is required (<i>source</i>)	Service form

Key	Message	Applies to
<code>partner.name.required</code>	Partner name is required <i>(source)</i>	Partner form
<code>partner.link.required</code>	Link to partner's website is required <i>(source)</i>	Partner form
<code>article.heading.required</code>	Article heading is required <i>(source)</i>	Article form
<code>article.description.required</code>	Article description is required <i>(source)</i>	Article form
<code>article.content.required</code>	Article content is required <i>(source)</i>	Article form
<code>invoice.client.required</code>	Client name is required <i>(source)</i>	Invoice builder
<code>invoice.lines.required</code>	Add at least one line item	Invoice builder
<code>campaign.name.required</code>	Campaign name is required <i>(source)</i>	SMS composer
<code>campaign.subject.required</code>	Subject is required <i>(source)</i>	Email composer
<code>campaign.audience.required</code>	Send To is required <i>(source)</i>	Both composers
<code>campaign.message.required</code>	Message is required <i>(source)</i>	SMS composer
<code>campaign.content.required</code>	Content is required <i>(source)</i>	Email composer
<code>campaign.schedule.required</code>	Date and time are required for sending the campaign later <i>(source)</i>	Both composers
<code>campaign.contacts.required</code>	Contacts are required for a specific audience <i>(source)</i>	Both composers
<code>campaign.phone.invalid</code>	Invalid phone number. Please enter a 12-digit number. <i>(source)</i>	SMS composer
<code>campaign.email.invalid</code>	Invalid email. Please enter a valid email. <i>(source)</i>	Email composer
<code>review.rating.required</code>	Please choose a rating	Review form
<code>review.body.required</code>	Please write a few words about this product	Review form
<code>comment.body.required</code>	Comment body is required <i>(source)</i>	Comment form
<code>contact.message.required</code>	Message is required <i>(source)</i>	Contact form
<code>upload.type.invalid</code>	Only JPEG, PNG, WebP and AVIF images are accepted	Any uploader
<code>upload.size.tooLarge</code>	Files must be 3 MB or smaller	Any uploader
<code>upload.failed</code>	Upload failed. Please try again.	Any uploader
<code>filter.price.range</code>	Minimum price must be less than maximum price	Catalogue filter

UI Copy Deck

Non-validation user-facing copy, grouped by where it appears. Three strings shipped by the reference platform contain errors and are corrected here: "No products to diplay" → "No products to display"; "Sarturday" → "Saturday"; "Oops!" → "Oops!".

Context	Copy
Header search placeholder	"Search products, SKUs, part numbers..."
Header buttons	"Search" · "Login Register"
Nav	"PRODUCTS" · "SHOP" · "SUPPORT" · "SERVICES"
Support dropdown	"CONTACT US" · "BLOG" · "FAQ"
Home sections	"OUR PRODUCTS" · "NEW ARRIVALS" · "HOT-SELLING PRODUCTS" · "OUR PARTNERS" · "VIEW MORE"
Product card	"ADD TO CART" · "CHOOSE OPTIONS" · "OUT OF STOCK" · "NOTIFY ME"
Catalogue controls	"Sort by:" · "Best Selling" · "New Arrivals" · "Price" · "Filters" · "Clear all" · "MIN PRICE" · "MAX PRICE" · "Apply" · "In stock only" · "On offer"
Catalogue empty	"No products found" · "No products match your filters" · "No products found in the selected category"
Product detail	"QUANTITY:" · "AVAILABLE STOCK:" · "ADD TO CART" · "ADD TO WISHLIST" · "BUY NOW" · "NOTIFY ME WHEN AVAILABLE" · "DESCRIPTION" · "SPECIFICATIONS" · "REVIEWS" · "RELATED PRODUCTS" · "Recently viewed"
Product detail lead	"Shop for products that have received wide attention of customers."
Reviews	"Verified purchase" · "Please sign in to write a review" · "Thank you — your review will appear once approved."
Cart panel	"SHOPPING CART" · "Your cart is empty" · "Browse products" · "CHECKOUT" · "View cart" · "Shipping calculated at checkout" · "Undo"
Wishlist panel	"WISHLIST" · "Your wishlist is empty" · "Move to cart"
Checkout	"Getting your order" · "Returning customer?" · "Click here to log in" · "Don't have a TechStar account?" · "Click here to create an account" · "CLIENT DETAILS" · "SHIPPING METHOD" · "ESTIMATED COST" · "SHIPPING DETAILS" · "PAYMENT METHOD" · "Someone else is paying for this order" · "Your Order" · "PLACE ORDER" · "Sub Total:" · "Shipping Cost:" · "TOTAL"
Payment	"Lipa Namba" · "Cash" · "Payment number" · "The mobile money number you will pay from." · "Pay exactly {amount}" · "MAKE PAYMENT"
Confirmation	"Order placed" · "What happens next" · "We have emailed a copy to {email}" · "View order" · "Print" · "Continue shopping" · "Payment confirmed"
Delegated payment	"Payment request" · "{name} has asked you to pay for this order." · "This order has already been paid." · "This payment link has expired."
Orders	"My orders" · "You have not placed any orders yet" · "Order again" · "Print invoice" · "Cancel order" · "Pay now" · "Are you sure you want to cancel this order?"
Order detail labels	"Client:" · "Email:" · "Phone:" · "Receiver's name:" · "Country:" · "City/Town:" · "Street Address:" · "Postal Code:" · "Quantity:" · "Anonymous" · "Not specified"
Account	"Account Settings" · "Account" · "Shipping Address" · "Change Password" · "Upload image" · "Details similar to my account" · "Update" · "Successfully updated user account" · "Successfully updated shipping details"
Auth modal	"LOGIN" · "REGISTER" · "EMAIL OR PHONE" · "PASSWORD" · "Forgot password?" · "Don't have an account?" · "Create your account here" · "Continue with Facebook" · "Continue with Google" · "SEND RESET LINK" · "If an account exists for that address, we have sent a reset link."

Context	Copy
Verification	"Please verify your email address to receive order updates." · "Send verification link" · "Verification link sent" · "Email verification successful" · "Verification failed"
Contact	"GET IN TOUCH WITH US" · "CONTACT DETAILS" · "Your name" · "Send Message" · "Send me a copy of this message" · "Thank you for getting in touch with TechStar Store" · "Submit another message"
FAQ	"Frequently Asked Questions" · "Can't find your answer?" · "Contact us" · "Chat on WhatsApp"
About	"Our Story" · "Vision" · "Mission" · "Core values" · "Read More" · "Read Less"
Blog	"Blog" · "Read more" · "Latest Articles" · "Write something here" · "Post comment" · "Please create an account or log in to comment" · "Thank you for your comment — it will appear once approved." · "No articles yet"
Services	"Other Services" · "ASK FOR DESIGN SERVICE"
Errors	"Oops!" · "Page not found" · "The page you're looking for doesn't exist." · "Go home" · "Something went wrong at our end." · "Try again" · "You appear to be offline." · "This is taking longer than usual..."
Admin — generic	"New {entity}" · "Search" · "Delete" · "Edit" · "Confirm" · "Cancel" · "Close" · "Are you sure you want to proceed with this action?" · "Loading..." · "No {entity} found"
Admin — products	"PRODUCTS" · "CREATE YOUR PRODUCT" · "EDIT YOUR PRODUCT" · "Select one" · "Enter Product Name" · "Enter price" · "Enter Stock" · "Add specifications to your product here. Skip this step if the product does not have any special specifications." · "Add Attributes" · "Attribute Name" · "Add New Values" · "Delete Table" · "Product Added Successfully" · "Product Edited Successfully"
Admin — categories	"PRODUCT CATEGORIES" · "PRODUCT SUBCATEGORIES" · "CREATE CATEGORY" · "EDIT CATEGORY" · "SKU (2 digits)" · "SKU (4 digits)" · "Created New Category" · "Category edited" · "Category Deleted"
Admin — orders	"ORDERS MADE" · "All Orders" · "Awaiting verification" · "Verify payment" · "Record manual payment" · "Issue refund" · "Mark shipped" · "Mark delivered" · "Print packing slip" · "No orders"
Admin — invoices	"INVOICES" · "Create Invoice" · "No Invoices" · "Please search here ..." · "Print Invoice" · "Share link" · "Mark paid" · "Void"
Admin — shipping	"SHIPPING METHODS" · "CREATE SHIPPING METHOD" · "EDIT SHIPPING METHOD" · "Check the box if this shipping method requires the client to provide shipping details." · "Check the box if this shipping method can receive mobile payments."
Admin — banners	"BANNERS" · "CREATE BANNER" · "EDIT BANNER" · "Enter Title" · "Enter Subtitle" · "Choose your background colour" · "Where does the banner redirect to" · "Enter Button text" · "Preview" · "PREVIEW MODAL"
Admin — campaigns	"CAMPAIGNS" · "CREATE A NEW SMS CAMPAIGN" · "CREATE A NEW EMAIL CAMPAIGN" · "Send To" · "Specific contacts" · "All contacts" · "Additional contacts" · "Include contacts" · "When to send:" · "Immediately" · "Later" · "Chars. Remaining ({n}) {used}/160" · "Campaign created successfully" · "No campaigns found."
Admin — searches	"SEARCHES" · "Search Query" · "Number Of Searches" · "No searches found" · "Zero-result searches"
Admin — settings	"GENERAL SITE SETTINGS" · "About Us Page Image" · "Privacy Policy Page Image" · "Terms and Conditions Page Image" · "PREVIEW" · "UPDATE" · "Image successfully updated"
Admin — currency	"CURRENCY FORMATTING" · "DISPLAY CURRENCY" · "FRACTION DIGITS" · "Update Currency Format" · "CURRENCY TYPE" · "MAXIMUM FRACTION DIGITS" · "Changing the display currency does not convert stored prices."
Admin — uploads	"Drag & drop your files or Browse" · "Drag & drop your image" · "Failed to delete image"
Admin — dashboard	"ANALYTICS" · "CUSTOMERS" · "BEST SELLING" · "Sales Analytics" · "Orders Analytics" · "Last 7 Days" · "Last 30 Days" · "Last 12 Months" · "No products to display"
Weekdays	"Sunday" · "Monday" · "Tuesday" · "Wednesday" · "Thursday" · "Friday" · "Saturday"

Environment Variables

Parsed once at boot through a Zod schema (\$20.7). A missing or malformed value stops the process with a message naming the variable. **R** = required in production.

Variable	R	Example	Purpose
Core			
NODE_ENV	✓	production	Runtime mode
APP_NAME	✓	TechStar Store	Used in emails and page titles
APP_URL	✓	https://techstar.co.tz	Canonical origin; used to build absolute links
API_PORT	✓	4000	HTTP listen port
TZ	✓	UTC	Process timezone; display conversion is done in code
LOG_LEVEL	—	info	pino level
Database and cache			
DATABASE_URL	✓	mysql://user:pass@host:3306/techstar	Prisma connection string
DATABASE_POOL_SIZE	—	10	Connection pool
REDIS_URL	✓	redis://127.0.0.1:6379	Cache, queues, rate limits, socket adapter
Security			
JWT_ACCESS_SECRET	✓	32+ random bytes, base64	Access token signing
JWT_ACCESS_TTL	—	15m	Access token lifetime
REFRESH_TOKEN_TTL_DAYS	—	30	Refresh cookie lifetime
COOKIE_DOMAIN	✓	.techstar.co.tz	Refresh cookie scope
ENCRYPTION_KEY	✓	32 random bytes, base64	Encrypts provider credentials stored in settings
CORS_ORIGINS	—	https://techstar.co.tz	Only when a separate front-end origin exists
OAuth			
GOOGLE_CLIENT_ID / _SECRET	—		Google sign-in; feature disabled when absent
FACEBOOK_APP_ID / _SECRET	—		Facebook sign-in; feature disabled when absent
Storage			
STORAGE_DRIVER	✓	local	local · s3 · cloudinary
MEDIA_ROOT	—	/var/lib/techstar/media	local driver only
MEDIA_PUBLIC_URL	✓	https://techstar.co.tz/media	Base URL for served derivatives
S3_*	—	endpoint, region, bucket, key, secret	s3 driver only
Email			
SMTP_HOST _PORT _USER _PASSWORD _SECURE	✓		Nodemailer transport

Variable	R	Example	Purpose
MAIL_FROM_NAME / MAIL_FROM_ADDRESS	✓	TechStar Store / orders@...	Envelope identity
MAIL_REPLY_TO	—	support@...	
SMS			
SMS_PROVIDER	✓	beem	beem · nextsms · console
SMS_API_KEY / SMS_API_SECRET	✓		Provider credentials
SMS_SENDER_ID	✓	TECHSTAR	Registered with operators in advance
Payments			
PAYMENT_PROVIDER	—	manual	manual · selcom · clickpesa · azampay
PAYMENT_API_KEY / _SECRET / _BASE_URL	—		Aggregator credentials
PAYMENT_WEBHOOK_SECRET	—		Signature verification
RESERVATION_HOURS	—	48	Unpaid order reservation window
Observability and limits			
SENTRY_DSN	—		Error tracking; disabled when absent
OTEL_EXPORTER_OTLP_ENDPOINT	—		Tracing
RATE_LIMIT_ENABLED	—	true	Off only in test
MAX_UPLOAD_MB	—	3	Per-file cap
Front end (Vite, exposed to the browser — never put a secret here)			
VITE_API_URL	✓	/api/v1	Same-origin by default
VITE_SOCKET_URL	✓	/	Socket.IO endpoint
VITE_GA_MEASUREMENT_ID	—	G-XXXXXXXXXX	Analytics; omitted disables the tag entirely
VITE_WHATSAPP_NUMBER	✓	255XXXXXXXXX	Floating chat button

Complete MySQL DDL

The executable schema for all **54 tables**, ordered so that no foreign key references a table that does not yet exist. It runs top to bottom against a clean MySQL 8.0 server. This listing is generated directly from `docs/assets/schema.sql`, which is the file engineers run — the document and the schema cannot drift apart.

Tables: `media_files` · `users` · `user_addresses` · `oauth_accounts` · `sessions` · `email_verification_tokens` · `password_reset_tokens` · `roles` · `permissions` · `role_permissions` · `user_roles` · `regions` · `districts` · `categories` · `subcategories` · `products` · `product_slug_history` · `product_attributes` · `product_variants` · `variant_attribute_values` · `product_images` · `tags` · `product_tags` · `discounts` · `carts` · `cart_items` · `wishlists` · `wishlist_items` · `shipping_methods` · `orders` · `order_items` · `order_shipping_addresses` · `order_events` · `payments` · `payment_events` · `stock_movements` · `invoices` · `invoice_items` · `banners` · `services` · `partners` · `articles` · `comments` · `reviews` · `faqs` · `site_settings` · `contacts` · `campaigns` · `campaign_recipients` · `search_logs` · `contact_messages` · `notify_requests` · `audit_logs` · `ft_stopwords`

```
-- =====
-- TechStar Store – complete MySQL 8.0 schema
-- Specification: Part Nine. Runs top to bottom against a clean schema.
-- Engine InnoDB · utf8mb4 · all timestamps UTC · money in minor units
-- =====

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 1;

CREATE DATABASE IF NOT EXISTS techstar
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE techstar;

-- -----
-- 1. MEDIA (referenced by almost everything, so created first)
-- -----

CREATE TABLE media_files (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  public_id   CHAR(26)        NOT NULL,
  storage_driver VARCHAR(20)  NOT NULL DEFAULT 'local',
  storage_key  VARCHAR(512)   NOT NULL,
  original_filename VARCHAR(255) NOT NULL DEFAULT '',
  mime_type   VARCHAR(80)     NOT NULL,
  byte_size   INT UNSIGNED    NOT NULL,
  width       INT UNSIGNED    NULL,
  height      INT UNSIGNED    NULL,
  checksum_sha256 CHAR(64)    NOT NULL,
  derivatives  JSON           NOT NULL,
  alt_text    VARCHAR(255)    NULL,
  uploaded_by BIGINT UNSIGNED NULL,
  reference_count INT UNSIGNED NOT NULL DEFAULT 0,
  created_at   DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at   DATETIME(3)    NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_media_public_id (public_id),
  KEY idx_media_checksum (checksum_sha256),
  KEY idx_media_refcount (reference_count, created_at),
  CONSTRAINT chk_media_driver CHECK (storage_driver IN ('local','s3','cloudinary'))
) ENGINE=InnoDB;

-- -----
-- 2. IDENTITY AND ACCESS
-- -----
```

```

CREATE TABLE users (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  public_id   CHAR(26)        NOT NULL,
  email       VARCHAR(255)    NULL,
  email_verified_at DATETIME(3) NULL,
  phone       VARCHAR(15)     NULL,
  phone_verified_at DATETIME(3) NULL,
  username    VARCHAR(80)     NOT NULL,
  password_hash VARCHAR(255)  NULL,
  account_type VARCHAR(16)    NOT NULL DEFAULT 'customer',
  status      VARCHAR(16)    NOT NULL DEFAULT 'active',
  profile_image_id BIGINT UNSIGNED NULL,
  marketing_opt_in TINYINT(1)   NOT NULL DEFAULT 0,
  last_login_at DATETIME(3)    NULL,
  failed_login_count SMALLINT UNSIGNED NOT NULL DEFAULT 0,
  locked_until DATETIME(3)     NULL,
  created_at   DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at   DATETIME(3)     NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_users_public_id (public_id),
  UNIQUE KEY uq_users_email (email),
  UNIQUE KEY uq_users_phone (phone),
  KEY idx_users_status_created (status, created_at),
  KEY idx_users_account_type (account_type),
  CONSTRAINT fk_users_avatar FOREIGN KEY (profile_image_id) REFERENCES media_files(id) ON DELETE SET NULL,
  CONSTRAINT chk_users_reachable CHECK (email IS NOT NULL OR phone IS NOT NULL),
  CONSTRAINT chk_users_type CHECK (account_type IN ('customer','staff')),
  CONSTRAINT chk_users_status CHECK (status IN ('active','suspended','closed'))
) ENGINE=InnoDB;

ALTER TABLE media_files
  ADD CONSTRAINT fk_media_uploader FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE SET NULL;

CREATE TABLE user_addresses (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id     BIGINT UNSIGNED NOT NULL,
  label       VARCHAR(40)     NULL,
  receiver_name VARCHAR(120)  NOT NULL,
  email       VARCHAR(255)    NULL,
  phone       VARCHAR(15)     NOT NULL,
  country     VARCHAR(64)     NOT NULL DEFAULT 'Tanzania',
  region      VARCHAR(64)     NOT NULL,
  district    VARCHAR(64)     NOT NULL,
  street_address VARCHAR(255) NOT NULL,
  postal_code VARCHAR(20)     NULL,
  is_default  TINYINT(1)     NOT NULL DEFAULT 0,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at  DATETIME(3)     NULL,
  PRIMARY KEY (id),
  KEY idx_addresses_user (user_id, deleted_at),
  CONSTRAINT fk_addresses_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE oauth_accounts (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id     BIGINT UNSIGNED NOT NULL,
  provider    VARCHAR(20)     NOT NULL,
  provider_user_id VARCHAR(191) NOT NULL,
  email       VARCHAR(255)    NULL,
  raw_profile JSON             NULL,
  linked_at   DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_oauth_provider_user (provider, provider_user_id),

```

```

    KEY idx_oauth_user (user_id),
    CONSTRAINT fk_oauth_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT chk_oauth_provider CHECK (provider IN ('google','facebook'))
) ENGINE=InnoDB;

CREATE TABLE sessions (
    id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id           BIGINT UNSIGNED NOT NULL,
    refresh_token_hash CHAR(64)        NOT NULL,
    user_agent        VARCHAR(255)     NULL,
    ip_address        VARBINARY(16)    NULL,
    expires_at        DATETIME(3)      NOT NULL,
    revoked_at        DATETIME(3)      NULL,
    replaced_by_id    BIGINT UNSIGNED NULL,
    created_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_sessions_token (refresh_token_hash),
    KEY idx_sessions_user (user_id, revoked_at),
    KEY idx_sessions_expiry (expires_at),
    CONSTRAINT fk_sessions_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE email_verification_tokens (
    id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id           BIGINT UNSIGNED NOT NULL,
    token_hash        CHAR(64)        NOT NULL,
    expires_at        DATETIME(3)      NOT NULL,
    used_at           DATETIME(3)      NULL,
    requested_ip      VARBINARY(16)    NULL,
    created_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_evt_token (token_hash),
    KEY idx_evt_user (user_id),
    CONSTRAINT fk_evt_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE password_reset_tokens (
    id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id           BIGINT UNSIGNED NOT NULL,
    token_hash        CHAR(64)        NOT NULL,
    expires_at        DATETIME(3)      NOT NULL,
    used_at           DATETIME(3)      NULL,
    requested_ip      VARBINARY(16)    NULL,
    created_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_prt_token (token_hash),
    KEY idx_prt_user (user_id),
    CONSTRAINT fk_prt_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE roles (
    id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    `key`             VARCHAR(40)      NOT NULL,
    name              VARCHAR(80)      NOT NULL,
    description        VARCHAR(255)    NULL,
    is_system         TINYINT(1)       NOT NULL DEFAULT 0,
    created_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_roles_key (`key`)
) ENGINE=InnoDB;

CREATE TABLE permissions (
    id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,

```

```

`key`          VARCHAR(60)      NOT NULL,
`group`        VARCHAR(40)      NOT NULL,
description    VARCHAR(255)     NULL,
created_at     DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at     DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
UNIQUE KEY uq_permissions_key (`key`)
) ENGINE=InnoDB;

CREATE TABLE role_permissions (
  role_id      BIGINT UNSIGNED NOT NULL,
  permission_id BIGINT UNSIGNED NOT NULL,
  PRIMARY KEY (role_id, permission_id),
  KEY idx_rp_permission (permission_id),
  CONSTRAINT fk_rp_role FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE,
  CONSTRAINT fk_rp_permission FOREIGN KEY (permission_id) REFERENCES permissions(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE user_roles (
  user_id      BIGINT UNSIGNED NOT NULL,
  role_id      BIGINT UNSIGNED NOT NULL,
  granted_by    BIGINT UNSIGNED NULL,
  granted_at    DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (user_id, role_id),
  KEY idx_ur_role (role_id),
  CONSTRAINT fk_ur_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_ur_role FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE,
  CONSTRAINT fk_ur_granter FOREIGN KEY (granted_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- -----
-- 3. LOCATIONS
-- -----

CREATE TABLE regions (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name        VARCHAR(64)     NOT NULL,
  code        VARCHAR(10)     NULL,
  position    INT             NOT NULL DEFAULT 0,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_regions_name (name)
) ENGINE=InnoDB;

CREATE TABLE districts (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  region_id   BIGINT UNSIGNED NOT NULL,
  name        VARCHAR(64)     NOT NULL,
  position    INT             NOT NULL DEFAULT 0,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_districts_region_name (region_id, name),
  CONSTRAINT fk_districts_region FOREIGN KEY (region_id) REFERENCES regions(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- -----
-- 4. CATALOGUE
-- -----

CREATE TABLE categories (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name        VARCHAR(120)    NOT NULL,
  slug        VARCHAR(96)     NOT NULL,
  sku_code    CHAR(2)         NOT NULL,
  description  VARCHAR(500)    NULL,
  image_id    BIGINT UNSIGNED NULL,
  position    INT             NOT NULL DEFAULT 0,

```

```

subcategory_count INT UNSIGNED NOT NULL DEFAULT 0,
product_count INT UNSIGNED NOT NULL DEFAULT 0,
is_active TINYINT(1) NOT NULL DEFAULT 1,
created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
deleted_at DATETIME(3) NULL,
PRIMARY KEY (id),
UNIQUE KEY uq_categories_slug (slug),
UNIQUE KEY uq_categories_sku (sku_code),
KEY idx_categories_position (position, id),
CONSTRAINT fk_categories_image FOREIGN KEY (image_id) REFERENCES media_files(id) ON DELETE SET NULL,
CONSTRAINT chk_categories_sku CHECK (sku_code REGEXP '^[0-9]{2}$')
) ENGINE=InnoDB;

CREATE TABLE subcategories (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  category_id BIGINT UNSIGNED NOT NULL,
  name VARCHAR(140) NOT NULL,
  slug VARCHAR(96) NOT NULL,
  sku_code CHAR(4) NOT NULL,
  description VARCHAR(500) NULL,
  image_id BIGINT UNSIGNED NULL,
  position INT NOT NULL DEFAULT 0,
  product_count INT UNSIGNED NOT NULL DEFAULT 0,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at DATETIME(3) NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_subcategories_slug (slug),
  UNIQUE KEY uq_subcategories_sku (sku_code),
  KEY idx_subcategories_category (category_id, position),
  CONSTRAINT fk_subcategories_category FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE RESTRICT,
  CONSTRAINT fk_subcategories_image FOREIGN KEY (image_id) REFERENCES media_files(id) ON DELETE SET
  NULL,
  CONSTRAINT chk_subcategories_sku CHECK (sku_code REGEXP '^[0-9]{4}$')
) ENGINE=InnoDB;

CREATE TABLE products (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  subcategory_id BIGINT UNSIGNED NOT NULL,
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(96) NOT NULL,
  sku_prefix CHAR(7) NOT NULL,
  short_description VARCHAR(500) NULL,
  description_html MEDIUMTEXT NOT NULL,
  description_text MEDIUMTEXT NOT NULL,
  brand VARCHAR(120) NULL,
  manufacturer_part_number VARCHAR(80) NULL,
  display_image_id BIGINT UNSIGNED NULL,
  status VARCHAR(16) NOT NULL DEFAULT 'draft',
  total_stock INT NOT NULL DEFAULT 0,
  min_price_amount BIGINT NOT NULL DEFAULT 0,
  max_price_amount BIGINT NOT NULL DEFAULT 0,
  order_count INT UNSIGNED NOT NULL DEFAULT 0,
  view_count INT UNSIGNED NOT NULL DEFAULT 0,
  review_count INT UNSIGNED NOT NULL DEFAULT 0,
  average_rating DECIMAL(3,2) NOT NULL DEFAULT 0.00,
  is_featured TINYINT(1) NOT NULL DEFAULT 0,
  meta_title VARCHAR(70) NULL,
  meta_description VARCHAR(160) NULL,
  published_at DATETIME(3) NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
  CURRENT_TIMESTAMP(3),
  deleted_at DATETIME(3) NULL,
  PRIMARY KEY (id),

```



```

    UNIQUE KEY uq_products_slug (slug),
    UNIQUE KEY uq_products_sku_prefix (sku_prefix),
    KEY idx_products_subcat_status (subcategory_id, status, published_at),
    KEY idx_products_status_published (status, published_at),
    KEY idx_products_order_count (status, order_count),
    KEY idx_products_min_price (status, min_price_amount),
    KEY idx_products_mpn (manufacturer_part_number),
    FULLTEXT KEY ft_products_search (name, description_text, brand, manufacturer_part_number),
    CONSTRAINT fk_products_subcategory FOREIGN KEY (subcategory_id) REFERENCES subcategories(id) ON DELETE
    RESTRICT,
    CONSTRAINT fk_products_image FOREIGN KEY (display_image_id) REFERENCES media_files(id) ON DELETE SET
    NULL,
    CONSTRAINT chk_products_status CHECK (status IN ('draft','active','archived'))
) ENGINE=InnoDB;

CREATE TABLE product_slug_history (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    product_id BIGINT UNSIGNED NOT NULL,
    slug VARCHAR(96) NOT NULL,
    retired_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_slug_history_slug (slug),
    KEY idx_slug_history_product (product_id),
    CONSTRAINT fk_slug_history_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE product_attributes (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    product_id BIGINT UNSIGNED NOT NULL,
    name VARCHAR(80) NOT NULL,
    unit VARCHAR(20) NULL,
    position INT NOT NULL DEFAULT 0,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    UNIQUE KEY uq_attr_product_name (product_id, name),
    CONSTRAINT fk_attr_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE product_variants (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    product_id BIGINT UNSIGNED NOT NULL,
    sku VARCHAR(20) NOT NULL,
    attribute_hash CHAR(40) NOT NULL,
    price_amount BIGINT NOT NULL,
    compare_at_amount BIGINT NULL,
    cost_amount BIGINT NULL,
    currency CHAR(3) NOT NULL DEFAULT 'TZS',
    stock_quantity INT NOT NULL DEFAULT 0,
    reserved_quantity INT NOT NULL DEFAULT 0,
    low_stock_threshold INT NOT NULL DEFAULT 5,
    weight_grams INT NULL,
    barcode VARCHAR(64) NULL,
    image_id BIGINT UNSIGNED NULL,
    position INT NOT NULL DEFAULT 0,
    is_active TINYINT(1) NOT NULL DEFAULT 1,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    deleted_at DATETIME(3) NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_variants_sku (sku),
    UNIQUE KEY uq_variants_product_attrs (product_id, attribute_hash),
    KEY idx_variants_product_active (product_id, is_active, position),
    KEY idx_variants_stock (stock_quantity),
    KEY idx_variants_price (price_amount),
    CONSTRAINT fk_variants_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
    CONSTRAINT fk_variants_image FOREIGN KEY (image_id) REFERENCES media_files(id) ON DELETE SET NULL,

```

```

CONSTRAINT chk_variants_price CHECK (price_amount > 0),
CONSTRAINT chk_variants_stock CHECK (stock_quantity >= 0),
CONSTRAINT chk_variants_reserved CHECK (reserved_quantity >= 0)
) ENGINE=InnoDB;

CREATE TABLE variant_attribute_values (
  variant_id BIGINT UNSIGNED NOT NULL,
  product_attribute_id BIGINT UNSIGNED NOT NULL,
  value VARCHAR(120) NOT NULL,
  PRIMARY KEY (variant_id, product_attribute_id),
  KEY idx_vav_attr_value (product_attribute_id, value),
  CONSTRAINT fk_vav_variant FOREIGN KEY (variant_id) REFERENCES product_variants(id) ON DELETE
  CASCADE,
  CONSTRAINT fk_vav_attr FOREIGN KEY (product_attribute_id) REFERENCES product_attributes(id) ON DELETE
  CASCADE
) ENGINE=InnoDB;

CREATE TABLE product_images (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  product_id BIGINT UNSIGNED NOT NULL,
  media_file_id BIGINT UNSIGNED NOT NULL,
  variant_id BIGINT UNSIGNED NULL,
  alt_text VARCHAR(255) NULL,
  position INT NOT NULL DEFAULT 0,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_product_images (product_id, media_file_id),
  KEY idx_product_images_order (product_id, position),
  CONSTRAINT fk_pi_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
  CONSTRAINT fk_pi_media FOREIGN KEY (media_file_id) REFERENCES media_files(id) ON DELETE RESTRICT,
  CONSTRAINT fk_pi_variant FOREIGN KEY (variant_id) REFERENCES product_variants(id) ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE tags (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name VARCHAR(60) NOT NULL,
  slug VARCHAR(60) NOT NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_tags_slug (slug)
) ENGINE=InnoDB;

CREATE TABLE product_tags (
  product_id BIGINT UNSIGNED NOT NULL,
  tag_id BIGINT UNSIGNED NOT NULL,
  PRIMARY KEY (product_id, tag_id),
  KEY idx_pt_tag (tag_id),
  CONSTRAINT fk_pt_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
  CONSTRAINT fk_pt_tag FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE discounts (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  scope VARCHAR(20) NOT NULL,
  scope_id BIGINT UNSIGNED NOT NULL,
  type VARCHAR(16) NOT NULL,
  value INT NOT NULL,
  starts_at DATETIME(3) NOT NULL,
  ends_at DATETIME(3) NOT NULL,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  created_by BIGINT UNSIGNED NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at DATETIME(3) NULL,
  PRIMARY KEY (id),

```

```

KEY idx_discounts_scope (scope, scope_id, is_active, starts_at, ends_at),
CONSTRAINT fk_discounts_creator FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL,
CONSTRAINT chk_discounts_scope CHECK (scope IN ('variant', 'product', 'subcategory', 'category')),
CONSTRAINT chk_discounts_type CHECK (type IN ('percentage', 'fixed')),
CONSTRAINT chk_discounts_value CHECK (value > 0),
CONSTRAINT chk_discounts_window CHECK (ends_at > starts_at)
) ENGINE=InnoDB;

-- -----
-- 5. SHOPPING
-- -----

CREATE TABLE carts (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id BIGINT UNSIGNED NULL,
  guest_token CHAR(32) NULL,
  currency CHAR(3) NOT NULL DEFAULT 'TZS',
  last_activity_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  converted_order_id BIGINT UNSIGNED NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_carts_user (user_id),
  UNIQUE KEY uq_carts_guest (guest_token),
  KEY idx_carts_activity (last_activity_at),
  CONSTRAINT fk_carts_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT chk_carts_owner CHECK (user_id IS NOT NULL OR guest_token IS NOT NULL)
) ENGINE=InnoDB;

CREATE TABLE cart_items (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  cart_id BIGINT UNSIGNED NOT NULL,
  product_variant_id BIGINT UNSIGNED NOT NULL,
  quantity INT NOT NULL,
  unit_price_amount BIGINT NOT NULL,
  added_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_cart_items (cart_id, product_variant_id),
  KEY idx_cart_items_variant (product_variant_id),
  CONSTRAINT fk_ci_cart FOREIGN KEY (cart_id) REFERENCES carts(id) ON DELETE CASCADE,
  CONSTRAINT fk_ci_variant FOREIGN KEY (product_variant_id) REFERENCES product_variants(id) ON DELETE
  RESTRICT,
  CONSTRAINT chk_ci_quantity CHECK (quantity > 0 AND quantity <= 999)
) ENGINE=InnoDB;

CREATE TABLE wishlists (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id BIGINT UNSIGNED NULL,
  guest_token CHAR(32) NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_wishlists_user (user_id),
  UNIQUE KEY uq_wishlists_guest (guest_token),
  CONSTRAINT fk_wishlists_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE wishlist_items (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  wishlist_id BIGINT UNSIGNED NOT NULL,
  product_id BIGINT UNSIGNED NOT NULL,
  product_variant_id BIGINT UNSIGNED NULL,
  added_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_wishlist_items (wishlist_id, product_id, product_variant_id),
  CONSTRAINT fk_wi_wishlist FOREIGN KEY (wishlist_id) REFERENCES wishlists(id) ON DELETE

```

```

CASCADE,
    CONSTRAINT fk_wi_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE
CASCADE,
    CONSTRAINT fk_wi_variant FOREIGN KEY (product_variant_id) REFERENCES product_variants(id) ON DELETE SET
NULL
) ENGINE=InnoDB;

```

-- 6. ORDERS AND PAYMENTS

```

CREATE TABLE shipping_methods (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(80) NOT NULL,
    slug VARCHAR(60) NOT NULL,
    description VARCHAR(500) NOT NULL,
    cost_amount BIGINT NOT NULL DEFAULT 0,
    requires_shipping_address TINYINT(1) NOT NULL DEFAULT 1,
    accepts_mobile_payment TINYINT(1) NOT NULL DEFAULT 1,
    is_cash_on_delivery TINYINT(1) NOT NULL DEFAULT 0,
    free_over_amount BIGINT NULL,
    estimated_days_min SMALLINT NULL,
    estimated_days_max SMALLINT NULL,
    available_regions JSON NULL,
    position INT NOT NULL DEFAULT 0,
    is_active TINYINT(1) NOT NULL DEFAULT 1,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
    deleted_at DATETIME(3) NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_shipping_slug (slug),
    CONSTRAINT chk_shipping_cost CHECK (cost_amount >= 0)
) ENGINE=InnoDB;

```

```

CREATE TABLE orders (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    order_number VARCHAR(24) NOT NULL,
    user_id BIGINT UNSIGNED NULL,
    status VARCHAR(24) NOT NULL DEFAULT 'awaiting_payment',
    payment_status VARCHAR(24) NOT NULL DEFAULT 'unpaid',
    fulfilment_status VARCHAR(24) NOT NULL DEFAULT 'unfulfilled',
    customer_name VARCHAR(120) NOT NULL,
    customer_email VARCHAR(255) NULL,
    customer_phone VARCHAR(15) NOT NULL,
    shipping_method_id BIGINT UNSIGNED NULL,
    shipping_method_name VARCHAR(80) NOT NULL,
    subtotal_amount BIGINT NOT NULL,
    discount_amount BIGINT NOT NULL DEFAULT 0,
    shipping_amount BIGINT NOT NULL DEFAULT 0,
    tax_amount BIGINT NOT NULL DEFAULT 0,
    total_amount BIGINT NOT NULL,
    paid_amount BIGINT NOT NULL DEFAULT 0,
    currency CHAR(3) NOT NULL DEFAULT 'TZS',
    payment_method VARCHAR(24) NULL,
    payment_number VARCHAR(15) NULL,
    delegated_payer_email VARCHAR(255) NULL,
    payment_token_hash CHAR(64) NULL,
    payment_token_expires_at DATETIME(3) NULL,
    confirmation_token_hash CHAR(64) NULL,
    idempotency_key CHAR(36) NULL,
    customer_note VARCHAR(500) NULL,
    internal_note TEXT NULL,
    cancel_reason VARCHAR(255) NULL,
    cancelled_at DATETIME(3) NULL,
    placed_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    paid_at DATETIME(3) NULL,
    shipped_at DATETIME(3) NULL,

```

```

delivered_at          DATETIME(3)      NULL,
reservation_expires_at DATETIME(3)      NULL,
source                VARCHAR(20)      NOT NULL DEFAULT 'web',
created_at            DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at            DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
UNIQUE KEY uq_orders_number (order_number),
UNIQUE KEY uq_orders_idempotency (idempotency_key),
UNIQUE KEY uq_orders_payment_token (payment_token_hash),
KEY idx_orders_user_placed (user_id, placed_at),
KEY idx_orders_status_placed (status, placed_at),
KEY idx_orders_payment_status (payment_status, placed_at),
KEY idx_orders_phone (customer_phone),
KEY idx_orders_paid_at (payment_status, paid_at),
KEY idx_orders_reservation (status, reservation_expires_at),
CONSTRAINT fk_orders_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET
NULL,
CONSTRAINT fk_orders_shipping FOREIGN KEY (shipping_method_id) REFERENCES shipping_methods(id) ON DELETE SET
NULL,
CONSTRAINT chk_orders_status CHECK (status IN
('awaiting_payment', 'confirmed', 'processing', 'shipped', 'delivered', 'completed', 'cancelled', 'expired', 'refunded'
)),
CONSTRAINT chk_orders_payment_status CHECK (payment_status IN
('unpaid', 'pending_verification', 'paid', 'partially_refunded', 'refunded', 'failed')),
CONSTRAINT chk_orders_fulfilment CHECK (fulfilment_status IN
('unfulfilled', 'processing', 'shipped', 'delivered', 'returned')),
CONSTRAINT chk_orders_payment_method CHECK (payment_method IS NULL OR payment_method IN
('lipa_namba', 'cash', 'aggregator')),
CONSTRAINT chk_orders_source CHECK (source IN ('web', 'admin', 'phone'))
) ENGINE=InnoDB;

CREATE TABLE order_items (
id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
order_id BIGINT UNSIGNED NOT NULL,
product_id BIGINT UNSIGNED NULL,
product_variant_id BIGINT UNSIGNED NULL,
product_name VARCHAR(255) NOT NULL,
product_slug VARCHAR(96) NULL,
variant_sku VARCHAR(20) NOT NULL,
variant_attributes JSON NULL,
image_url VARCHAR(512) NULL,
unit_price_amount BIGINT NOT NULL,
quantity INT NOT NULL,
discount_amount BIGINT NOT NULL DEFAULT 0,
line_total_amount BIGINT NOT NULL,
created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
KEY idx_order_items_order (order_id),
KEY idx_order_items_product (product_id),
KEY idx_order_items_variant (product_variant_id),
CONSTRAINT fk_oi_order FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
CONSTRAINT fk_oi_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE SET
NULL,
CONSTRAINT fk_oi_variant FOREIGN KEY (product_variant_id) REFERENCES product_variants(id) ON DELETE SET
NULL,
CONSTRAINT chk_oi_quantity CHECK (quantity > 0)
) ENGINE=InnoDB;

CREATE TABLE order_shipping_addresses (
id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
order_id BIGINT UNSIGNED NOT NULL,
receiver_name VARCHAR(120) NOT NULL,
email VARCHAR(255) NULL,
phone VARCHAR(15) NOT NULL,

```

```

country          VARCHAR(64)      NOT NULL DEFAULT 'Tanzania',
region           VARCHAR(64)      NOT NULL,
district         VARCHAR(64)      NOT NULL,
street_address   VARCHAR(255)     NOT NULL,
postal_code      VARCHAR(20)      NULL,
created_at       DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
UNIQUE KEY uq_osa_order (order_id),
CONSTRAINT fk_osa_order FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE order_events (
  id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  order_id        BIGINT UNSIGNED NOT NULL,
  event_type      VARCHAR(40)      NOT NULL,
  from_value      VARCHAR(40)      NULL,
  to_value        VARCHAR(40)      NULL,
  actor_type      VARCHAR(16)      NOT NULL DEFAULT 'system',
  actor_user_id   BIGINT UNSIGNED NULL,
  message         VARCHAR(500)     NULL,
  metadata        JSON             NULL,
  created_at      DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
KEY idx_order_events_order (order_id, created_at),
CONSTRAINT fk_oe_order FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
CONSTRAINT fk_oe_actor FOREIGN KEY (actor_user_id) REFERENCES users(id) ON DELETE SET NULL,
CONSTRAINT chk_oe_actor_type CHECK (actor_type IN ('customer', 'staff', 'system', 'webhook'))
) ENGINE=InnoDB;

CREATE TABLE payments (
  id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  order_id        BIGINT UNSIGNED NOT NULL,
  method          VARCHAR(24)      NOT NULL,
  provider        VARCHAR(40)      NULL,
  amount          BIGINT           NOT NULL,
  currency        CHAR(3)          NOT NULL DEFAULT 'TZS',
  status          VARCHAR(24)      NOT NULL DEFAULT 'initiated',
  payer_phone     VARCHAR(15)      NULL,
  payer_name      VARCHAR(120)     NULL,
  provider_reference VARCHAR(120)  NULL,
  merchant_reference VARCHAR(64)   NOT NULL,
  verified_by     BIGINT UNSIGNED NULL,
  verified_at     DATETIME(3)      NULL,
  failure_reason  VARCHAR(255)     NULL,
  raw_payload     JSON             NULL,
  created_at      DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at      DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
UNIQUE KEY uq_payments_provider_ref (provider, provider_reference),
KEY idx_payments_order (order_id),
KEY idx_payments_status (status, created_at),
KEY idx_payments_payer (payer_phone),
CONSTRAINT fk_payments_order FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT,
CONSTRAINT fk_payments_verifier FOREIGN KEY (verified_by) REFERENCES users(id) ON DELETE SET NULL,
CONSTRAINT chk_payments_status CHECK (status IN
  ('initiated', 'pending', 'succeeded', 'failed', 'cancelled', 'refunded')),
CONSTRAINT chk_payments_amount CHECK (amount > 0)
) ENGINE=InnoDB;

CREATE TABLE payment_events (
  id              BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  payment_id      BIGINT UNSIGNED NULL,
  provider        VARCHAR(40)      NOT NULL,
  event_type      VARCHAR(60)      NOT NULL,
  external_event_id VARCHAR(120)   NOT NULL,
  signature_valid TINYINT(1)       NOT NULL DEFAULT 0,
  payload         JSON             NOT NULL,

```

```

processed_at      DATETIME(3)      NULL,
processing_error   VARCHAR(500)     NULL,
created_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
UNIQUE KEY uq_payment_events_external (provider, external_event_id),
KEY idx_payment_events_unprocessed (processed_at, created_at),
CONSTRAINT fk_pe_payment FOREIGN KEY (payment_id) REFERENCES payments(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE stock_movements (
  id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  product_variant_id BIGINT UNSIGNED NOT NULL,
  delta             INT             NOT NULL,
  balance_after     INT             NOT NULL,
  reason            VARCHAR(32)     NOT NULL,
  reference_type     VARCHAR(32)     NULL,
  reference_id       BIGINT UNSIGNED NULL,
  actor_user_id      BIGINT UNSIGNED NULL,
  note              VARCHAR(255)     NULL,
  created_at        DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
PRIMARY KEY (id),
KEY idx_stock_variant (product_variant_id, created_at),
KEY idx_stock_reason (reason, created_at),
CONSTRAINT fk_sm_variant FOREIGN KEY (product_variant_id) REFERENCES product_variants(id) ON DELETE
RESTRICT,
CONSTRAINT fk_sm_actor FOREIGN KEY (actor_user_id) REFERENCES users(id) ON DELETE SET
NULL,
CONSTRAINT chk_sm_reason CHECK (reason IN
  ('purchase', 'sale', 'reservation', 'release', 'adjustment', 'return', 'damage', 'stocktake'))
) ENGINE=InnoDB;

CREATE TABLE invoices (
  id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  invoice_number     VARCHAR(24)     NOT NULL,
  order_id           BIGINT UNSIGNED NULL,
  client_name        VARCHAR(120)    NOT NULL,
  client_email       VARCHAR(255)    NOT NULL,
  client_phone       VARCHAR(15)     NULL,
  client_address     VARCHAR(500)    NULL,
  shipping_method_name VARCHAR(80)    NULL,
  subtotal_amount    BIGINT          NOT NULL DEFAULT 0,
  shipping_amount    BIGINT          NOT NULL DEFAULT 0,
  tax_amount         BIGINT          NOT NULL DEFAULT 0,
  total_amount       BIGINT          NOT NULL DEFAULT 0,
  currency           CHAR(3)         NOT NULL DEFAULT 'TZS',
  status             VARCHAR(20)     NOT NULL DEFAULT 'draft',
  issued_at          DATETIME(3)     NULL,
  due_at             DATETIME(3)     NULL,
  paid_at            DATETIME(3)     NULL,
  share_token_hash   CHAR(64)        NULL,
  notes              TEXT            NULL,
  created_by         BIGINT UNSIGNED NULL,
  created_at         DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at         DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at         DATETIME(3)     NULL,
PRIMARY KEY (id),
UNIQUE KEY uq_invoices_number (invoice_number),
UNIQUE KEY uq_invoices_share_token (share_token_hash),
KEY idx_invoices_status (status, issued_at),
KEY idx_invoices_order (order_id),
CONSTRAINT fk_invoices_order FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE SET NULL,
CONSTRAINT fk_invoices_creator FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL,
CONSTRAINT chk_invoices_status CHECK (status IN ('draft', 'issued', 'paid', 'void'))
) ENGINE=InnoDB;

CREATE TABLE invoice_items (
  id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,

```



```

invoice_id          BIGINT UNSIGNED NOT NULL,
product_variant_id  BIGINT UNSIGNED NULL,
description          VARCHAR(255)    NOT NULL,
sku                 VARCHAR(20)      NULL,
unit_price_amount   BIGINT           NOT NULL,
quantity            INT              NOT NULL,
line_total_amount   BIGINT           NOT NULL,
position            INT              NOT NULL DEFAULT 0,
PRIMARY KEY (id),
KEY idx_invoice_items_invoice (invoice_id, position),
CONSTRAINT fk_ii_invoice FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE CASCADE,
CONSTRAINT fk_ii_variant FOREIGN KEY (product_variant_id) REFERENCES product_variants(id) ON DELETE SET
NULL,
CONSTRAINT chk_ii_quantity CHECK (quantity > 0)
) ENGINE=InnoDB;

```

```

-- -----
-- 7. CONTENT
-- -----

```

```

CREATE TABLE banners (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  title       VARCHAR(120)    NOT NULL,
  title_color CHAR(7)         NOT NULL DEFAULT 'FFFFFF',
  subtitle    VARCHAR(300)    NULL,
  subtitle_color CHAR(7)     NOT NULL DEFAULT 'FFFFFF',
  background_color CHAR(7)   NOT NULL DEFAULT '#0A5C43',
  image_id    BIGINT UNSIGNED NOT NULL,
  link_url    VARCHAR(512)    NULL,
  button_text VARCHAR(40)     NULL,
  button_background CHAR(7)  NOT NULL DEFAULT '#0E7C5A',
  button_text_color CHAR(7)  NOT NULL DEFAULT 'FFFFFF',
  placement   VARCHAR(20)     NOT NULL DEFAULT 'home',
  position    INT             NOT NULL DEFAULT 0,
  starts_at   DATETIME(3)     NULL,
  ends_at     DATETIME(3)     NULL,
  is_active   TINYINT(1)      NOT NULL DEFAULT 1,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at  DATETIME(3)     NULL,
  PRIMARY KEY (id),
  KEY idx_banners_placement (placement, is_active, position),
  CONSTRAINT fk_banners_image FOREIGN KEY (image_id) REFERENCES media_files(id) ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

CREATE TABLE services (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  heading     VARCHAR(160)    NOT NULL,
  slug        VARCHAR(96)     NOT NULL,
  excerpt     VARCHAR(500)    NULL,
  content_html MEDIUMTEXT     NOT NULL,
  content_text MEDIUMTEXT     NOT NULL,
  image_id    BIGINT UNSIGNED NULL,
  position    INT             NOT NULL DEFAULT 0,
  is_active   TINYINT(1)      NOT NULL DEFAULT 1,
  published_at DATETIME(3)     NULL,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at  DATETIME(3)     NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_services_slug (slug),
  CONSTRAINT fk_services_image FOREIGN KEY (image_id) REFERENCES media_files(id) ON DELETE SET NULL
) ENGINE=InnoDB;

```

```

CREATE TABLE partners (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name        VARCHAR(120)    NOT NULL,
  logo_id     BIGINT UNSIGNED NULL,

```



```

website_url VARCHAR(512)    NULL,
position     INT            NOT NULL DEFAULT 0,
is_active    TINYINT(1)     NOT NULL DEFAULT 1,
created_at   DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at   DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
deleted_at   DATETIME(3)    NULL,
PRIMARY KEY (id),
CONSTRAINT fk_partners_logo FOREIGN KEY (logo_id) REFERENCES media_files(id) ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE articles (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  title       VARCHAR(200)    NOT NULL,
  slug        VARCHAR(96)     NOT NULL,
  excerpt     VARCHAR(500)    NULL,
  content_html MEDIUMTEXT     NOT NULL,
  content_text MEDIUMTEXT     NOT NULL,
  cover_image_id BIGINT UNSIGNED NULL,
  author_id    BIGINT UNSIGNED NULL,
  status       VARCHAR(16)    NOT NULL DEFAULT 'draft',
  published_at DATETIME(3)    NULL,
  view_count   INT UNSIGNED   NOT NULL DEFAULT 0,
  comment_count INT UNSIGNED   NOT NULL DEFAULT 0,
  meta_title   VARCHAR(70)    NULL,
  meta_description VARCHAR(160) NULL,
  created_at   DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at   DATETIME(3)    NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_articles_slug (slug),
  KEY idx_articles_status_published (status, published_at),
  FULLTEXT KEY ft_articles_search (title, content_text),
  CONSTRAINT fk_articles_image FOREIGN KEY (cover_image_id) REFERENCES media_files(id) ON DELETE SET NULL,
  CONSTRAINT fk_articles_author FOREIGN KEY (author_id) REFERENCES users(id) ON DELETE SET NULL,
  CONSTRAINT chk_articles_status CHECK (status IN ('draft','published','archived'))
) ENGINE=InnoDB;

CREATE TABLE comments (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  article_id   BIGINT UNSIGNED NOT NULL,
  user_id      BIGINT UNSIGNED NULL,
  parent_id    BIGINT UNSIGNED NULL,
  author_name  VARCHAR(120)    NOT NULL,
  body         TEXT            NOT NULL,
  status       VARCHAR(16)    NOT NULL DEFAULT 'pending',
  moderated_by BIGINT UNSIGNED NULL,
  moderated_at DATETIME(3)    NULL,
  ip_address   VARBINARY(16)  NULL,
  created_at   DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3)    NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at   DATETIME(3)    NULL,
  PRIMARY KEY (id),
  KEY idx_comments_article (article_id, status, created_at),
  KEY idx_comments_status (status, created_at),
  CONSTRAINT fk_comments_article FOREIGN KEY (article_id) REFERENCES articles(id) ON DELETE CASCADE,
  CONSTRAINT fk_comments_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
  CONSTRAINT fk_comments_parent FOREIGN KEY (parent_id) REFERENCES comments(id) ON DELETE CASCADE,
  CONSTRAINT fk_comments_moderator FOREIGN KEY (moderated_by) REFERENCES users(id) ON DELETE SET NULL,
  CONSTRAINT chk_comments_status CHECK (status IN ('pending','approved','rejected','spam'))
) ENGINE=InnoDB;

CREATE TABLE reviews (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  product_id  BIGINT UNSIGNED NOT NULL,
  user_id      BIGINT UNSIGNED NULL,
  order_item_id BIGINT UNSIGNED NULL,
  author_name  VARCHAR(120)    NOT NULL,

```

```

rating          TINYINT          NOT NULL,
title           VARCHAR(80)       NULL,
body            TEXT              NOT NULL,
status          VARCHAR(16)       NOT NULL DEFAULT 'pending',
helpful_count   INT UNSIGNED      NOT NULL DEFAULT 0,
moderated_by    BIGINT UNSIGNED   NULL,
moderated_at    DATETIME(3)       NULL,
created_at      DATETIME(3)       NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at      DATETIME(3)       NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
deleted_at      DATETIME(3)       NULL,
PRIMARY KEY (id),
UNIQUE KEY uq_reviews_product_user (product_id, user_id),
KEY idx_reviews_product_status (product_id, status, created_at),
KEY idx_reviews_status (status, created_at),
CONSTRAINT fk_reviews_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
CONSTRAINT fk_reviews_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
CONSTRAINT fk_reviews_orderitem FOREIGN KEY (order_item_id) REFERENCES order_items(id) ON DELETE SET NULL,
CONSTRAINT fk_reviews_moderator FOREIGN KEY (moderated_by) REFERENCES users(id) ON DELETE SET NULL,
CONSTRAINT chk_reviews_rating CHECK (rating BETWEEN 1 AND 5),
CONSTRAINT chk_reviews_status CHECK (status IN ('pending','approved','rejected','spam'))
) ENGINE=InnoDB;

CREATE TABLE faqs (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  question    VARCHAR(300)    NOT NULL,
  answer_html TEXT            NOT NULL,
  `group`     VARCHAR(60)     NOT NULL DEFAULT 'General',
  position    INT             NOT NULL DEFAULT 0,
  is_active   TINYINT(1)      NOT NULL DEFAULT 1,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at  DATETIME(3)     NULL,
  PRIMARY KEY (id),
  KEY idx_faqs_group (`group`, position)
) ENGINE=InnoDB;

CREATE TABLE site_settings (
  `key`       VARCHAR(80)     NOT NULL,
  value       JSON            NOT NULL,
  `group`     VARCHAR(40)     NOT NULL DEFAULT 'general',
  description VARCHAR(255)    NULL,
  updated_by  BIGINT UNSIGNED NULL,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (`key`),
  KEY idx_settings_group (`group`),
  CONSTRAINT fk_settings_updater FOREIGN KEY (updated_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- -----
-- 8. MARKETING AND ANALYTICS
-- -----

CREATE TABLE contacts (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id     BIGINT UNSIGNED NULL,
  name        VARCHAR(120)    NOT NULL,
  email       VARCHAR(255)    NULL,
  phone       VARCHAR(15)     NULL,
  source      VARCHAR(24)     NOT NULL DEFAULT 'manual',
  email_opt_in TINYINT(1)     NOT NULL DEFAULT 0,
  sms_opt_in  TINYINT(1)     NOT NULL DEFAULT 0,
  unsubscribed_at DATETIME(3) NULL,
  tags        JSON            NULL,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at  DATETIME(3)     NULL,
  PRIMARY KEY (id),

```

```

    UNIQUE KEY uq_contacts_email (email),
    UNIQUE KEY uq_contacts_phone (phone),
    KEY idx_contacts_user (user_id),
    CONSTRAINT fk_contacts_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT chk_contacts_source CHECK (source IN
        ('registration','order','manual','import','contact_form'))
) ENGINE=InnoDB;

CREATE TABLE campaigns (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    channel VARCHAR(10) NOT NULL,
    name VARCHAR(160) NOT NULL,
    audience VARCHAR(16) NOT NULL DEFAULT 'all',
    audience_filter JSON NULL,
    body_text TEXT NOT NULL,
    body_html MEDIUMTEXT NULL,
    additional_recipients JSON NULL,
    scheduled_for DATETIME(3) NULL,
    status VARCHAR(16) NOT NULL DEFAULT 'draft',
    recipient_count INT NOT NULL DEFAULT 0,
    sent_count INT NOT NULL DEFAULT 0,
    delivered_count INT NOT NULL DEFAULT 0,
    failed_count INT NOT NULL DEFAULT 0,
    cost_amount BIGINT NULL,
    created_by BIGINT UNSIGNED NULL,
    sent_at DATETIME(3) NULL,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
    deleted_at DATETIME(3) NULL,
    PRIMARY KEY (id),
    KEY idx_campaigns_status (status, scheduled_for),
    CONSTRAINT fk_campaigns_creator FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT chk_campaigns_channel CHECK (channel IN ('sms','email')),
    CONSTRAINT chk_campaigns_audience CHECK (audience IN ('all','specific','tag','segment')),
    CONSTRAINT chk_campaigns_status CHECK (status IN
        ('draft','scheduled','sending','sent','failed','cancelled'))
) ENGINE=InnoDB;

CREATE TABLE campaign_recipients (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    campaign_id BIGINT UNSIGNED NOT NULL,
    contact_id BIGINT UNSIGNED NULL,
    destination VARCHAR(255) NOT NULL,
    status VARCHAR(16) NOT NULL DEFAULT 'queued',
    provider_message_id VARCHAR(120) NULL,
    error VARCHAR(255) NULL,
    sent_at DATETIME(3) NULL,
    delivered_at DATETIME(3) NULL,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),
    KEY idx_cr_campaign_status (campaign_id, status),
    CONSTRAINT fk_cr_campaign FOREIGN KEY (campaign_id) REFERENCES campaigns(id) ON DELETE CASCADE,
    CONSTRAINT fk_cr_contact FOREIGN KEY (contact_id) REFERENCES contacts(id) ON DELETE SET NULL,
    CONSTRAINT chk_cr_status CHECK (status IN
        ('queued','sent','delivered','failed','bounced','unsubscribed'))
) ENGINE=InnoDB;

CREATE TABLE search_logs (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    term VARCHAR(160) NOT NULL,
    normalised_term VARCHAR(160) NOT NULL,
    result_count INT NOT NULL DEFAULT 0,
    user_id BIGINT UNSIGNED NULL,
    session_id CHAR(32) NULL,
    clicked_product_id BIGINT UNSIGNED NULL,
    created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    PRIMARY KEY (id),

```

```

KEY idx_search_term (normalised_term, created_at),
KEY idx_search_zero (result_count, created_at),
CONSTRAINT fk_search_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
CONSTRAINT fk_search_product FOREIGN KEY (clicked_product_id) REFERENCES products(id) ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE contact_messages (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL,
  email VARCHAR(255) NOT NULL,
  subject VARCHAR(200) NULL,
  message TEXT NOT NULL,
  send_copy TINYINT(1) NOT NULL DEFAULT 0,
  status VARCHAR(16) NOT NULL DEFAULT 'new',
  handled_by BIGINT UNSIGNED NULL,
  ip_address VARBINARY(16) NULL,
  user_agent VARCHAR(255) NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at DATETIME(3) NULL,
  PRIMARY KEY (id),
  KEY idx_contact_messages_status (status, created_at),
  CONSTRAINT fk_cm_handler FOREIGN KEY (handled_by) REFERENCES users(id) ON DELETE SET NULL,
  CONSTRAINT chk_cm_status CHECK (status IN ('new', 'read', 'replied', 'archived', 'spam'))
) ENGINE=InnoDB;

CREATE TABLE notify_requests (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  product_variant_id BIGINT UNSIGNED NOT NULL,
  email VARCHAR(255) NOT NULL,
  user_id BIGINT UNSIGNED NULL,
  notified_at DATETIME(3) NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_notify_variant_email (product_variant_id, email),
  CONSTRAINT fk_nr_variant FOREIGN KEY (product_variant_id) REFERENCES product_variants(id) ON DELETE CASCADE,
  CONSTRAINT fk_nr_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- -----
-- 9. AUDIT (insert-only; grant the runtime user INSERT + SELECT only)
-- -----

CREATE TABLE audit_logs (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  actor_user_id BIGINT UNSIGNED NULL,
  actor_email VARCHAR(255) NULL,
  action VARCHAR(60) NOT NULL,
  entity_type VARCHAR(60) NOT NULL,
  entity_id BIGINT UNSIGNED NULL,
  entity_label VARCHAR(255) NULL,
  `before` JSON NULL,
  `after` JSON NULL,
  ip_address VARBINARY(16) NULL,
  user_agent VARCHAR(255) NULL,
  request_id CHAR(26) NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_audit_entity (entity_type, entity_id, created_at),
  KEY idx_audit_actor (actor_user_id, created_at),
  KEY idx_audit_action (action, created_at),
  CONSTRAINT fk_audit_actor FOREIGN KEY (actor_user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- -----
-- Full-text tuning (server variables - set in my.cnf, shown for reference)
-- innodb_ft_min_token_size = 2
-- innodb_ft_user_stopword_table = techstar/ft_stopwords (empty table)

```

```
-- -----  
CREATE TABLE ft_stopwords (value VARCHAR(30)) ENGINE=InnoDB;
```

Seed Data Plan

Order matters: SKU allocation reads the parent record, so a child can never be seeded before its parent. The seed is idempotent — running it twice produces the same database, never duplicates — which is what makes it safe to re-run against a developer's existing schema.

Step	Data	Contents and notes
1	Permissions	All ~60 permission keys from §11.5, grouped. Idempotent upsert on <code>key</code> .
2	Roles	owner, manager, catalogue, fulfilment, marketing, support — all flagged <code>is_system</code> , with their permission sets per the matrix in §11.5.
3	Owner account	One staff user from <code>SEED_OWNER_EMAIL</code> and <code>SEED_OWNER_PASSWORD</code> . In production the password must be changed at first sign-in; the seed refuses to run with a default password when <code>NODE_ENV=production</code> .
4	Regions	The 25 mainland regions: Dar es Salaam, Dodoma, Geita, Iringa, Kagera, Katavi, Kigoma, Kilimanjaro, Lindi, Manyara, Mara, Mbeya, Morogoro, Mtwara, Mwanza, Njombe, Pwani, Rukwa, Ruvuma, Shinyanga, Simiyu, Singida, Songwe, Tabora, Tanga. Zanzibar regions should be added if the store ships there — decide before launch, since it affects the delivery promise.
5	Districts	Per region. Dar es Salaam: Ilala, Ilala CBD, Kigamboni, Kinondoni, Temeke, Ubungu. The remainder from the official administrative list.
6	Site settings	Every key from §14.16 with sensible defaults and empty credentials. Payment till numbers and instruction blocks seeded as clearly marked placeholders so a missed configuration is visible rather than silent.
7	Currency	TZS, maximum fraction digits 0.
8	Shipping methods	Three, mirroring the proven set: Savings TZS 15,000 (address required, mobile payment yes) for upcountry; Cash on delivery TZS 5,000 (address required, mobile payment no, cash-on-delivery yes); Office pickup TZS 0 (no address, mobile payment yes).
9	Categories	The 17 categories from Appendix A with their two-digit codes.
10	Subcategories	The 102 subcategories from Appendix A with their four-digit codes.
11	FAQs	A starter set across Ordering, Payment, Shipping, Returns, Products and Account — placeholder answers clearly marked for replacement.
12	Demo content (<i>development only</i>)	30 products spread across subcategories, some with variants; 4 banners; 3 partners; 2 services; 3 articles with comments; 20 customers; 40 orders across every status; payments in each state; search logs including zero-result terms. Enough to exercise every screen and every empty state. Guarded by <code>SEED_DEMO=true</code> and refused in production.

Order Status Reference

A one-page operational reference for staff. Lifecycle, payment and fulfilment are independent (§12.7).

Lifecycle	Customer sees	Stock effect	Next steps available
awaiting_payment	"Awaiting payment"	Reserved	Verify payment → confirmed · Cancel · Expires at 48 h
confirmed	"Confirmed"	Deducted (prepaid) or still reserved (cash on delivery)	Start processing · Cancel (staff)
processing	"Being prepared"	Deducted	Mark shipped · Cancel (staff)
shipped	"On the way"	Deducted	Mark delivered
delivered	"Delivered"	Deducted	Auto-completes after 7 days · Refund
completed	"Completed"	Deducted	Terminal. Review invitation sent.
cancelled	"Cancelled"	Released / restored	Terminal
expired	"Expired"	Released	Terminal. Reorder link sent.
refunded	"Refunded"	Restored only on physical return	Terminal

Payment status	Meaning
unpaid	Nothing received. Normal for a new order and for cash on delivery until handover.
pending_verification	The customer says they have paid. Sitting in the verification queue. This is the state that needs a human.
paid	Money confirmed against a provider reference.
partially_refunded / refunded	Some or all returned.
failed	An aggregator declined the transaction. The customer may retry.

Alternative Stack: PHP / Laravel

Decision D1 selected Node.js. If PHP is preferred — and the workstation's existing XAMPP install is a fair reason to prefer it — nothing in Parts 5 to 18 changes. The data model, API contract, business rules, UI specification and admin design are all runtime-neutral. Only the server implementation is affected.

Node.js design	Laravel 11 equivalent
Express router + controllers	<code>routes/api.php</code> with API resource controllers
Zod schemas in <code>packages/contracts</code>	Form Requests for validation; the schemas can no longer be shared with React, so the front end keeps its own Zod copies and a contract test asserts they agree — this is the single real cost of the switch
Prisma	Eloquent with migrations; the DDL in Appendix E translates directly
Service layer	Action classes (one public method each)
<code>domain/</code> pure logic	Plain PHP classes under <code>app/Domain</code> , no framework dependency
JWT + refresh rotation	Sanctum with a custom refresh-token table, keeping the rotation and reuse-detection design of §11.2 — Sanctum's default is weaker
BullMQ + Redis	Laravel Queues on Redis; Horizon replaces the failed-job admin view
Scheduled jobs	Laravel Scheduler
Socket.IO	Laravel Reverb, or Pusher
sharp	Intervention Image with the Imagick driver
Nodemailer + React Email	Laravel Mail with Blade templates
pino	Monolog with a JSON formatter
Vitest / Supertest	Pest or PHPUnit with feature tests
PM2 cluster	PHP-FPM behind Nginx

Points to preserve regardless of runtime: money as integers; the order state machine as a single guarded transition table; row locking in a consistent order for stock; unique constraints on payment references; the insert-only audit table enforced by database privileges; and the token strategy of §11.2, which Laravel's defaults do not give you.

Reference Screenshots

Captured from the reference platform at 1440 px, used to derive the layouts in Part 7. Full-resolution originals, plus mobile captures and the complete rendered DOM outlines, are in [research/screenshots/](#) and [research/ui/](#).

+255 (0) 746 624 875

info@bafredo.co.tz

Dar es salaam, Tanzania

BAFREDO

Electronics

Imagination. Creativity. Innovation.

Please search here ...

Search

0

TZS 0

0

Login | Register

PRODUCTS SHOP SUPPORT SERVICES

Getting your order

Returning Customer? [Click Here to Login](#)

Do not have a Bafredo account? [Click here to Create an account](#)

CLIENT DETAILS

Username

Phone

Username

255XXXXXXXXX

SHIPPING METHOD

☐ Savings

☐ Cash on delivery

☐ Office Pickup

ESTIMATED COST

TZS 15,000

TZS 5,000

TZS 0

Your Order

PLACE ORDER

Sub Total:

TZS 0

Shipping Cost:

TZS 0

TOTAL

TZS 0

COMPANY INFORMATION

BAFREDO Electronics Limited

P. O. Box 33761

Dar es Salaam, Tanzania

+255 (0) 746 624 875

+255-22-246 0157

info@bafredo.co.tz

COMPANY INFORMATION

Country of registration : Tanzania

Registration Agency : BRELLA

TIN NUMBER : 135-644-951

Certificate of incorporation 120435

Dar es Salaam, Tanzania

+255 (0) 746 624 875

+255-22-246 0157

info@bafredo.co.tz

POLICIES

Privacy policy

Terms & Conditions

SUPPORT

FAQ's

About Us

FOLLOW US

f

yt

in

ig

tw

Copyright 2026 BAFREDO Electronics Limited. All Rights Reserved.

Checkout. The single-page design — identity, shipping method with estimated cost, and a sticky order summary whose PLACE ORDER button sits above the totals — is the reference platform's strongest screen and is kept almost unchanged in §7.7. What is added: visible method descriptions, server-side price and stock revalidation, idempotency, and the delegated-payer disclosure.